The Transport Layer

Transport Services, Primitives, Sockets & Protocols

Based on Computer Networks, Fifth Edition by A.S. Tanenbaum and D. J. Wetherall and TCP/IP Protocol Suite by B. Forouzan

Disclaimer

Compilation of Sources: These slides compile content from various academic resources, standard networking textbooks (primarily Computer Networks by Andrew S. Tanenbaum & Wetherall), and instructional materials.

AI-Assisted Content Generation: Certain diagrams, visual aids, and conceptual illustrations in these slides were generated, refined, or structured with the help of AI tools (such as Gemini, ChatGPT, and Claude) for educational clarity.

Academic Purpose & Permissions: These slides are prepared strictly for educational, non-commercial teaching purposes. Academic use, sharing, and distribution for classroom instruction are permitted, provided proper attribution to the original author and source materials is maintained.

Chapter Outline



1. Transport Service

Upper layer services, primitives, Berkeley Sockets, and client-server socket programming.



2. Protocol Elements

Addressing, connection establishment, connection release, flow control, and crash recovery.



3. Internet Protocols

Detailed analysis of UDP, TCP state machine, sliding windows, and congestion control.

Role of the Transport Layer

The Transport Layer provides **end-to-end data transport** from the source host to the destination host, independently of the physical mesh of routers.

- **Quality of Service:** Enhances network layer QoS by making unreliable networks appear error-free.
- **Process-to-Process:** Moves messages between application processes, not just network interfaces.
- **End-to-End Boundary:** Transport protocols run entirely on user machines (hosts), not routers.

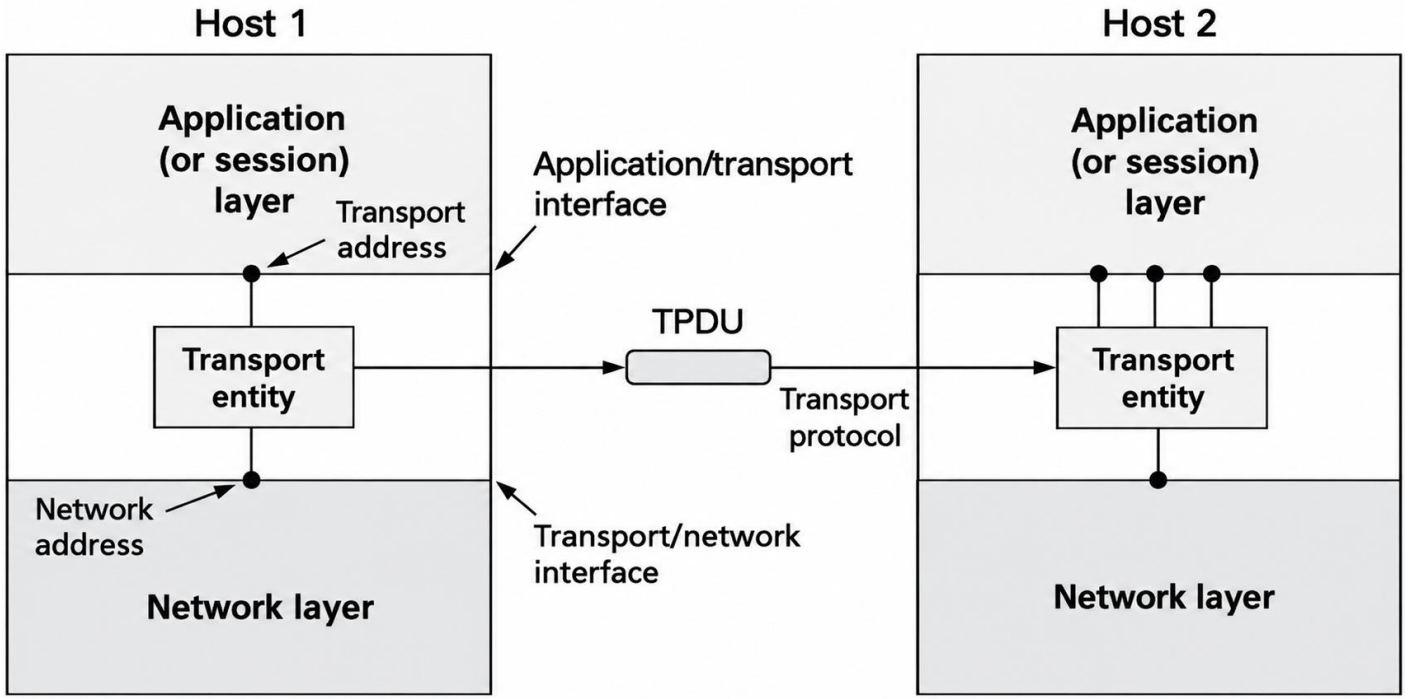
> **Key Architecture Distinction:** Layers 1 to 3 run on intermediate routers and subnet nodes. Layer 4 (Transport) is the lowest *true end-to-end layer*, running exclusively on edge hosts.

Services Provided to the Upper Layers

The transport layer acts as the bridge between application processes and the network infrastructure.

The transport code uses network service primitives to send and receive **TPDUs (Transport Protocol Data Units)**.

- **Application Interface:** Exposes TSAPs (Transport Service Access Points).
- **Peer Communication:** Transport entities communicate across the network using TPDUs.



Transport Provider vs. Transport User

Transport Service Provider

Layers 1 through 4 form the complete **Transport Service Provider**.

It shields the upper layers from network hardware details, routing complexities, packet drops, and topological shifts.

Transport Service User

Layers 5 and above (Session, Presentation, Application) represent the **Transport Service User**.

Application programmers write software assuming a clean, reliable pipe connecting user processes.

The Pipe Abstraction: Like a UNIX pipe, a reliable transport connection guarantees that bytes inserted at one end emerge in the exact same order at the other end with zero loss.

Transport Service vs. Network Service

Why do we need a Transport Layer when the Network Layer already provides packet delivery?

Attribute	Network Layer Service	Transport Layer Service
Target Audience	Internal router-to-router communication.	Application developers & end processes.
Reliability	Often lossy, connectionless (IP datagrams).	Reliable, error-free byte streams (TCP).
Control Location	Managed by ISPs & carrier routers.	Fully controlled by user software / OS kernel.
Addressing	IP addresses (Host to Host).	Ports / TSAPs (Process to Process).

Transport Service Primitives

Applications access transport services through an API composed of transport service primitives.

Abstraction Layer: Allows programmers to establish, use, and tear down connections without writing custom network drivers.

Dual Service Types:

- *Connection-Oriented:* Reliable stream (e.g., TCP-style primitives).
- *Connectionless:* Unreliable datagrams (e.g., UDP-style primitives).

Imperfection Hiding: The transport entity handles retransmissions, acknowledgments, and buffer overflows behind the scenes thus hiding the imperfections of the network layer.

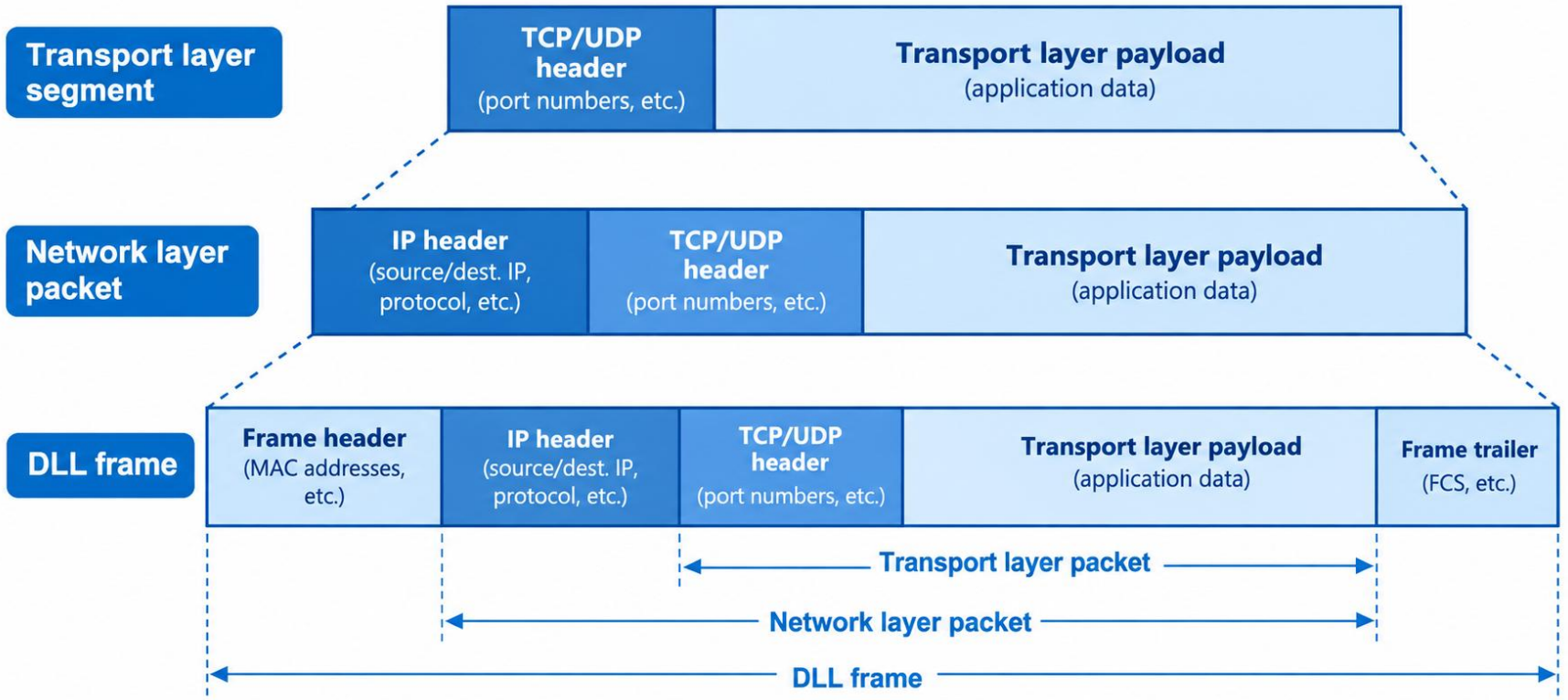
A Simple Transport Service Interface

An abstract set of 5 basic primitives illustrating connection-oriented transport operations:

Primitive	Packet Sent	Meaning / Operation
LISTEN	(None)	Block until a remote process attempts to connect.
CONNECT	CONNECTION REQ.	Actively attempt to establish a transport connection.
SEND	DATA	Transmit data over an established connection.
RECEIVE	(None)	Block until a DATA packet arrives.
DISCONNECT	DISCONNECTION REQ.	Initiate graceful connection termination.

TPDU, Packet, and Frame Nesting

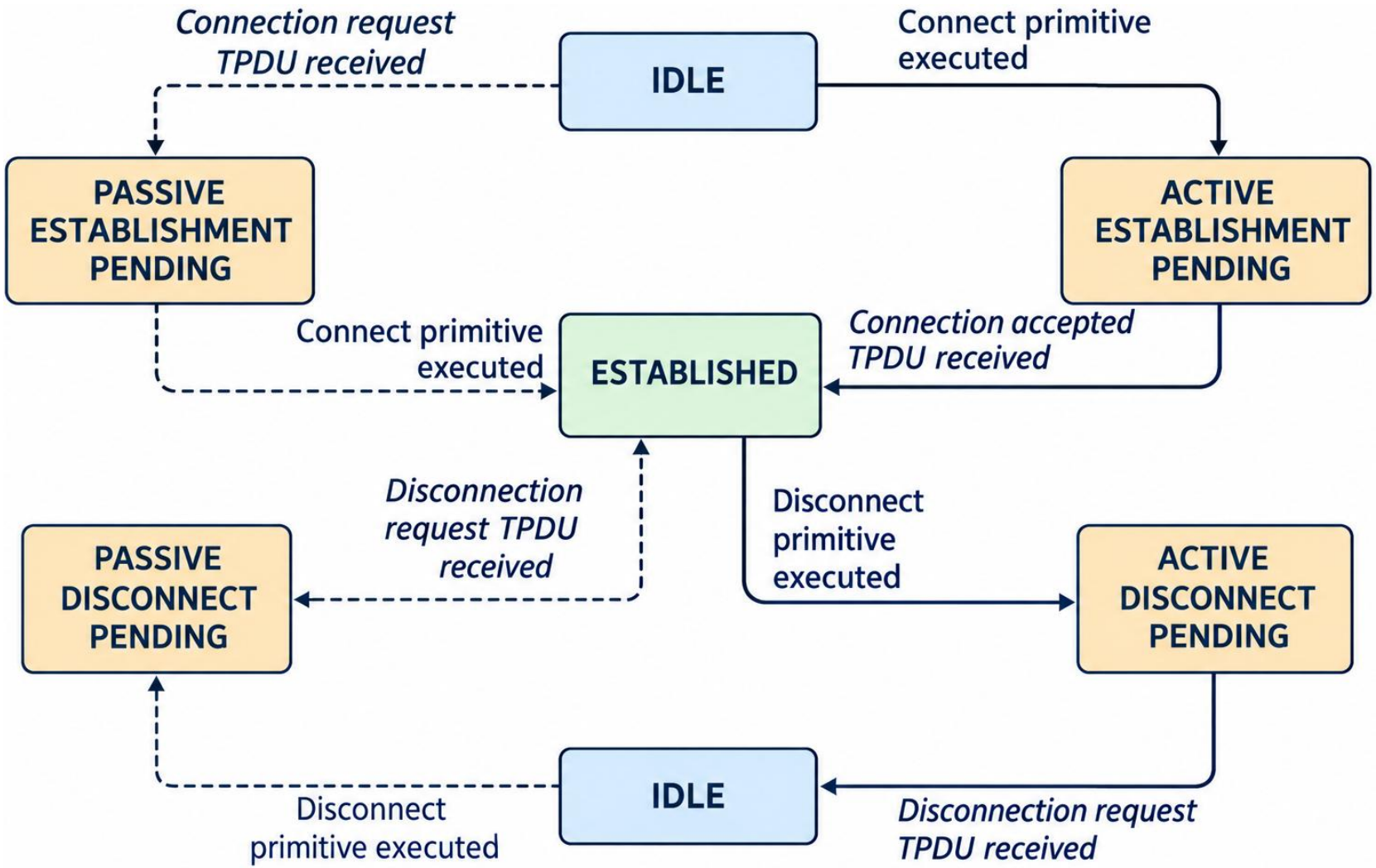
Data units are encapsulated hierarchically as they move down the network protocol stack.



Overhead Hierarchy: Each layer appends its own control metadata (headers). Routers inspect the Network (Packet) header, switches inspect the Data Link (Frame) header, and only the destination host unwraps the TPDU header.

Simple Connection State Diagram

Connection management transition sequence for simple transport primitives:



Simple Connection State Diagram

Connection management transition sequence for simple transport primitives:

Server Path (Passive)

1. Start in `IDLE`
2. **Execute** `LISTEN`. → Enter `PASSIVE ESTABLISHMENT PENDING`.
3. Receive CR → Enter `ESTABLISHED`
4. Receive DR → Enter `PASSIVE DISCONNECT PENDING` `IDLE`

Client Path (Active)

1. Start in `IDLE`
2. **Execute** `CONNECT` → Enter `ACTIVE ESTABLISHMENT PENDING`.
3. Receive ACK → Enter `ESTABLISHED`
4. **Execute** `DISCONNECT` → Enter `ACTIVE DISCONNECT PENDING` → `IDLE`.

The Berkeley Sockets API

Developed in 1981 for BSD UNIX, Berkeley Sockets became the de facto standard interprocess communication interface for TCP/IP.

- **File Abstraction:** Operates using UNIX file descriptor concepts (`read` and `write`).
- **Universal Support:** Supported on UNIX, Linux, Windows (Winsock), macOS, and embedded OSs.
- **Protocol Agnostic:** Supports IPv4, IPv6, UNIX domain sockets, and raw frames.

> **Socket Endpoints:** A socket is an endpoint for communication defined by an **IP Address** and a 16-bit **Port Number** (e.g., `192.168.1.10:80`).

Berkeley Socket Primitives for TCP

The standard C API primitives used for connection-oriented TCP socket programming:

Primitive	Executed By	Meaning / Purpose
SOCKET	Both	Create a new communication endpoint and allocate OS resources.
BIND	Server	Associate a local IP address and port number with a socket.
LISTEN	Server	Announce willingness to accept connections; define queue size.
ACCEPT	Server	Block passively waiting for an incoming connection request.
CONNECT	Client	Actively attempt to establish a connection with a remote socket.
SEND / RECEIVE	Both	Transmit or receive data over the established connection.
CLOSE	Both	Release the connection and free socket resources.

Server Socket Lifecycle

The sequential order of primitive execution on the server side:

1. `SOCKET` : Allocates a file descriptor and table space in the OS transport entity.
2. `BIND` : Attaches the newly created socket to a specific well-known port (e.g., Port 80 for HTTP).
3. `LISTEN` : Allocates queue space for pending client connections (backlog queue).
4. `ACCEPT` : Blocks until a client arrives. On success, returns a *new socket descriptor* dedicated to that specific connection.

Example: Internet File Server

A practical C client-server application demonstrating socket primitives across a network.

Client Workflow

1. Lookup server IP via `gethostbyname()`
2. Create socket via `socket()`
3. Connect to server via `connect()`
4. Send requested filename over connection.
5. Read incoming file contents and print to stdout.

Server Workflow

1. Create, bind, and listen on port 12345.
2. Block in infinite loop at `accept()`
3. Upon connection, read filename from socket.
4. Open file on local disk & stream bytes back.
5. Close connection and await next client.

Client Implementation: Part 1

Header includes, constant definitions, variable declarations and Address lookup for the file client:

```
/* This page contains a client program that can request a file from the server program
 * on the next page. The server responds by sending the whole file.
 */

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345              /* arbitrary, but client & server must agree */
#define BUF_SIZE 4096                  /* block transfer size */

int main(int argc, char **argv)
{
  int c, s, bytes;
  char buf[BUF_SIZE];                  /* buffer for incoming file */
  struct hostent *h;                   /* info about server */
  struct sockaddr_in channel;          /* holds IP address */


if (argc != 3) fatal("Usage: client server-name file-name");
h = gethostbyname(argv[1]);                  /* look up host's IP address */
if (!h) fatal("gethostbyname failed");
```

Client Implementation: Part 2

Socket initialization, active connection establishment, sending the file request, reading streamed data, and terminal error handling:

```
 s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
 if (s <0) fatal("socket");
 memset(&channel, 0, sizeof(channel));
 channel.sin_family= AF_INET;
 memcpy(&channel.sin_addr.s_addr, h->h_addr, h->h_length);
 channel.sin_port= htons(SERVER_PORT);
 c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
 if (c < 0) fatal("connect failed");

 /* Connection is now established. Send file name including 0 byte at end. */
 write(s, argv[2], strlen(argv[2])+1);

 /* Go get the file and write it to standard output. */
 while (1) {
      bytes = read(s, buf, BUF_SIZE);          /* read from socket */
      if (bytes <= 0) exit(0);                 /* check for end of file */
      write(1, buf, bytes);                    /* write to standard output */
 }
}

fatal(char *string)
{
  printf("%s\n", string);
  exit(1);
}
```

Server Implementation: Part 1

Server-side header includes, socket constants, and buffer allocations:

```
#include <sys/types.h>                  /* This is the server code */
#include <sys/fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345               /* arbitrary, but client & server must agree */
#define BUF_SIZE 4096                   /* block transfer size */
#define QUEUE_SIZE 10

int main(int argc, char *argv[])
{
  int s, b, l, fd, sa, bytes, on = 1;
  char buf[BUF_SIZE];                   /* buffer for outgoing file */
  struct sockaddr_in channel;           /* holds IP address */

/* Build address structure to bind to socket. */
memset(&channel, 0, sizeof(channel));       /* zero channel */
channel.sin_family = AF_INET;
channel.sin_addr.s_addr = htonl(INADDR_ANY);
channel.sin_port = htons(SERVER_PORT);

/* Passive open. Wait for connection. */
s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);    /* create socket */
if (s < 0) fatal("socket failed");
setsockopt(s, SOL_SOCKET, SO_REUSEADDR, (char *) &on, sizeof(on));
```

Server Implementation: Part 2

Address binding, setting socket options, and initializing passive listening:

```
b = bind(s, (struct sockaddr *) &channel, sizeof(channel));
if (b < 0) fatal("bind failed");

l = listen(s, QUEUE_SIZE);                  /* specify queue size */
if (l < 0) fatal("listen failed");
```

The main service loop: accepting clients, reading filenames, and streaming files:

```
/* Socket is now set up and bound. Wait for connection and process it. */
while (1) {
    sa = accept(s, 0, 0);                   /* block for connection request */
    if (sa < 0) fatal("accept failed");

    read(sa, buf, BUF_SIZE);                /* read file name from socket */

    /* Get and return the file. */
    fd = open(buf, O_RDONLY);               /* open the file to be sent back */
    if (fd < 0) fatal("open failed");

    while (1) {
        bytes = read(fd, buf, BUF_SIZE); /* read from file */
        if (bytes <= 0) break;              /* check for end of file */
        write(sa, buf, bytes);              /* write bytes to socket */
    }
    close(fd);                              /* close file */
    close(sa);                              /* close connection */
  }
}
```

Elements of Transport Protocols

Although Transport protocols resemble Data Link protocols, the environment is far more hostile and complex.

Addressing

Dynamic process location via TSAPs & ports.

Connection Handling

Dealing with delayed duplicates over a lossy subnet.

Flow & Error Control

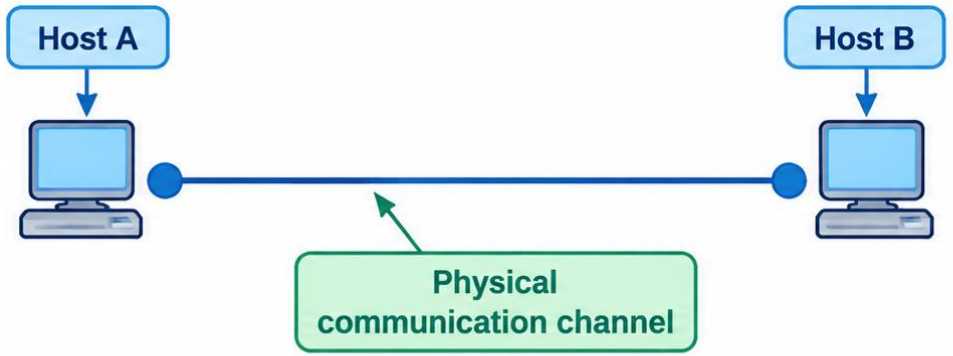
Dynamic buffering across competing streams.

Multiplexing

Sharing network connections efficiently.

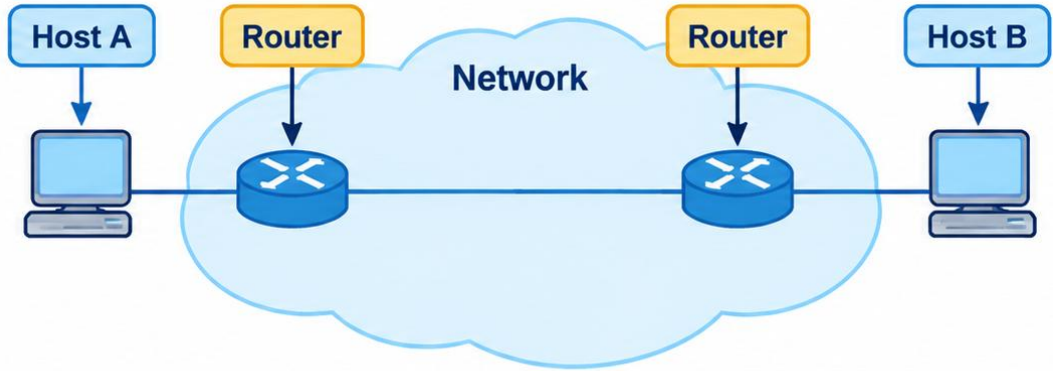
Data Link vs. Transport Environment

Key environment differences between Data Link and Transport protocols:



(a)

DLL protocols work in this environment
(point-to-point link)



(b)

Transport protocol works in this environment
(end-to-end across a network)

Issue	Data Link Layer	Transport Layer
Topology	Direct physical point-to-point wire.	Complex subnet with variable paths & storage.
Addressing	Simple (neighbor at other end of wire).	Explicit addressing required (IP + Port).
Packet Delivery	In-order, strictly deterministic.	Out-of-order, delayed, and duplicated packets.
Buffering	Fixed allocation per physical link.	Dynamic allocation among thousands of flows.

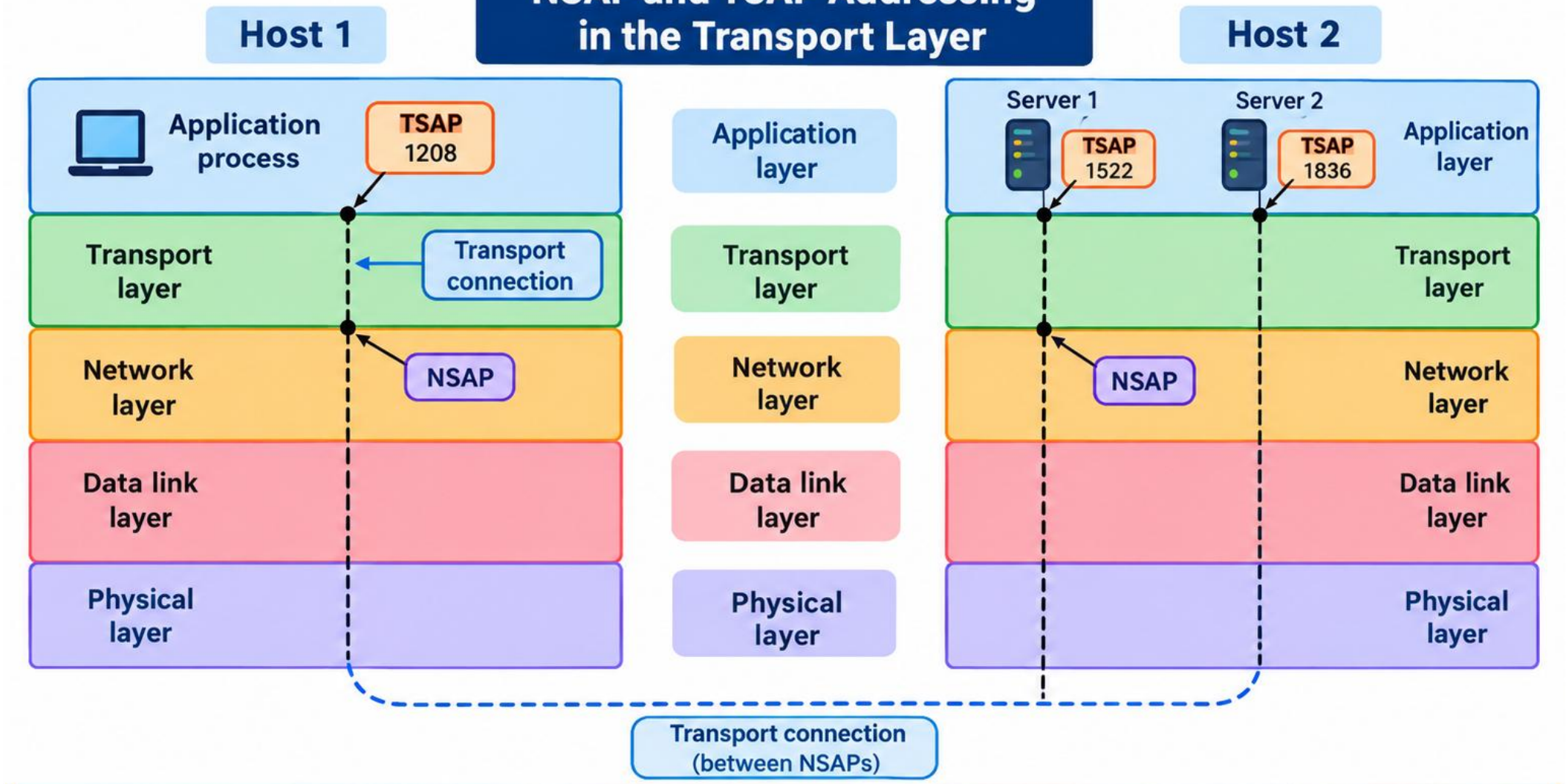
Addressing: TSAPs and NSAPs

To establish connections, transport entities need distinct service access points.

- **NSAP (Network Service Access Point):** IP Address identifying the host interface.
- **TSAP (Transport Service Access Point):** Port Number identifying the target process on the host.

> **Analogy:** The NSAP is like the street address of an apartment building (Host IP). The TSAP is the specific apartment unit number (Port) where a resident (Application Process) lives.

NSAP and TSAP Addressing in the Transport Layer



TSAP — Transport Service Access Point
- Identifies the application process (port)
- Example: 1208, 1522, 1836

NSAP — Network Service Access Point
- Identifies the host on the network
- Example: IP address or network address

How does the host 1 know that the mail server is attached to TSAP 1522?

Locating Target TSAPs / Ports

How does a client discover which port a specific server process is listening on?

1. Well-Known Ports

Fix standard services to universally recognized port numbers (e.g., HTTP on 80, SSH on 22, SMTP on 25).

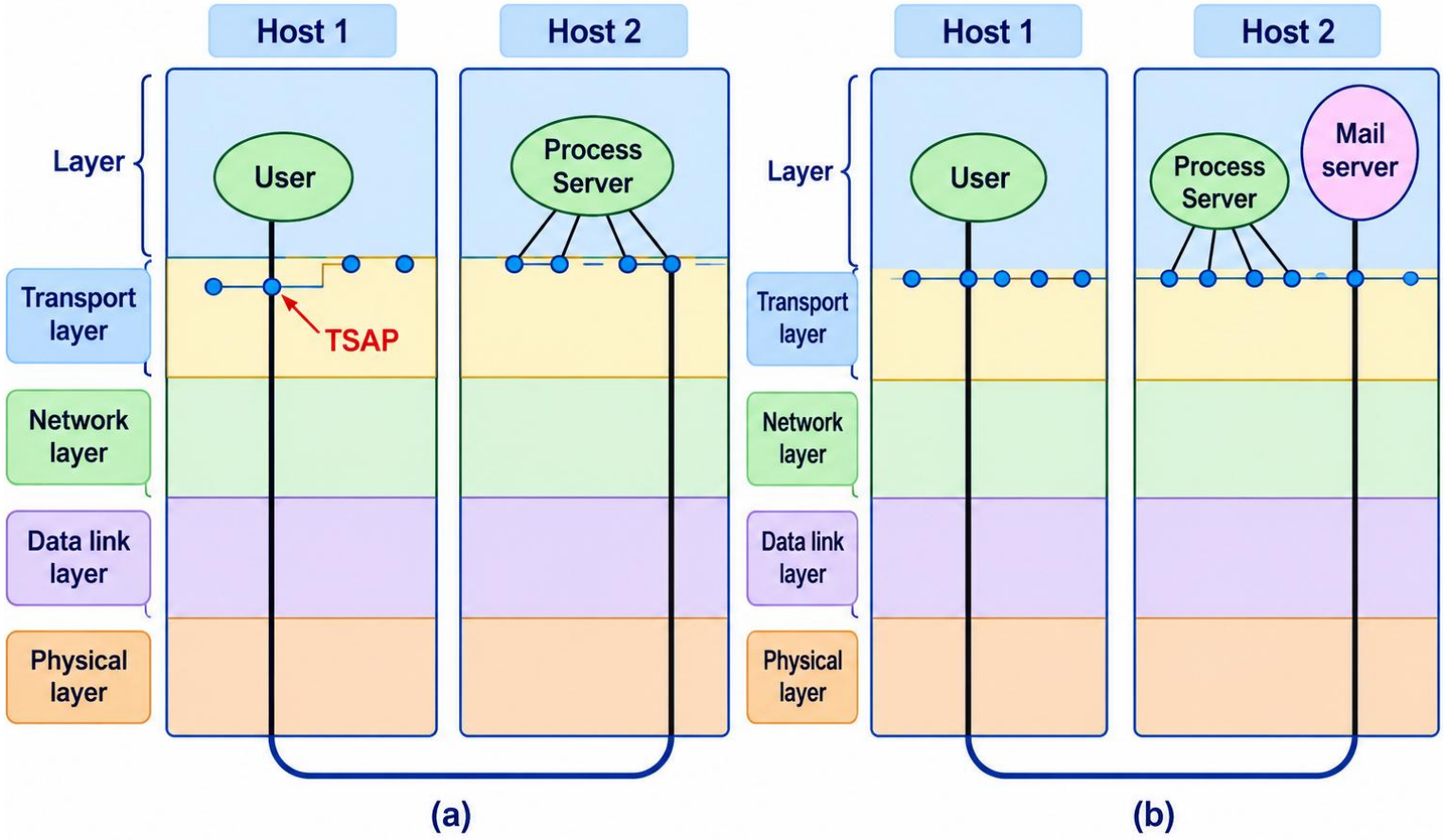
2. Portmapper / Name Server

Client sends a query to a central directory daemon specifying the service name; Portmapper replies with the dynamic TSAP.

The Process Server Model

Running hundreds of specialized server daemons permanently wastes CPU and memory resources.

- **Process Server (`inetd` / `systemd`):** A single super-daemon listens to a wide range of unused ports simultaneously.
- **On-Demand Spawning:** When a request arrives on a monitored port, `inetd` forks the corresponding specific server process (e.g., mail server) and hands over the connection socket.



Establishing Transport Connections

- Establishing connections over an unreliable network is deceptively difficult.
- If the network is fully congested, neither data or ack are delivered, leading to multiple retransmissions.
- If the user crashes and starts using the same set of sequences used earlier, it might lead to a duplicate packet delivery problem
- Financial Threat Example: In a bank transfer system, an old duplicate request could trigger duplicate money transfers if accepted blindly!
- Possible solutions to this problem can be –
  - Discard the port numbers used before the crash
  - Assign each connection a unique number
- A unique connection/sequence number requires some historical information to be maintained. If a machine crashes, it will not know which connection numbers were already used
- Rather, use mechanisms to kill off old packets

Restricting Packet Lifetime

To eliminate duplicate packet chaos, we enforce a maximum packet lifetime (T) typically 120 seconds).

Restricted Topology

Design subnets to prevent long loops or endless routing cycles.

Hop Counter (TTL)

Decrement a counter at every router; drop when it reaches zero (IPv4 TTL / IPv6 Hop Limit).

Time Stamping

Embed transmission time in packet and discard if elapsed time exceeds threshold.

Clock-Driven Sequence Numbers

Tomlinson's solution: Each host maintains a time-of-day clock to select initial sequence numbers.

- **Continuous Clock:** The clock increments steadily even if the machine crashes or reboots.
- **Initial Sequence Number (ISN):** When a connection opens, the low-order k bits of the master clock serve as the starting sequence number.
- **Recycling Protection:** Sequence numbers wrap around so slowly that old packets with identical numbers have expired long ago.

Using one of the above schemes, let T be the time for a packet and all of its acknowledgments to become dead

The Forbidden Region

- The idea is to ensure sequence numbers are not reused within T seconds
- The interval T and the packet rate will decide the size of the sequence number space
- Only one packet with a given sequence number may be outstanding at any given time.
- A delayed duplicate of an old packet should not beat a new packet with the same sequence number and be accepted by the destination
- One approach is to remain idle for T seconds after a crash to ensure all old packets are killed
- Other is to use time-of-day clock for sequence number
- To prevent sequence number confusion, segment transmission curves must not enter the **Forbidden Region**.

Sequence Space Constraints

- **Forbidden Region Boundaries:** Represents sequence numbers that are currently active or recently expired within time window T.
- **Data Rate Constraint:** If a host transmits data too fast, its sequence curve rises steeper than the clock curve, risking entering the forbidden region!
- **Crash Restart Rule:** Upon crashing, a host must wait T seconds before opening new connections to purge lingering TPDUs.

Sequence Space Size

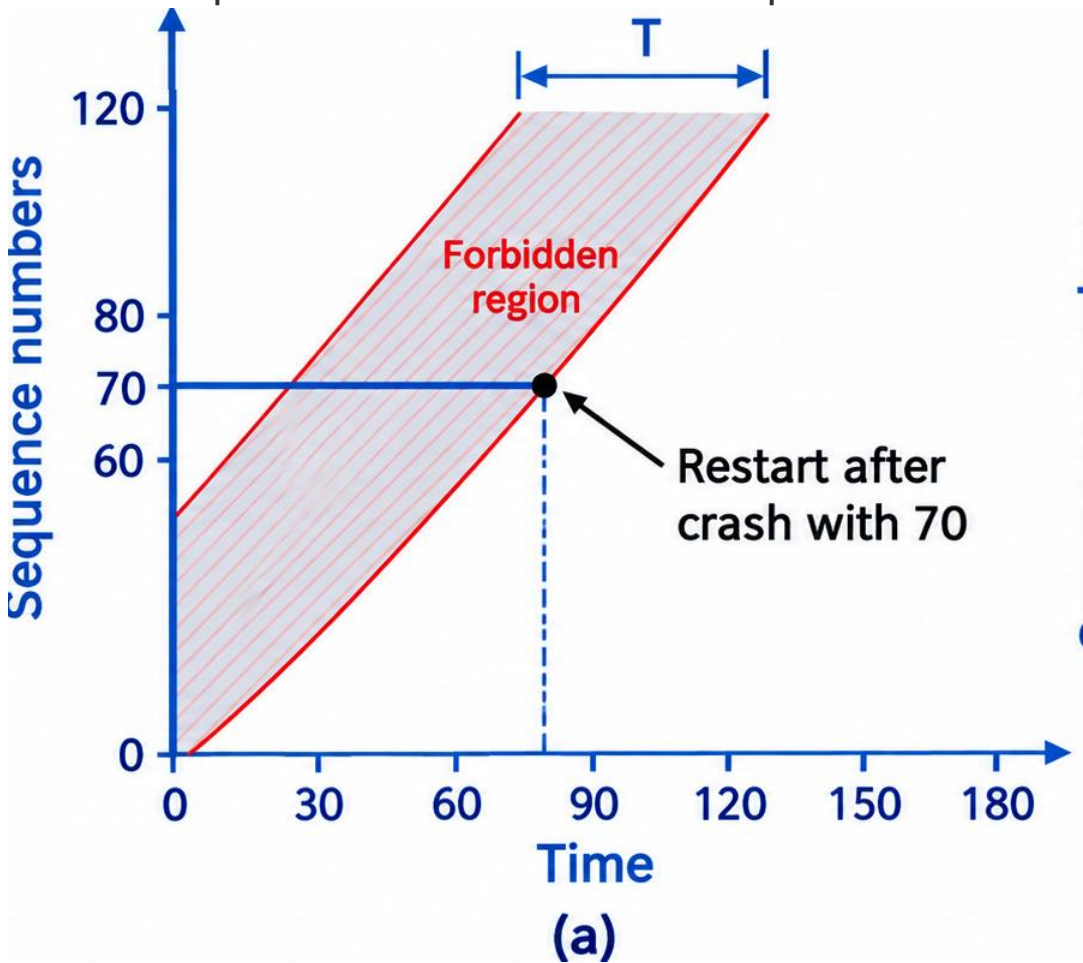
The total sequence space 2^k must be sufficiently large that by the time sequence numbers wrap around, any old packet with the same sequence number is guaranteed dead.

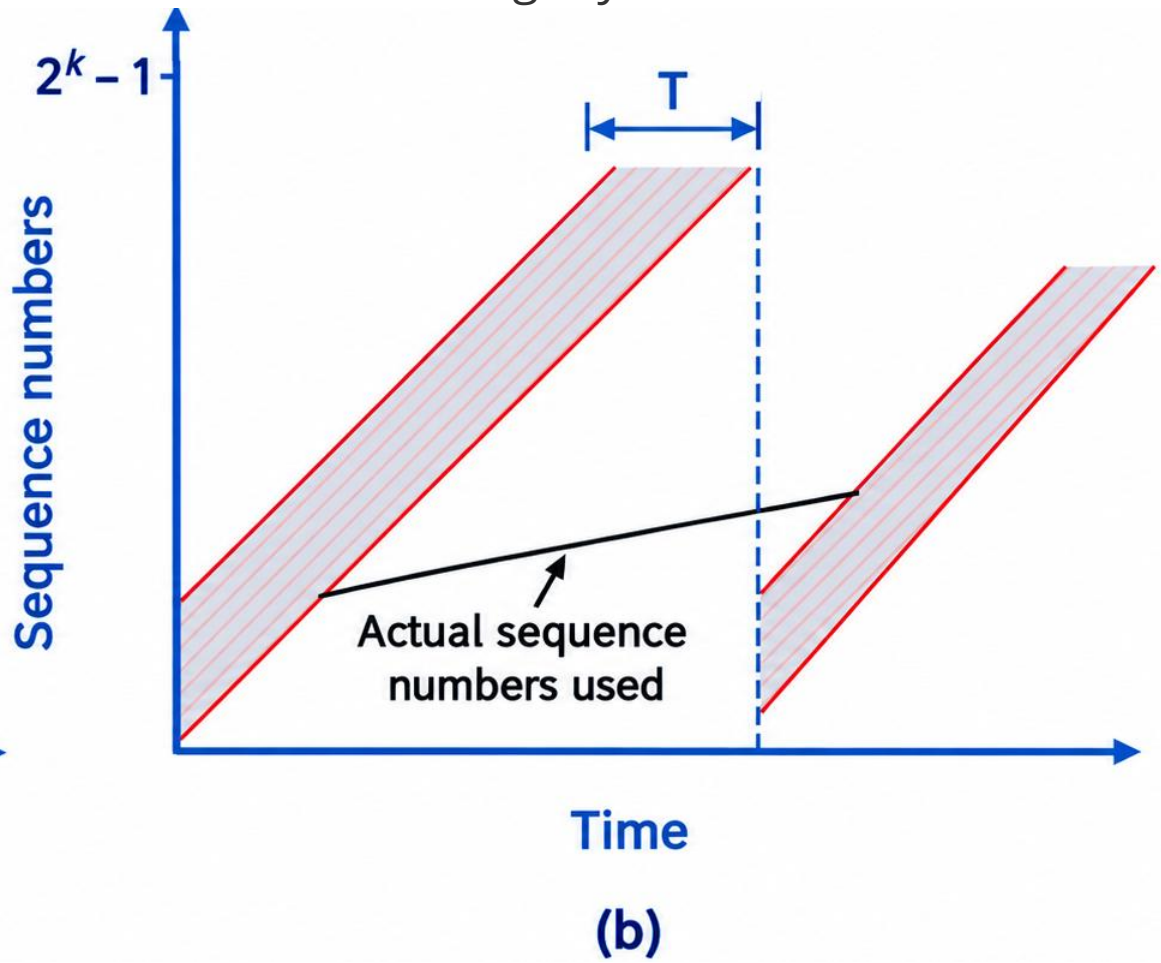
Maximum Data Rate

The maximum transmission speed over a connection is mathematically bounded by the requirement to avoid wrapping sequence numbers within maximum packet lifetime T.

The Forbidden Region

- Forbidden region shows the times for which segment sequence numbers are illegal
- If any segment is sent with a sequence number in this region, it could be delayed and impersonate a different packet with the same sequence number that will be issued slightly later





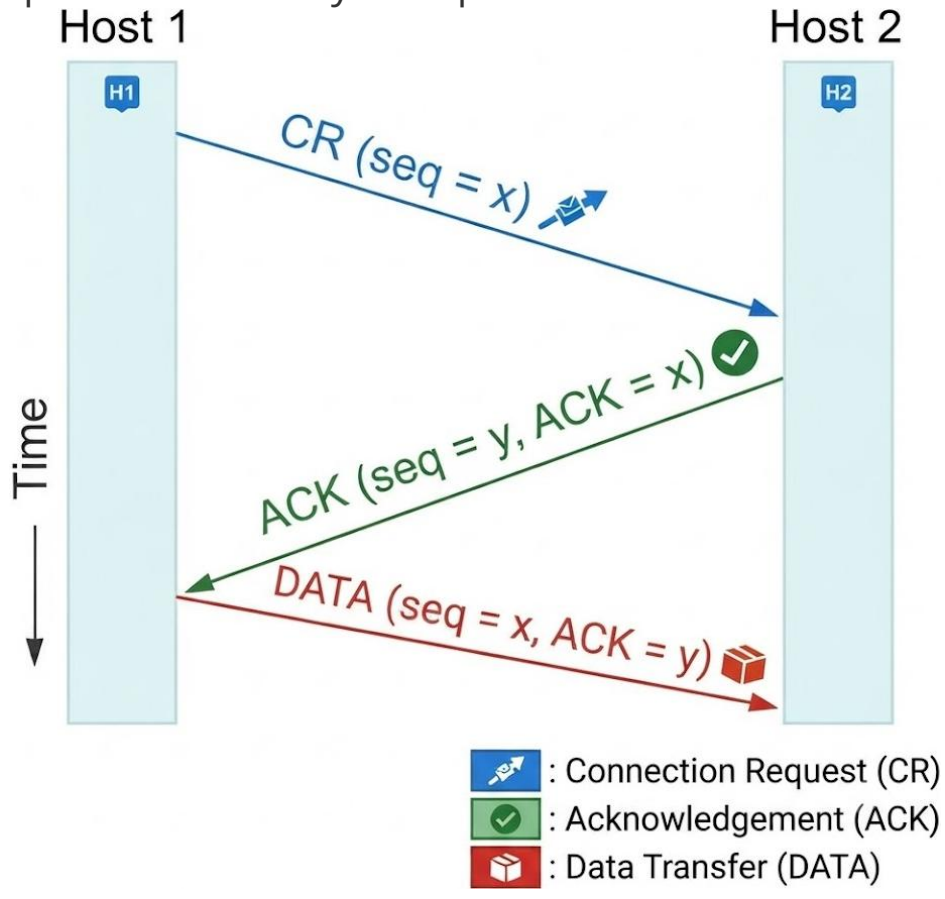
The Forbidden Region

- Each connection starts numbering its segments with a different initial sequence number
- The sequence space should be so large that by the time sequence numbers wrap around, old segments with the same sequence number are long gone.
- Linear relationship of seq no with time is shown in part (a) of figure on prev slide.
- If a host sends too much data too fast on a newly opened connection, the actual sequence number versus time curve may rise more steeply than the initial sequence number versus time curve, causing the sequence number to enter the forbidden region
- Any data rate less than the clock rate, the curve of actual sequence numbers used versus time will eventually run into the forbidden region

Three-Way Handshake: Normal Case

The **Three-Way Handshake** protocol reliably establishes connections even in the presence of delayed duplicates.

1. **Host 1 → Host 2:** Sends `CR (seq = x)`
2. **Host 2 → Host 1:** Acknowledges with `ACK (seq = y, ACK = x)`
3. **Host 1 → Host 2:** Final validation with `DATA (seq = x, ACK = y)`

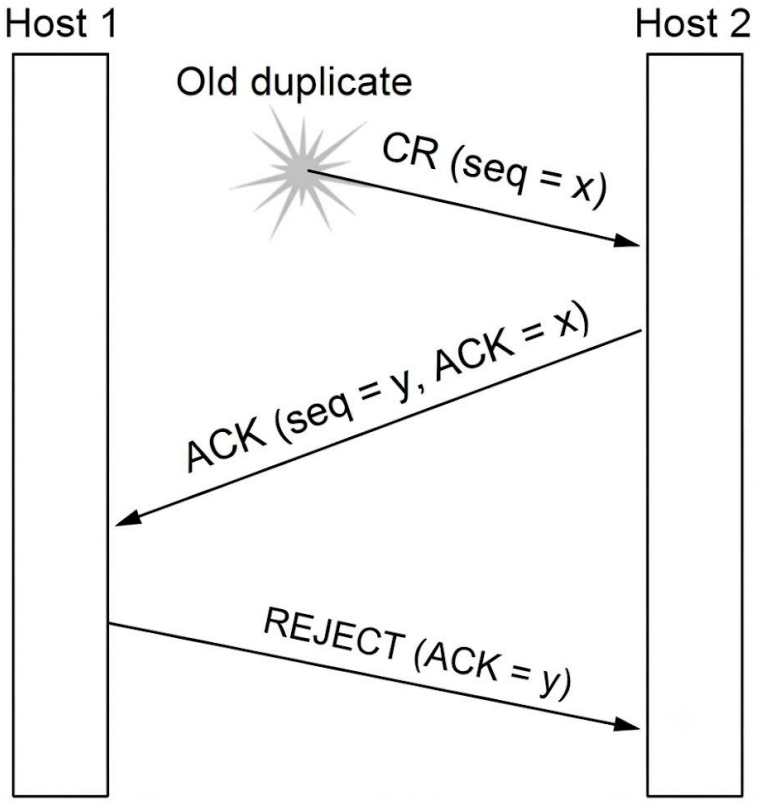


Mutual Verification: Each side chooses its own Initial Sequence Number (x and y) and explicitly verifies that the remote peer acknowledges its specific sequence number.

Handling Duplicate Connection Requests

How the Three-Way Handshake detects and rejects an old duplicate `CR`

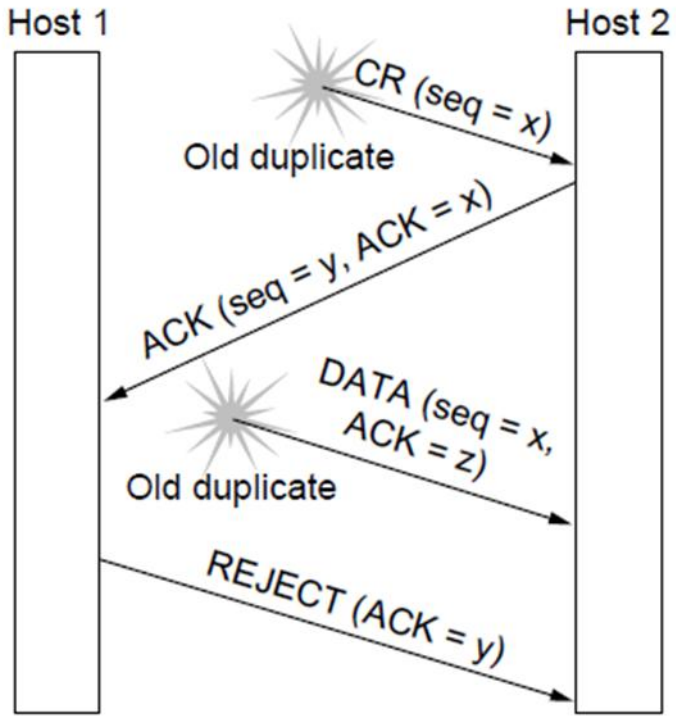
- An old duplicate `CR (seq = x)` suddenly arrives at Host 2 out of nowhere.
- Host 2 responds with `ACK (seq = y, ACK = x)` thinking it's a new connection.
- Host 1 receives the ACK for sequence `x`, recognizes it never sent `x` recently, and send `REJECT (ACK = y)`
- Host 2 safely aborts the bogus connection without delivering data.



Duplicate CR and Duplicate ACK Case

The worst-case scenario: Both an old duplicate `CR` AND an old duplicate `ACK` arrive!

- Old duplicate `CR (seq = x)` arrives at Host 2 → Host 2 sends `ACK (seq = y, ACK = x)`
- An old duplicate packet also arrives at Host 2! `DATA (seq = x, ACK = z)`
- Host 2 sees (which doesn't match its chosen sequence y). Host 1 explicitly REJECTS y, preventing a duplicate transaction.



Connection Release Styles

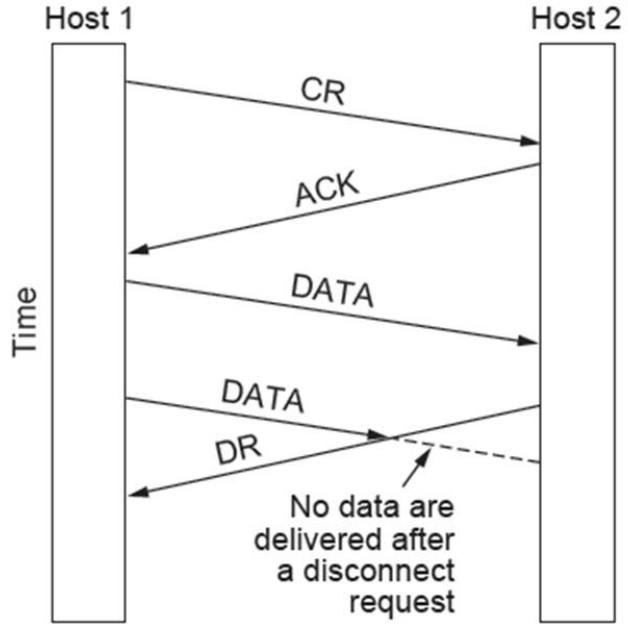
Releasing a connection can be asymmetric or symmetric:

Asymmetric Release

Either party hangs up abruptly (like telephone). Can lead to **data loss** if one side disconnects while the other is still sending data in flight.

Symmetric Release

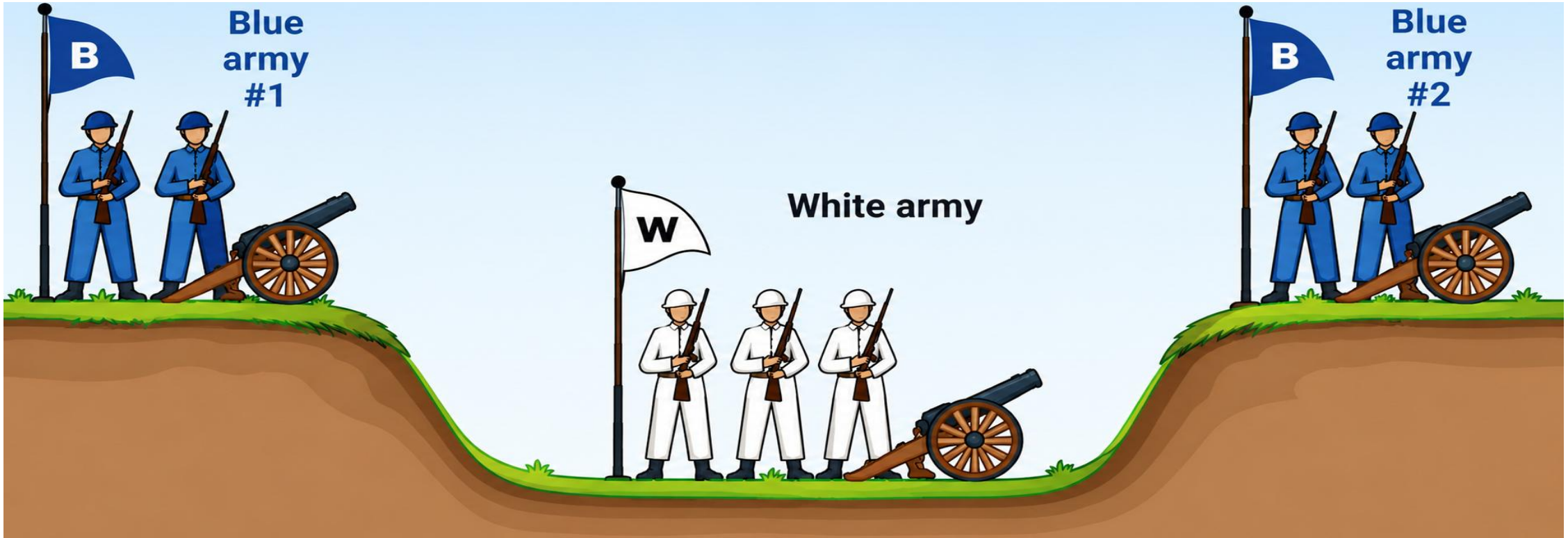
Treats connection as two independent unidirectional streams. Each direction must be closed independently when its sender is finished.



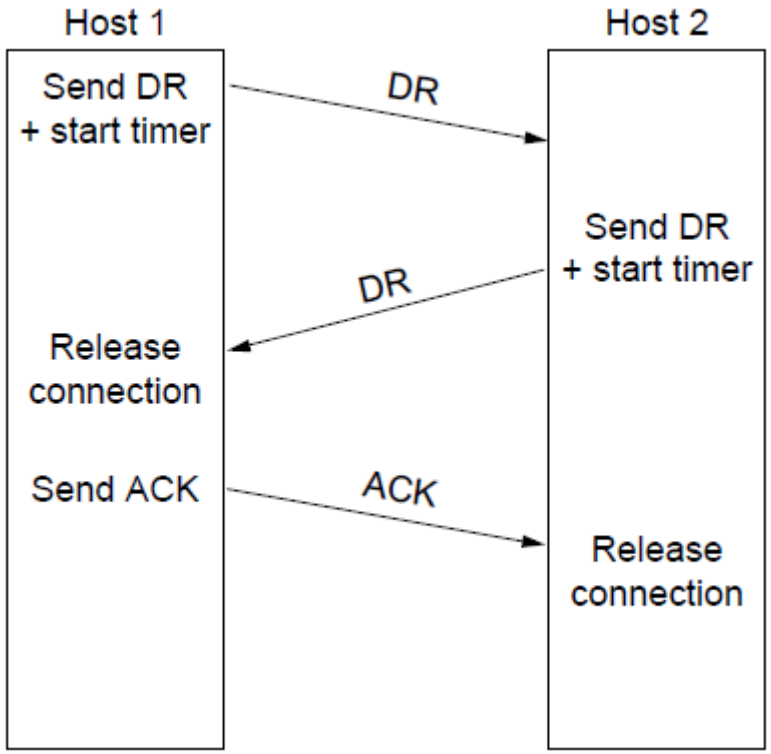
The Two-Army Problem

A fundamental mathematical proof showing that perfect consensus over an unreliable channel is impossible.

- **Scenario:** Blue Army #1 and Blue Army #2 must attack White Army simultaneously to win.
- **Unreliable Channel:** Messengers sent through the valley can be captured.
- **Infinite Regress:** Army 1 sends "Attack at dawn". Army 2 ACKs. But Army 2 won't attack unless it knows Army 1 received the ACK. So Army 1 must ACK the ACK... indefinitely!

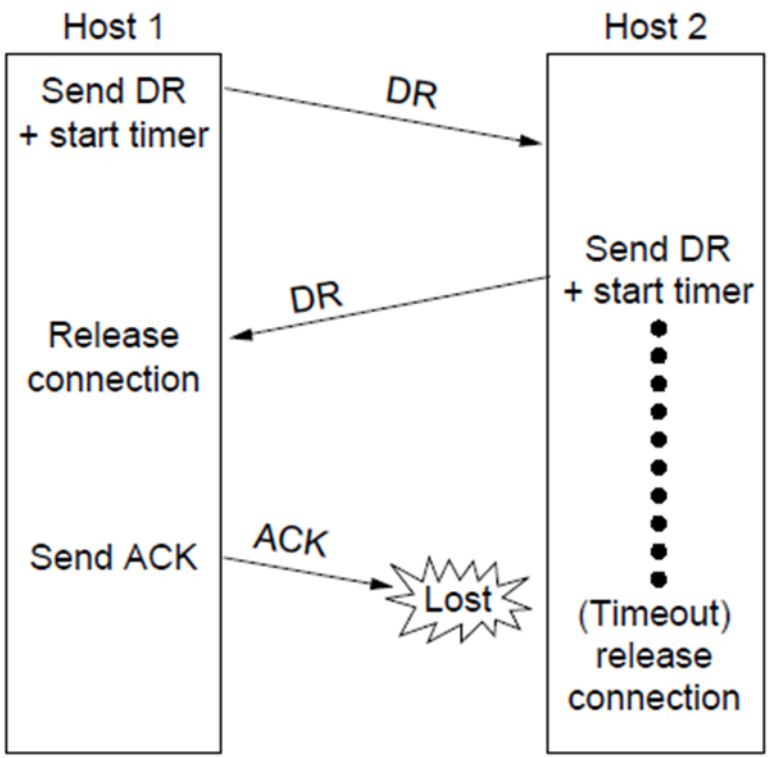


Connection Release: Scenario 1 (Normal)

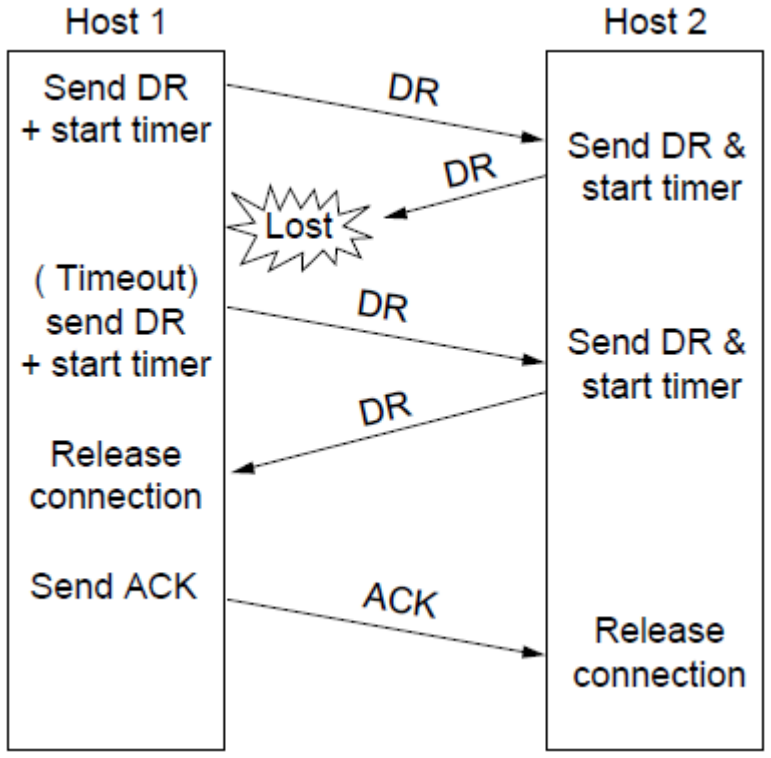


Four protocol scenarios for releasing a connection.
(a) Normal case of three-way handshake

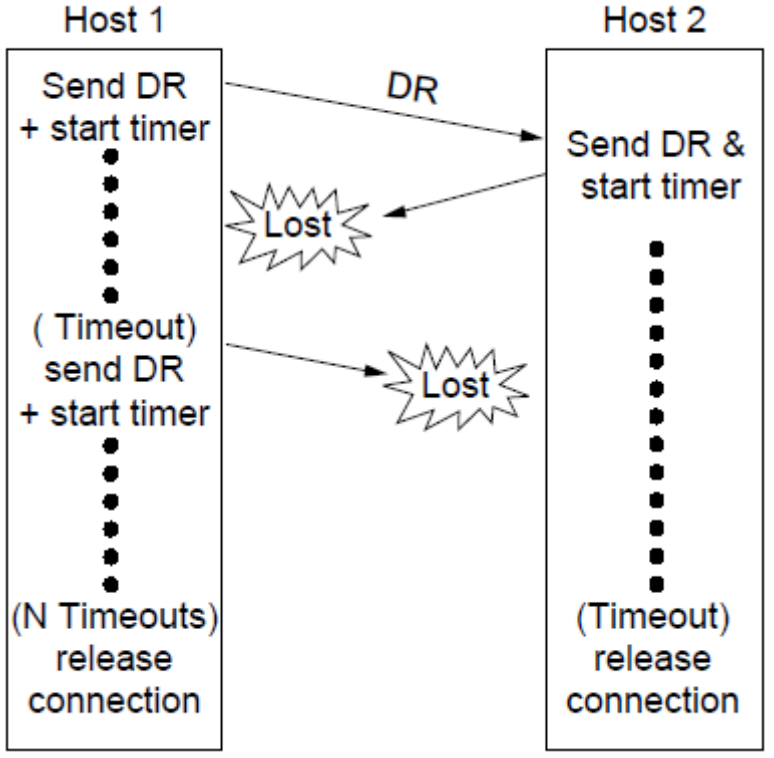
Connection Release: Lost Packets



(b) Final ACK lost



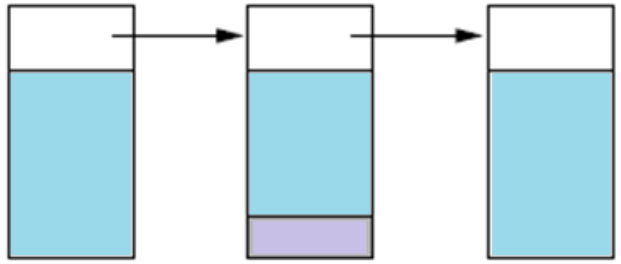
(c) Response lost



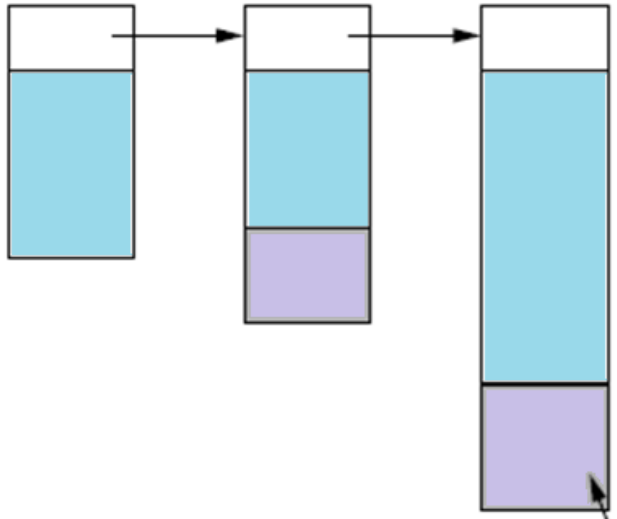
(d) Response lost and subsequent DRs lost.

Flow Control & Buffer Strategies

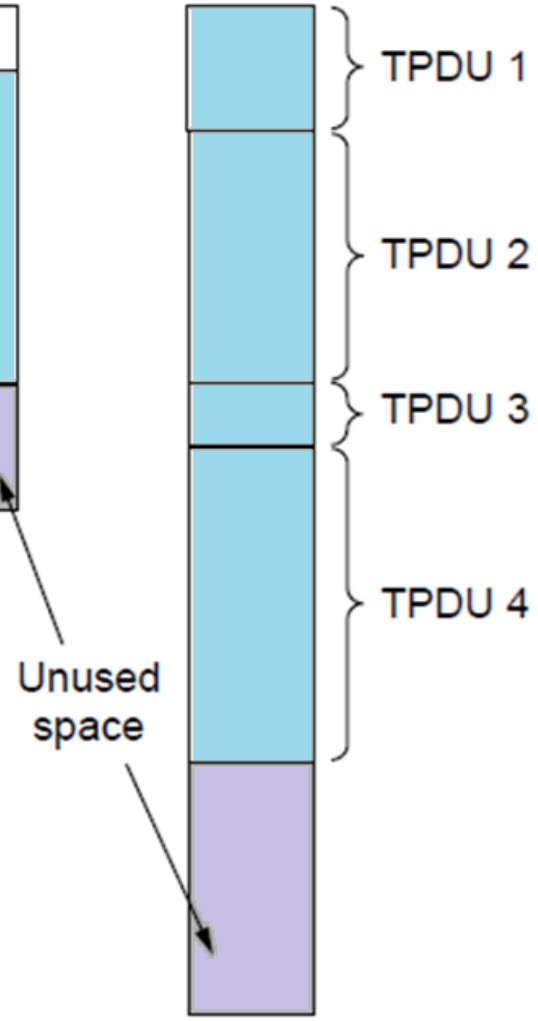
Managing buffers at the receiver transport entity:



(a) Chained fixed-size buffers



(b) Chained variable-sized buffers



(c) One large circular buffer per connection.

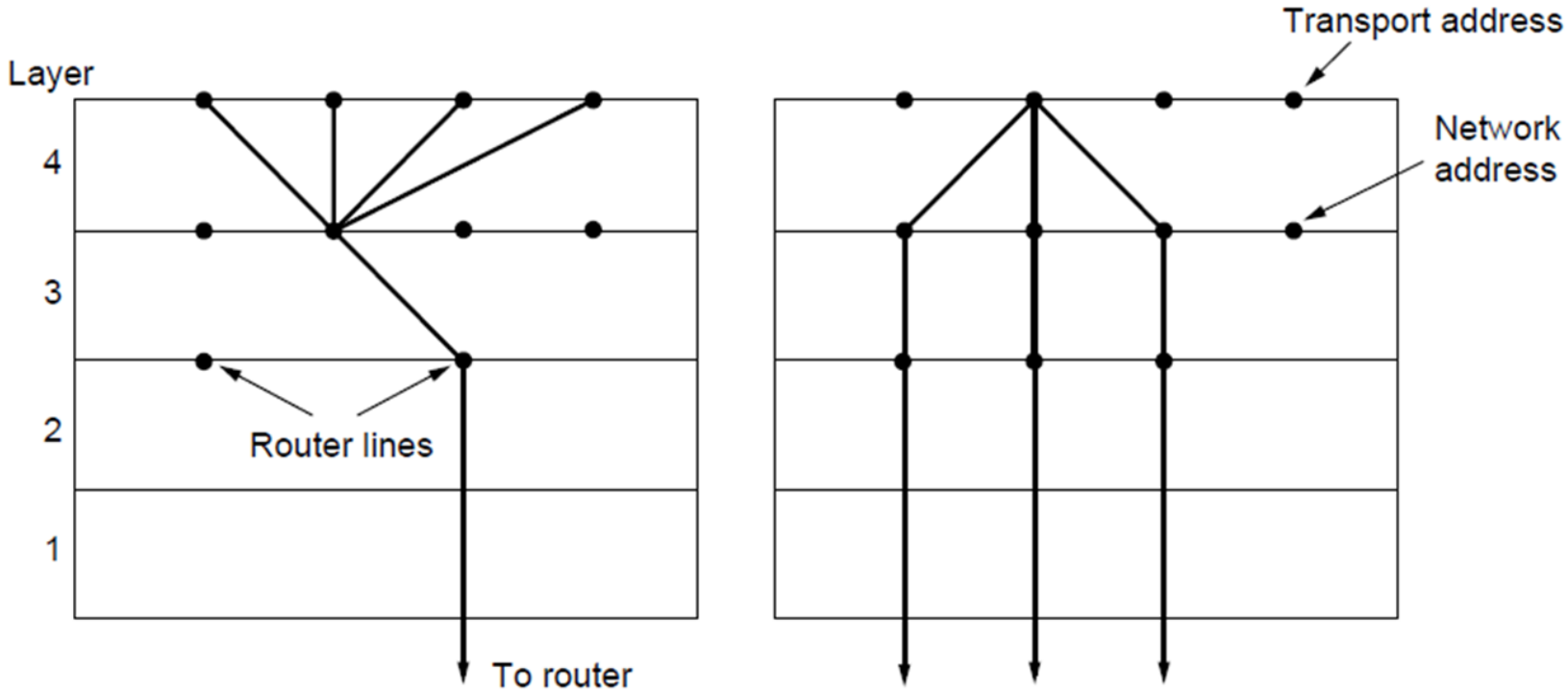
Dynamic Buffer Allocation

	A	Message	B	Comments
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	• • •	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	• • •	<ack = 6, buf = 4>	←	Potential deadlock

Dynamic buffer allocation. The arrows show the transmission direction. An ellipsis (...) indicates a lost TPDU

Transport Layer Multiplexing

Sharing network resources efficiently using multiplexing techniques:



Upward Multiplexing

Multiple transport connections (ports) share a single network address (IP). Essential when IP connections are scarce or expensive.

Downward (Inverse) Multiplexing

A single high-throughput transport connection is split across multiple network paths simultaneously to boost throughput or reliability.

Transport Layer Crash Recovery

- In case of a network or router crash, the protocol can easily recover by retransmitting the lost segment (as the transport entity is entirely in the hosts)
- In case of a host crash, it is desirable that the client should continue when the server crashes and quickly reboots
- When it comes back up, its tables are reinitialized, so it no longer knows precisely where it was.
  In an attempt to recover its previous status, the server might send a broadcast segment to all other hosts, announcing that it has just crashed and requesting that its clients to inform the status of all open connections.
- Each client can be in one of two states: one segment outstanding, S1, or no segments outstanding, S0.
- Obviously, the client should retransmit if and only if it has an unacknowledged segment outstanding (i.e., is in state S1) when it learns of the crash

Crash Recovery State Trade-offs

- Is it that simple?
- Server can implement either Ack then write OR Write then Ack approach. (Write means delivering data to the upper layer)

Strategy used by sending host	Strategy used by receiving host: First ACK, then write			Strategy used by receiving host: First write, then ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly
DUP = Protocol generates a duplicate message
LOST = Protocol loses a message

Conclusion: True crash recovery requires application-level checkpoints and persistent database logging above Layer 4.

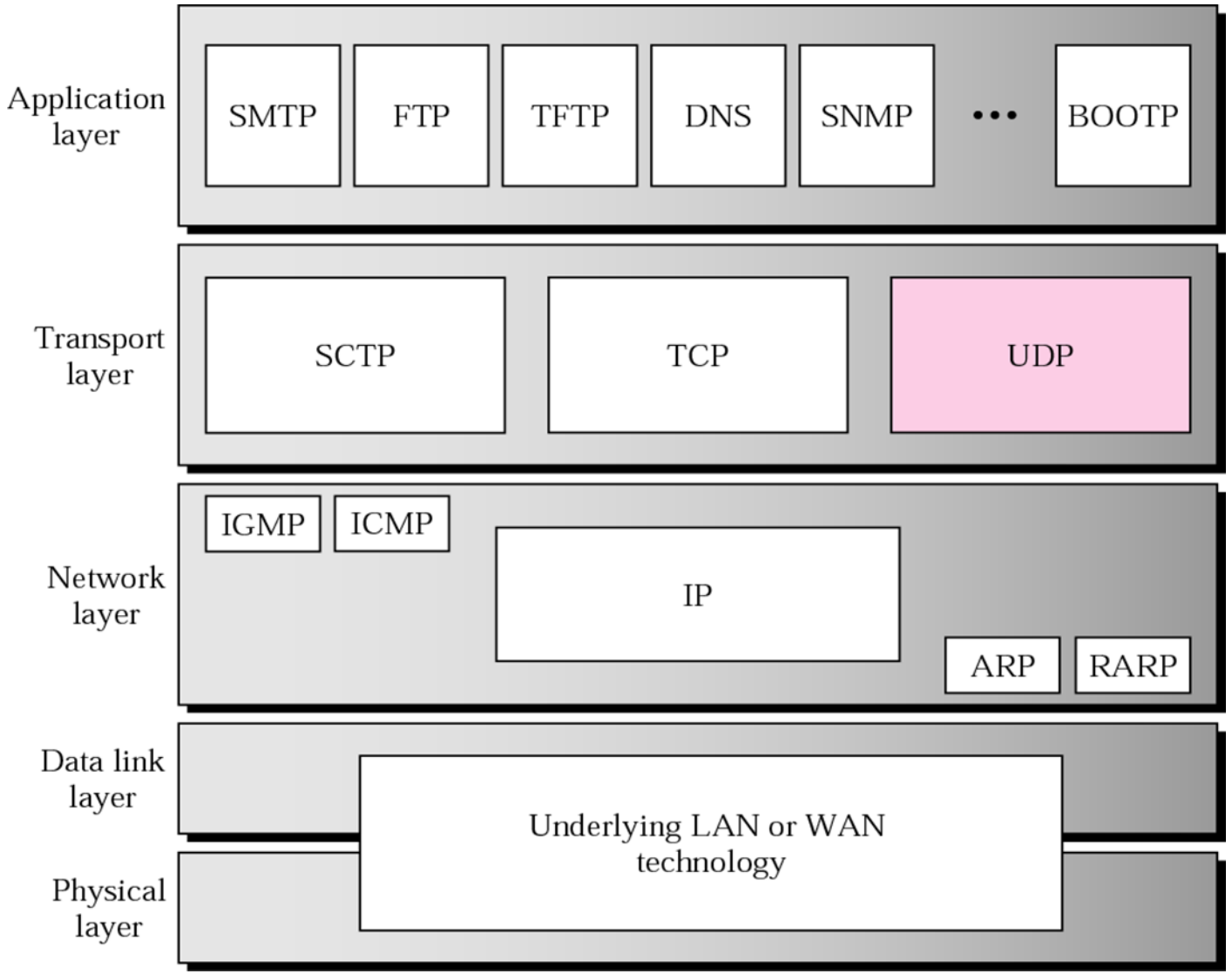
Example Transport Layer Protocols

- **From concepts to implementations**: The transport services we studied (process-to-process delivery, multiplexing, reliability, flow and congestion control) are realized in the Internet by specific protocols.
- **Three standard transport protocols are used over IP**:
  - **UDP**: User Datagram Protocol
  - **TCP**: Transmission Control Protocol
  - **SCTP**: Stream Control Transmission Protocol
- **Same network layer, different services**: All three run over best-effort IP, but each adds a different set of services on top of it.
- **Each protocol answers a different design choice**:
  - UDP: minimal service, simple and fast, with no connection and no reliability
  - TCP: connection-oriented, reliable, byte-stream service
  - SCTP: connection-oriented and reliable like TCP, but message-oriented, with multiple streams and multihoming
- **Application decides the protocol**: The choice depends on what the application needs, such as speed, reliability, ordering, and message boundaries.

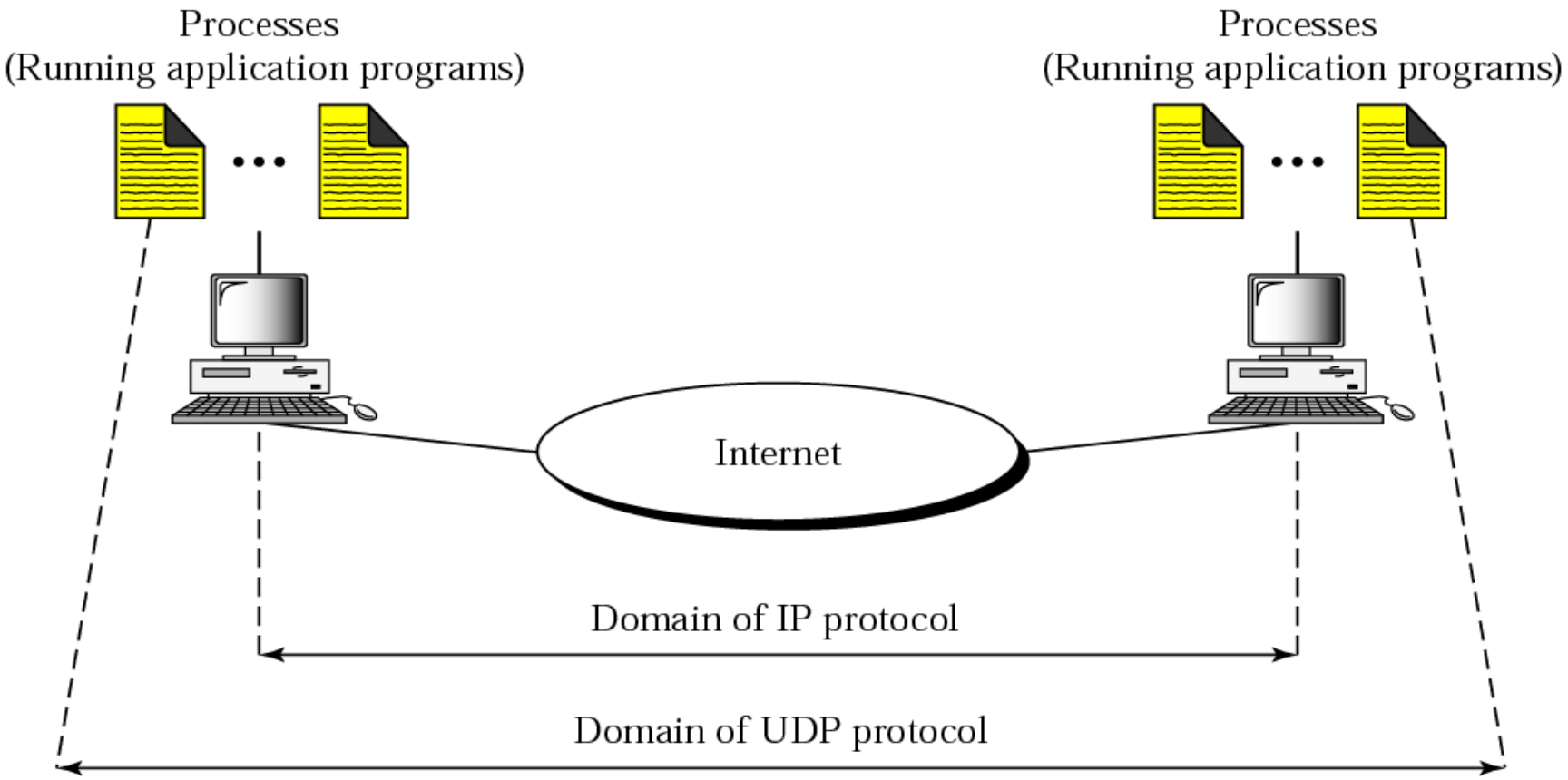
Example Transport Layer Protocols :User Datagram Protocol (UDP)

- UDP is unreliable connection-less protocol to support process-to-process delivery
- Low overhead, simplified header, no sequence numbers, no acknowledgement, no retransmissions makes it suitable for application concerned with timely delivery rather than reliability

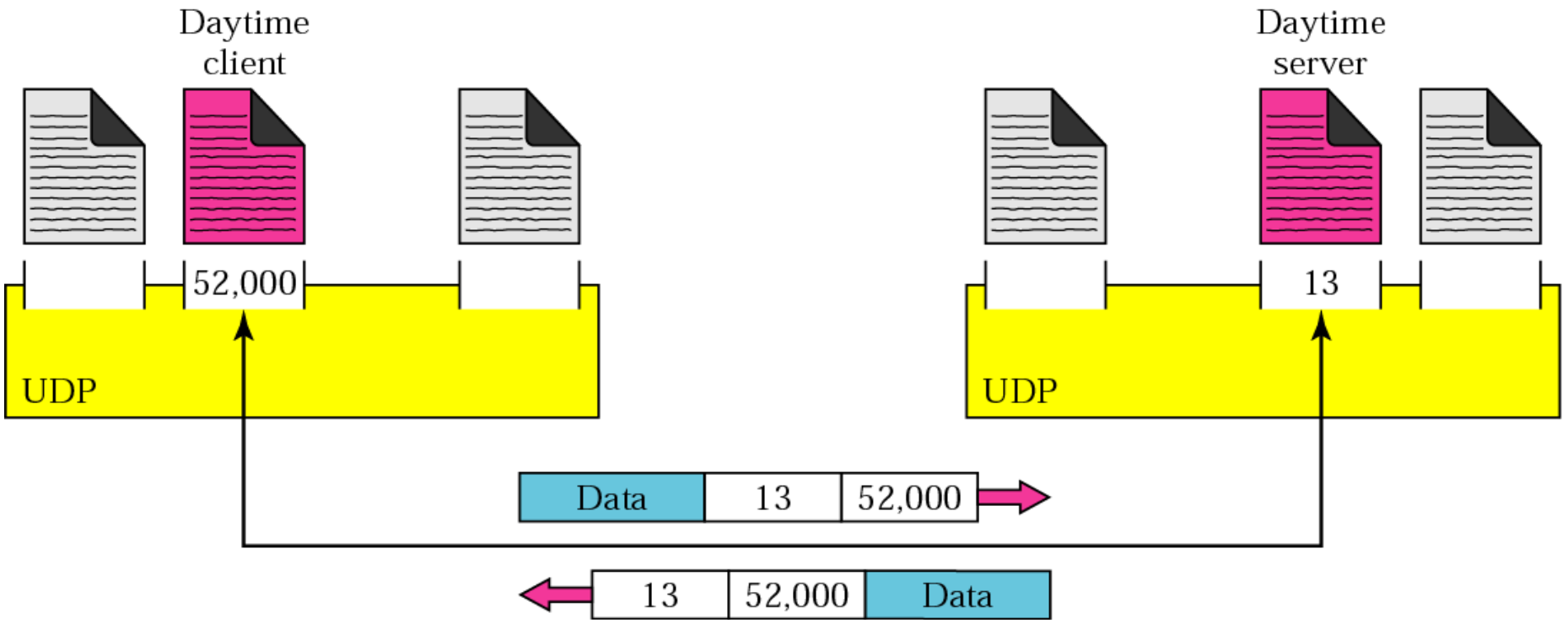
Position of UDP in the TCP/IP stack



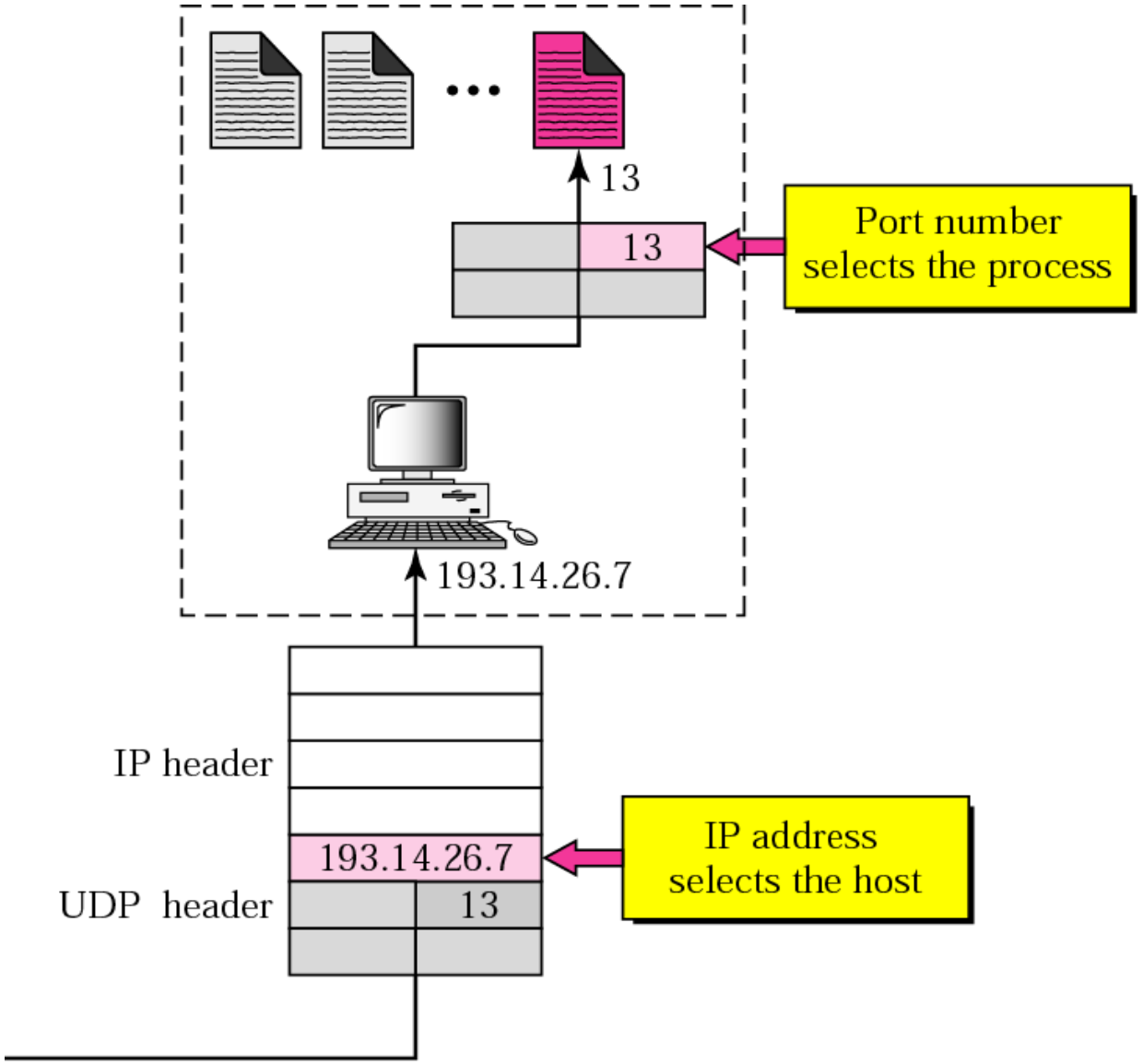
UDP versus IP



Port Numbers

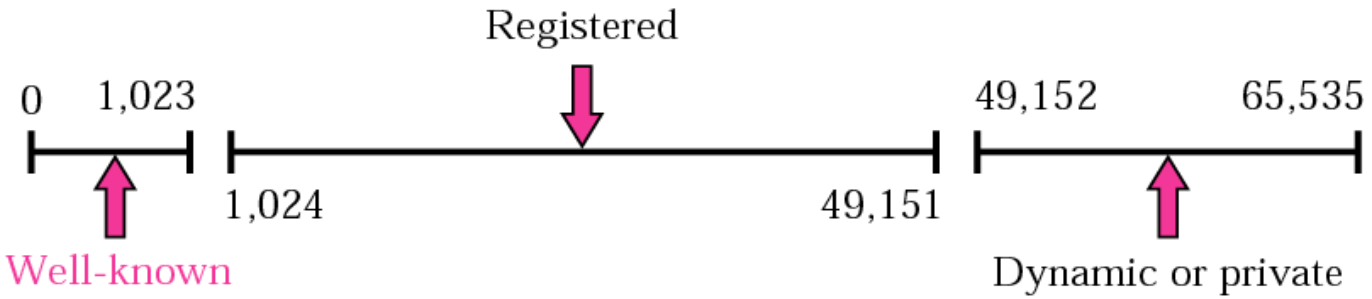


IP addresses versus port numbers



ICANN ranges

- The well-known port numbers are less than 1024
- Port numbers from 1024 to 49,151 are available for registered services
- Anything beyond 49,152 are used for OS for dynamic assignment to client processed



Well-known ports used with UDP

Port	*Protocol*	*Description*
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Nameserver	Domain Name Service
67	Bootps	Server port to download bootstrap information
68	Bootpc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

Example

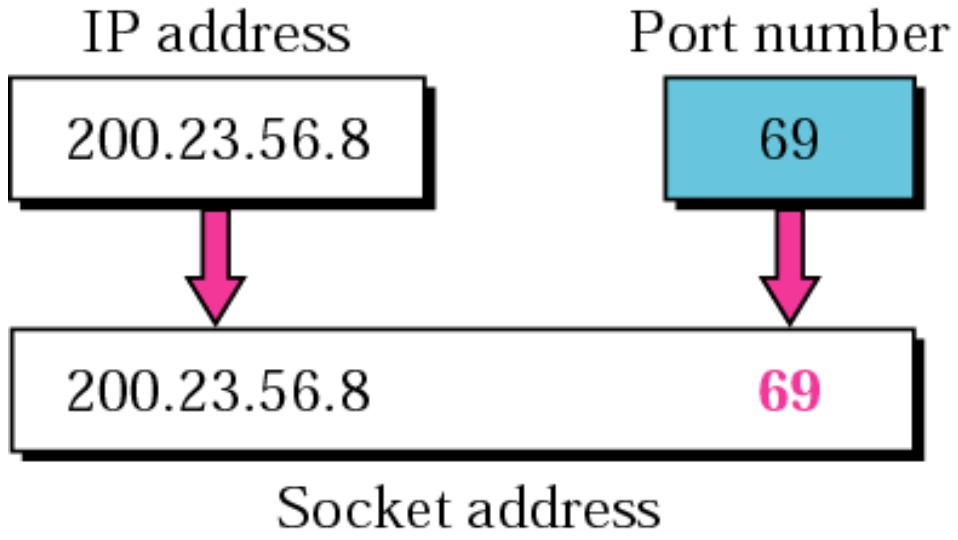
- In UNIX, the well-known ports are stored in a file called /etc/services. Each line in this file gives the name of the server and the well-known port number. We can use the grep utility to extract the line corresponding to the desired application. The following shows the port for TFTP. Note TFTP can use port 69 on either UDP or TCP.

```
$ grep tftp /etc/services
  tftp          69/tcp
  tftp          69/udp
```

```
$ grep snmp /etc/services
  snmp   161/tcp  #Simple Net Mgmt Proto
  snmp   161/udp #Simple Net Mgmt Proto
  snmptrap    162/udp #Traps for SNMP
```

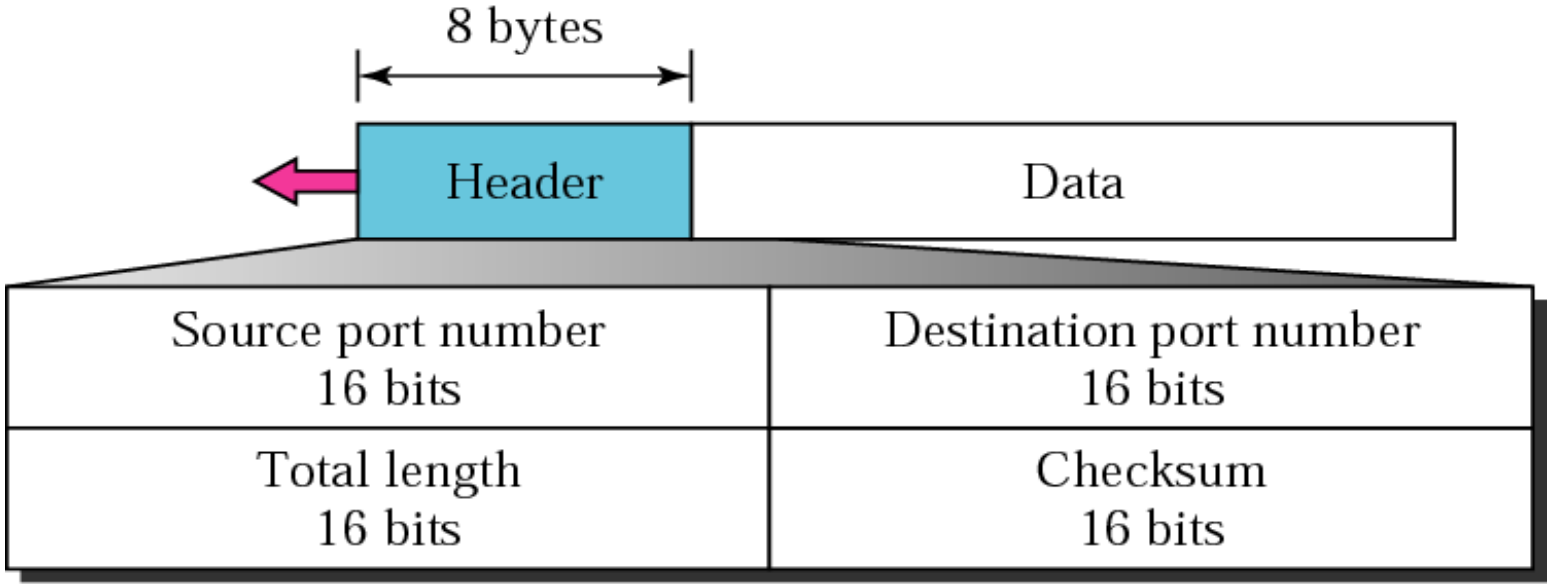
Socket Address

- Socket is communication endpoint of a channel
- It is identified by socket address on both ends
- A socket address is address of device and port number of the process running on that device



User Datagram

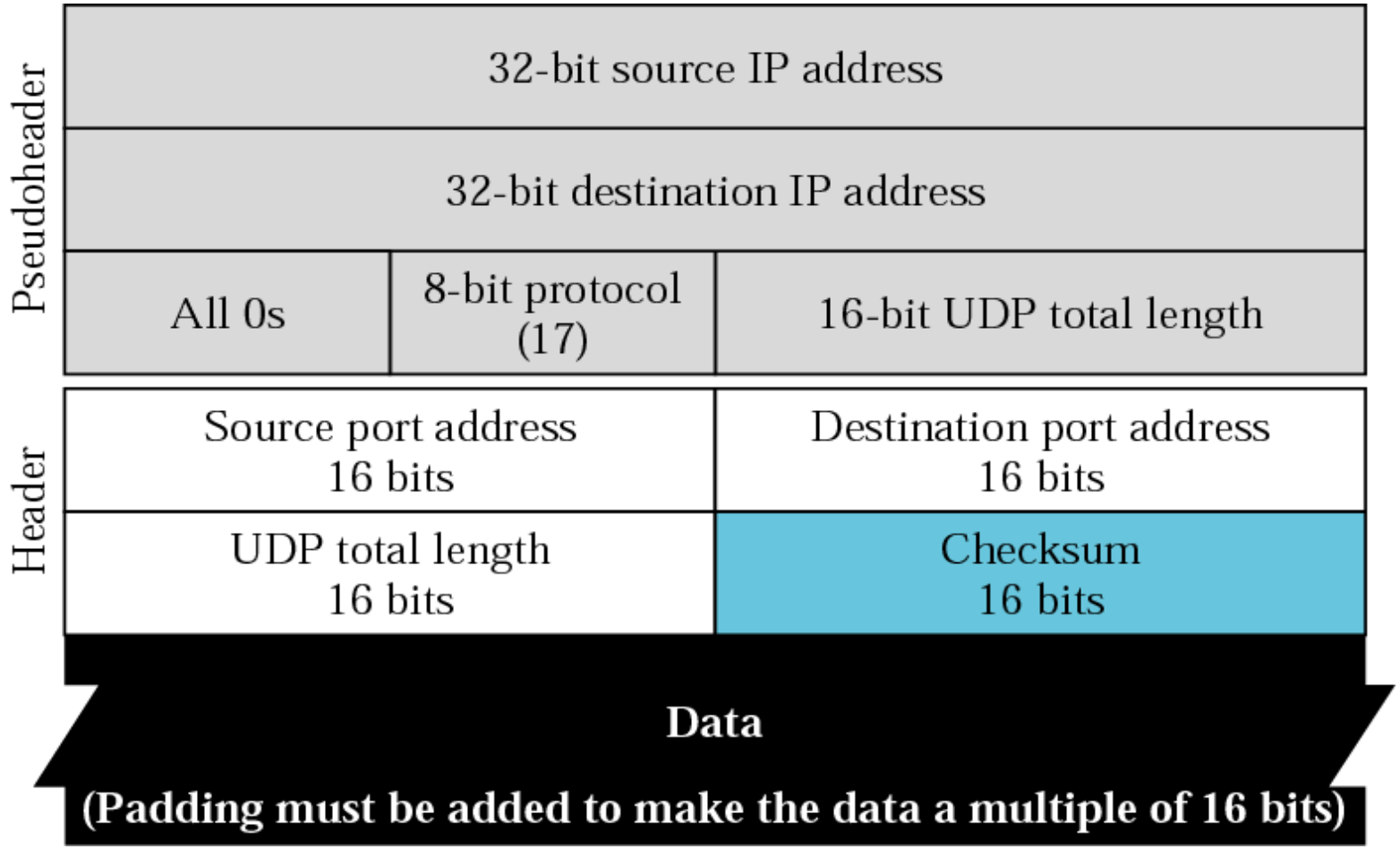
- UDP packets are called user datagrams and have a fixed-size header of 8 bytes.



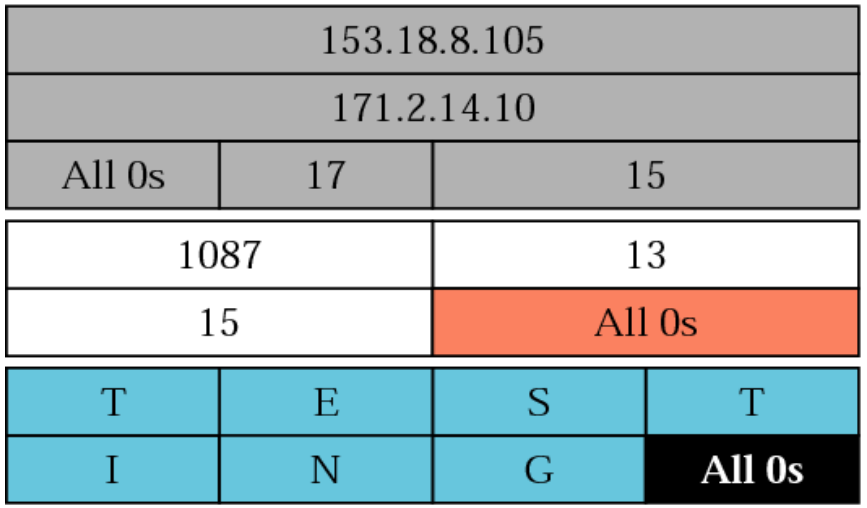
- UDP length = IP length − IP header's length (effectively it is IP payload length)

Checksum

- UDP checksum calculation is different from the one for IP and ICMP. Here the checksum includes three sections: a pseudoheader, the UDP header, and the data coming from the application layer.

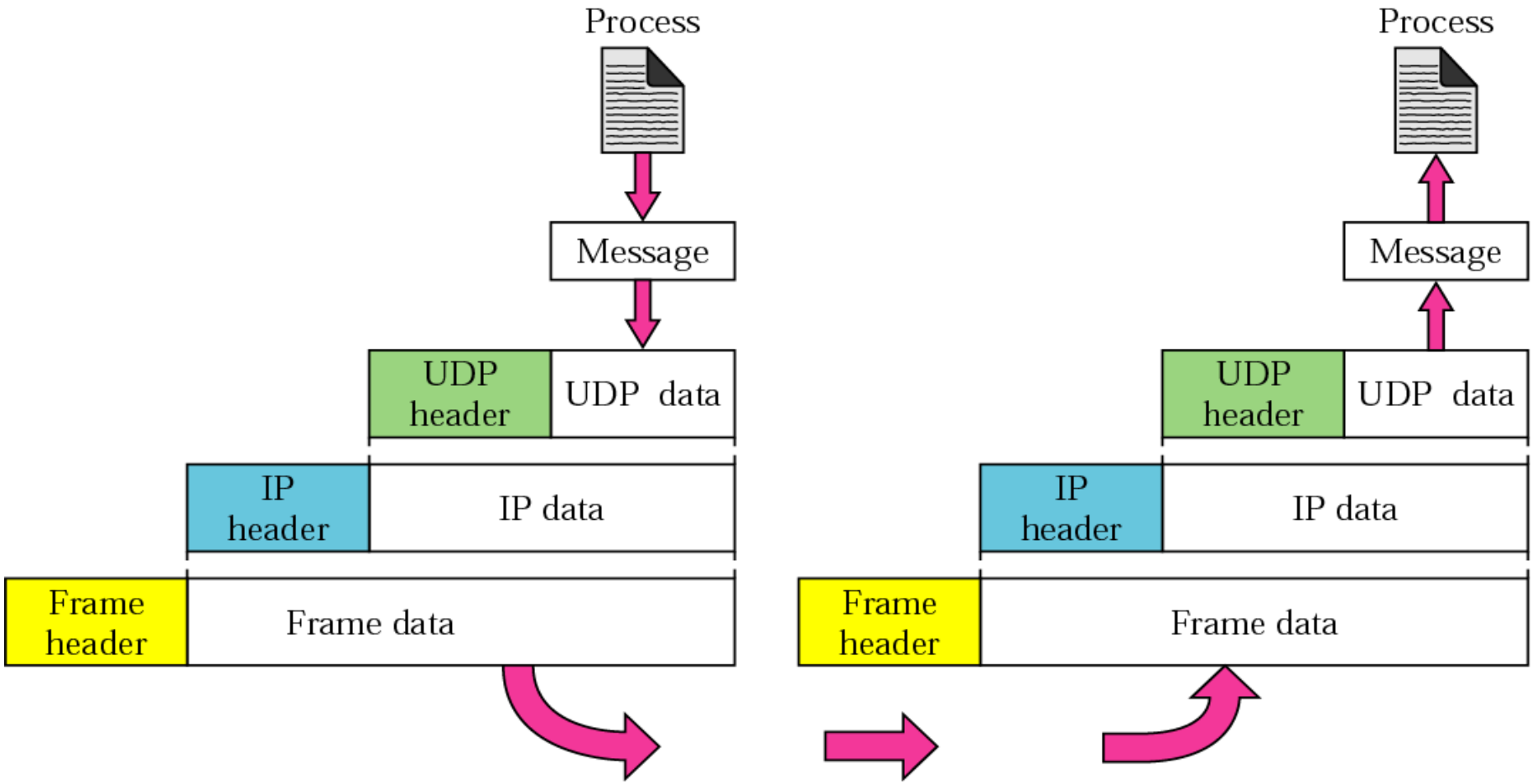


Checksum - Example

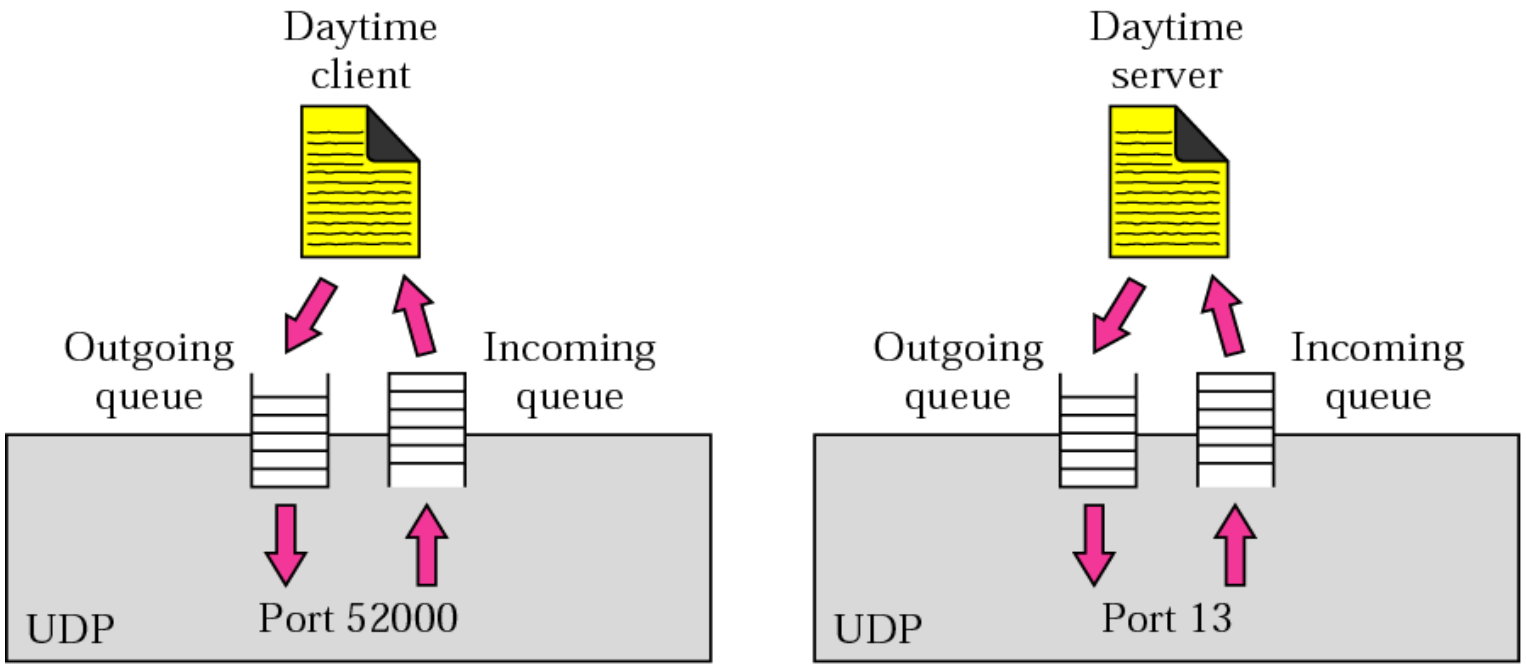


10011001 00010010	153.18
00001000 01101001	8.105
10101011 00000010	171.2
00001110 00001010	14.10
00000000 00010001	0 and 17
00000000 00001111	15
00000100 00111111	1087
00000000 00001101	13
00000000 00001111	15
00000000 00000000	0 (checksum)
01010100 01000101	T and E
01010011 01010100	S and T
01001001 01001110	I and N
01000111 00000000	G and 0 (padding)
10010110 11101011	Sum
01101001 00010100	Checksum

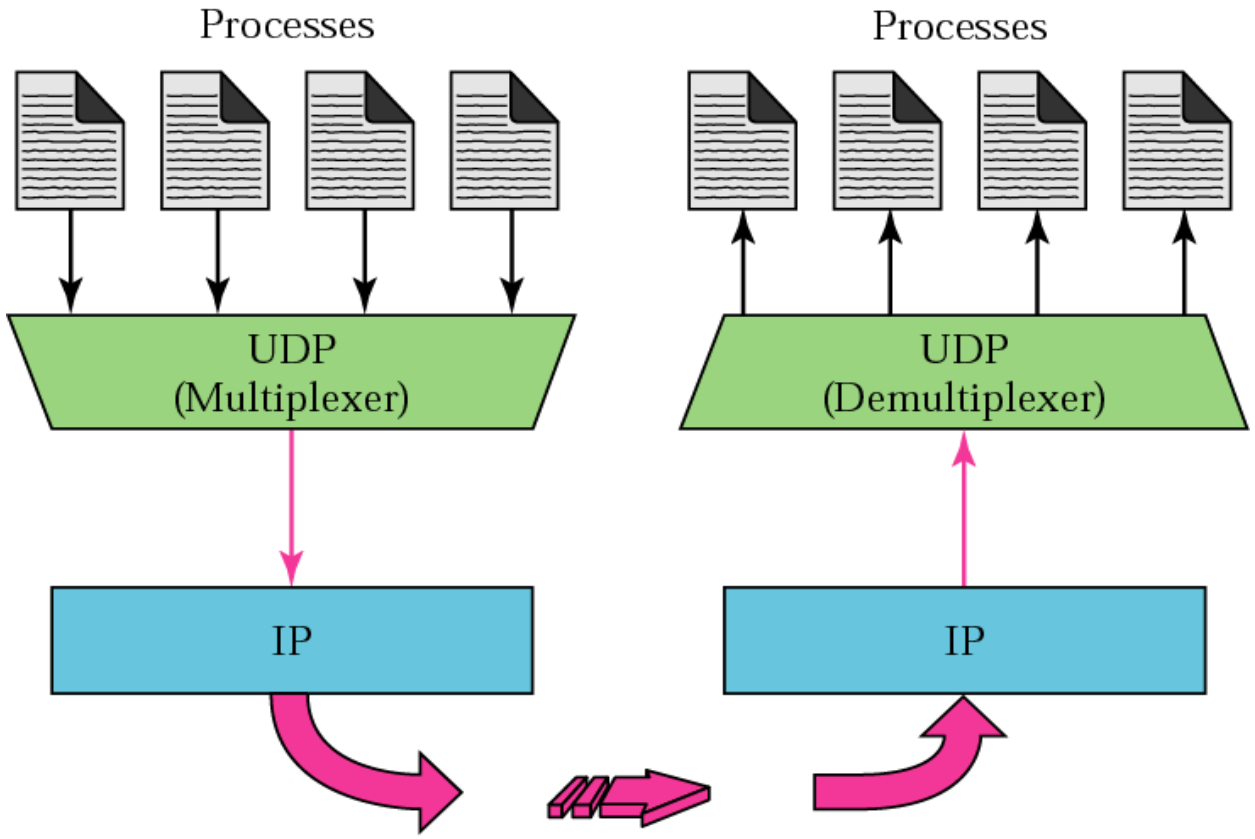
Encapsulation and decapsulation



Queues in UDP



Multiplexing and demultiplexing



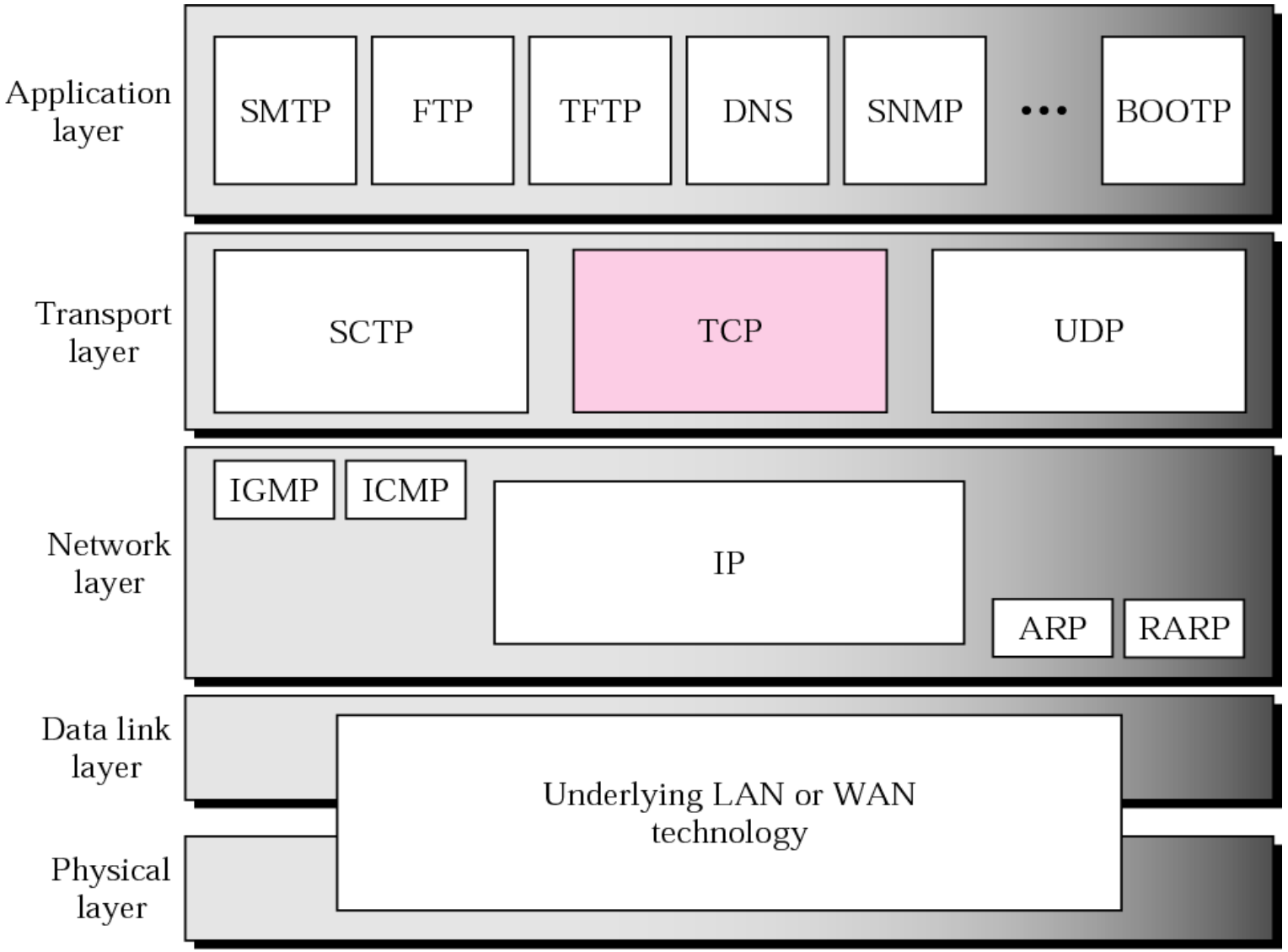
Uses of UDP

- UDP is a low overhead protocol
- Primarily suitable for applications which can tolerate packet loss
- Suitable for applications which needs timely but may not be reliable delivery like multimedia applications

Example Transport Layer Protocols :Transmission Control Protocol (TCP)

- TCP is reliable, connection-oriented, byte stream delivery protocol
- Full-duplex communication
- Connection needed before any bytes can be sent
- Each byte sent is acknowledged, making it extremely reliable protocol
- Suitable to application expecting reliability from the network

Position of UDP in the TCP/IP stack



Well-known ports used by TCP

Port	*Protocol*	*Description*
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20	FTP, Data	File Transfer Protocol (data connection)
21	FTP, Control	File Transfer Protocol (control connection)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol
111	RPC	Remote Procedure Call

Example

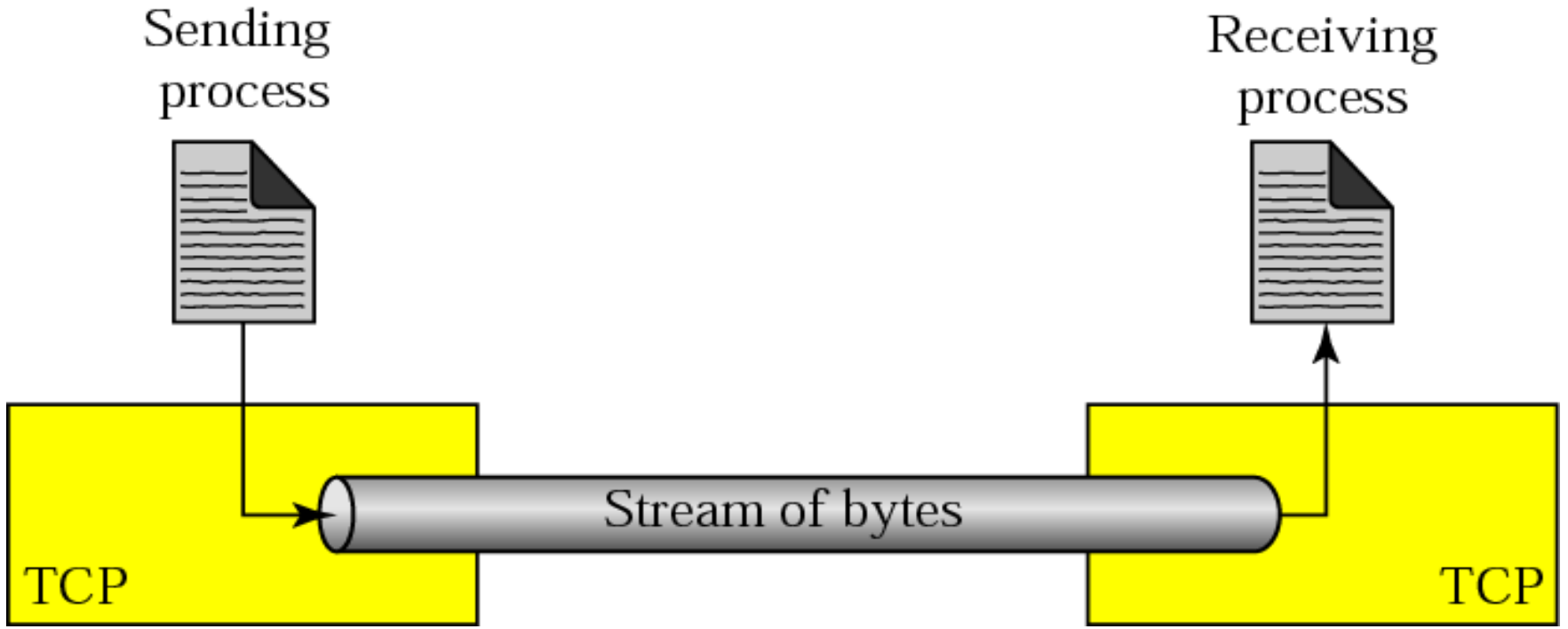
- In UNIX, the well-known ports are stored in a file called /etc/services. Each line in this file gives the name of the server and the well-known port number. We can use the grep utility to extract the line corresponding to the desired application. The following shows the ports for FTP.

```
$ grep  ftp   /etc/services

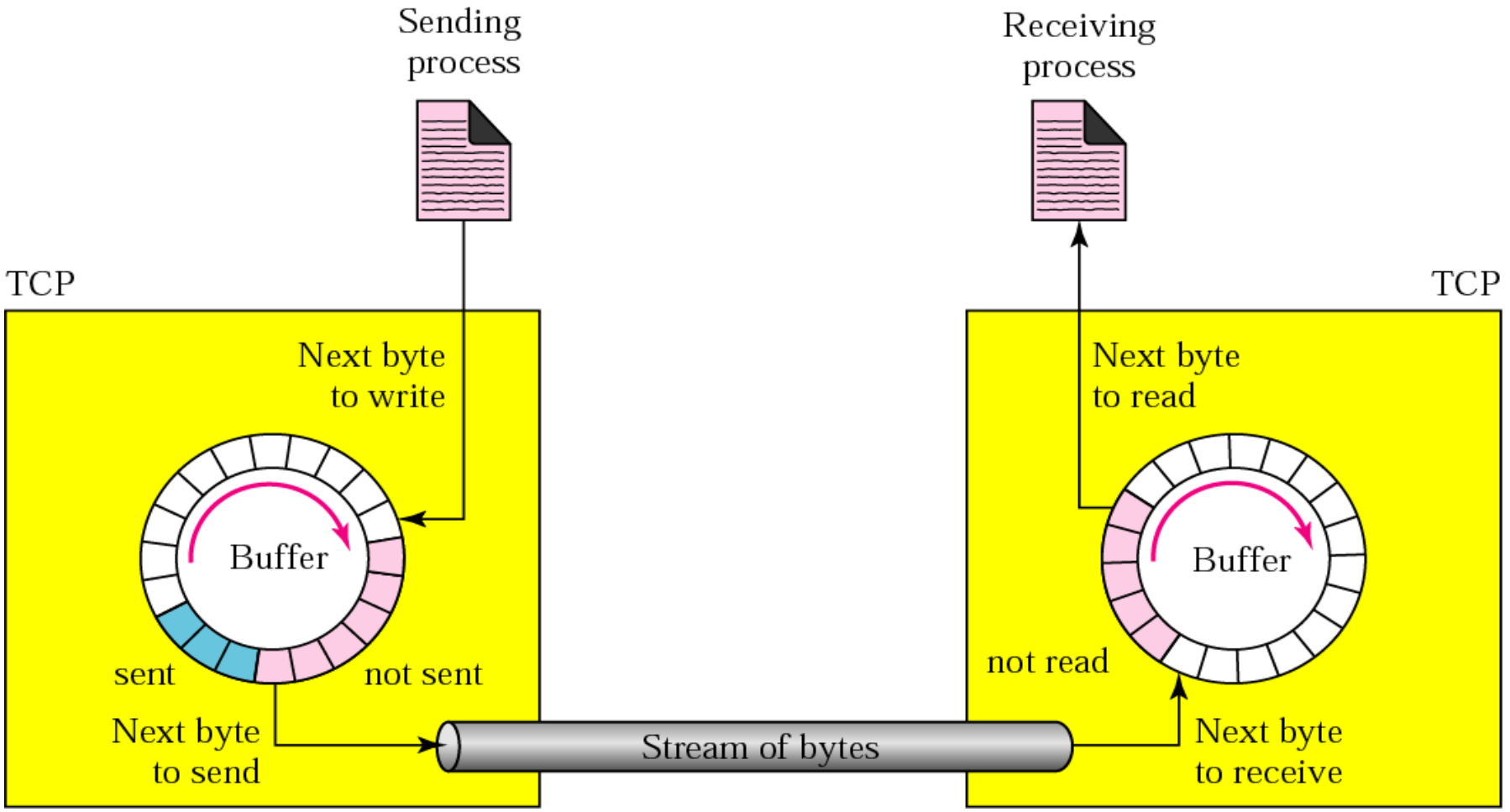
ftp-data      20/tcp
ftp-control 21/tcp
```

Stream Delivery

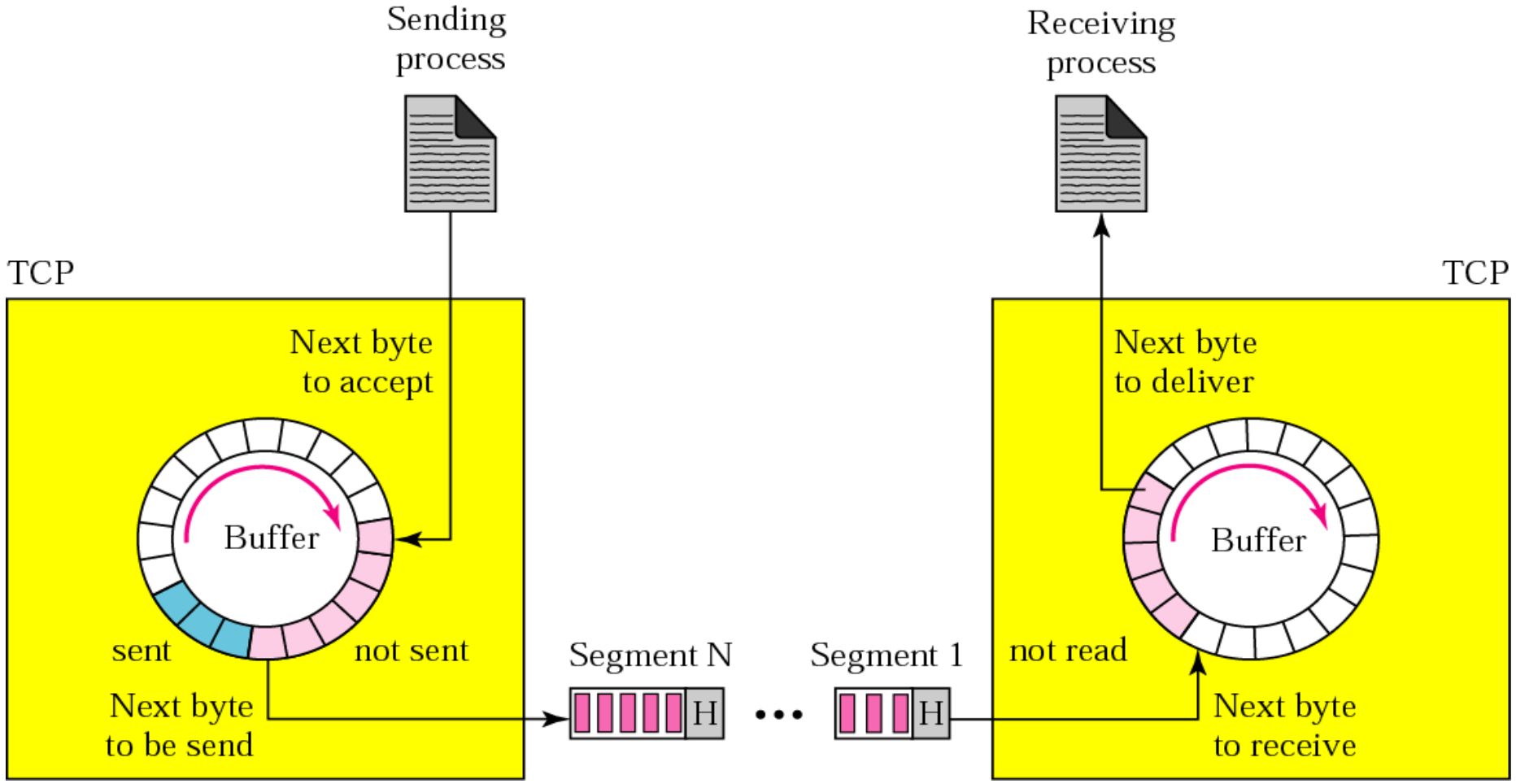
- TCP virtually offers a reliable pipe like channel between the communicating process.
- The pipe delivers all the bytes in same order to receiving process



Sending and receiving buffers



TCP segments



TCP Features

- Numbering System
  - The bytes of data being transferred in each connection are numbered by TCP. The numbering starts with a randomly generated number.
- Flow Control
  - TCP controls the rate of transmission to ensure receiver is not swamped by senders transmission rate
- Error Control
  - Each lost or corrupted segment is retransmitted ensuring error-free transmission
- Congestion Control
  - In case congestion detection, TCP slows down the transmission rate to help control the congestion in network

Example

- Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 10001. What are the sequence numbers for each segment if data is sent in five segments, each carrying 1000 bytes?

Segment 1 → *Sequence Number: 10,001 (range: 10,001 to 11,000)*

Segment 2 → *Sequence Number: 11,001 (range: 11,001 to 12,000)*

Segment 3 → *Sequence Number: 12,001 (range: 12,001 to 13,000)*

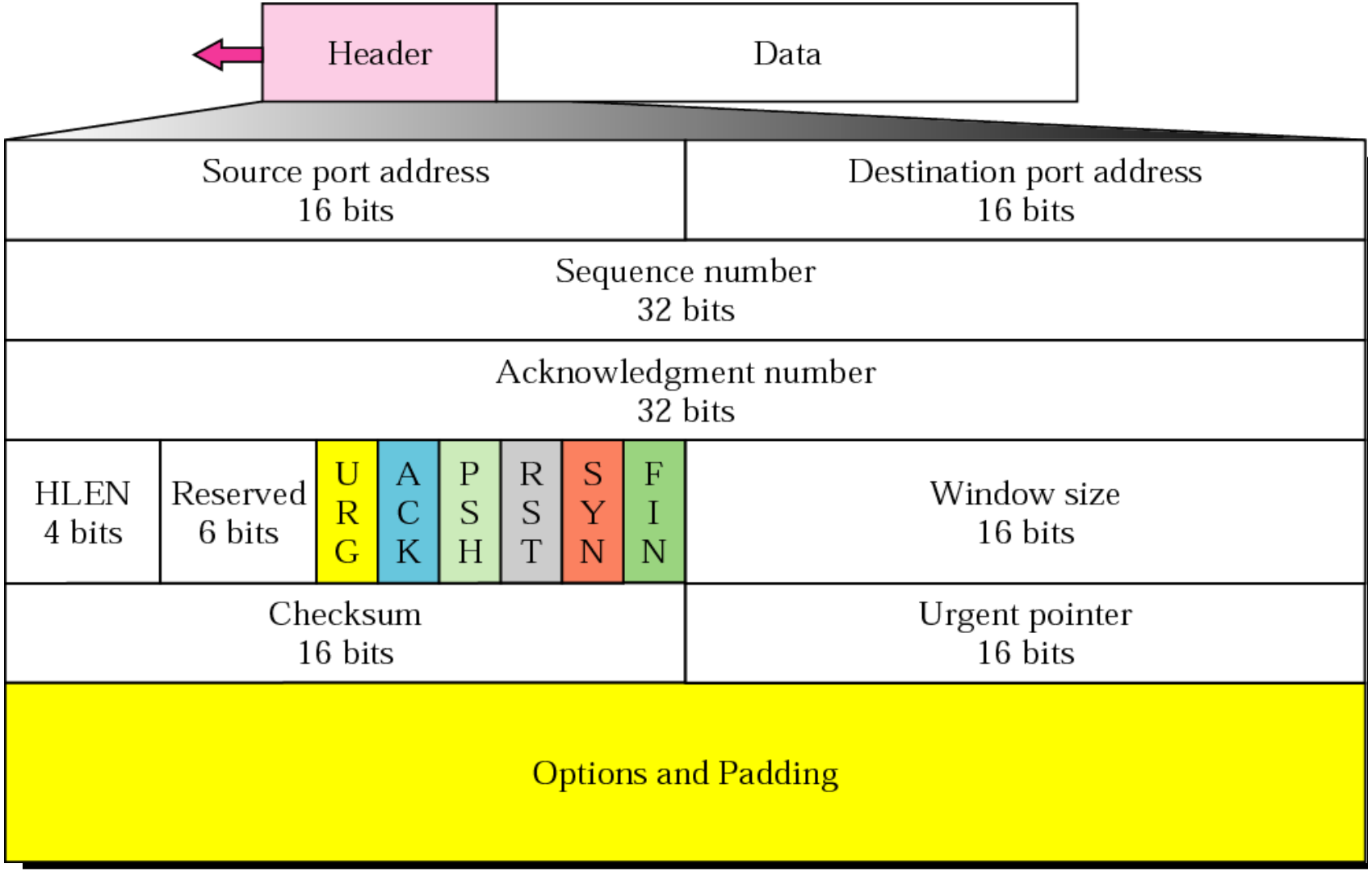
Segment 4 → *Sequence Number: 13,001 (range: 13,001 to 14,000)*

Segment 5 → *Sequence Number: 14,001 (range: 14,001 to 15,000)*

- The value in the sequence number field of a segment defines the number of the first data byte contained in that segment.

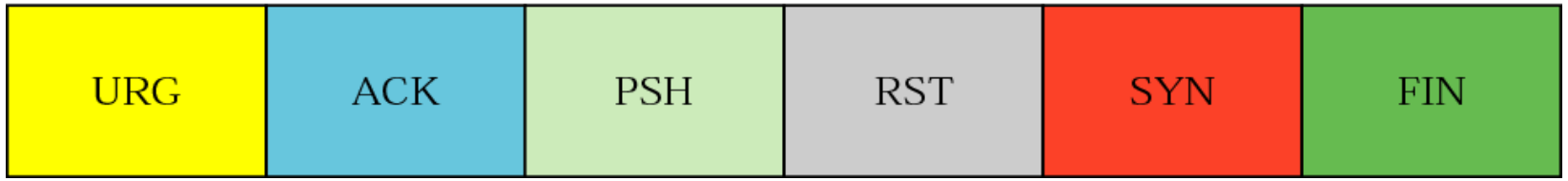
- The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive.
- The acknowledgment number is cumulative.

TCP segment format



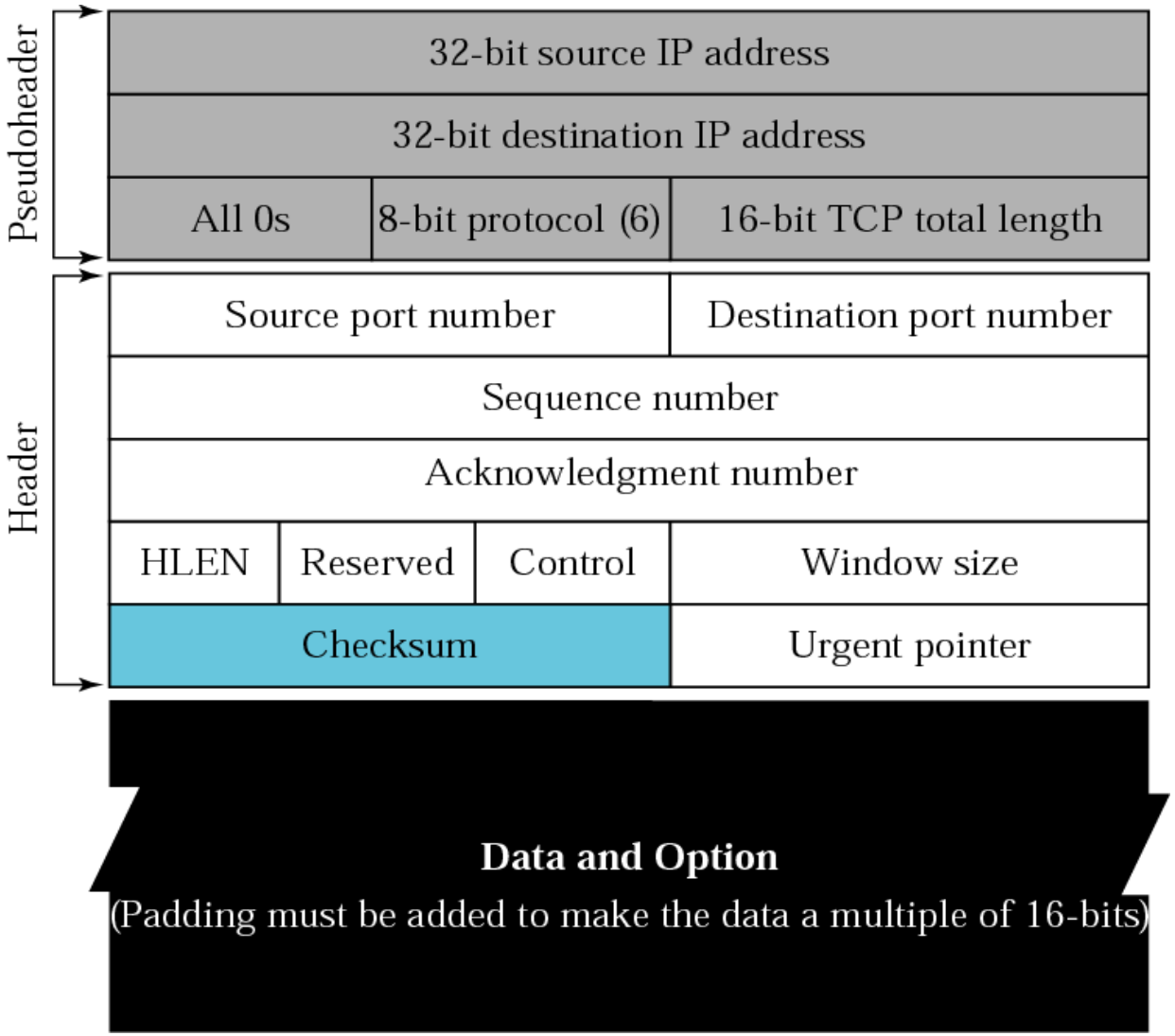
TCP Control fields

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push
RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection

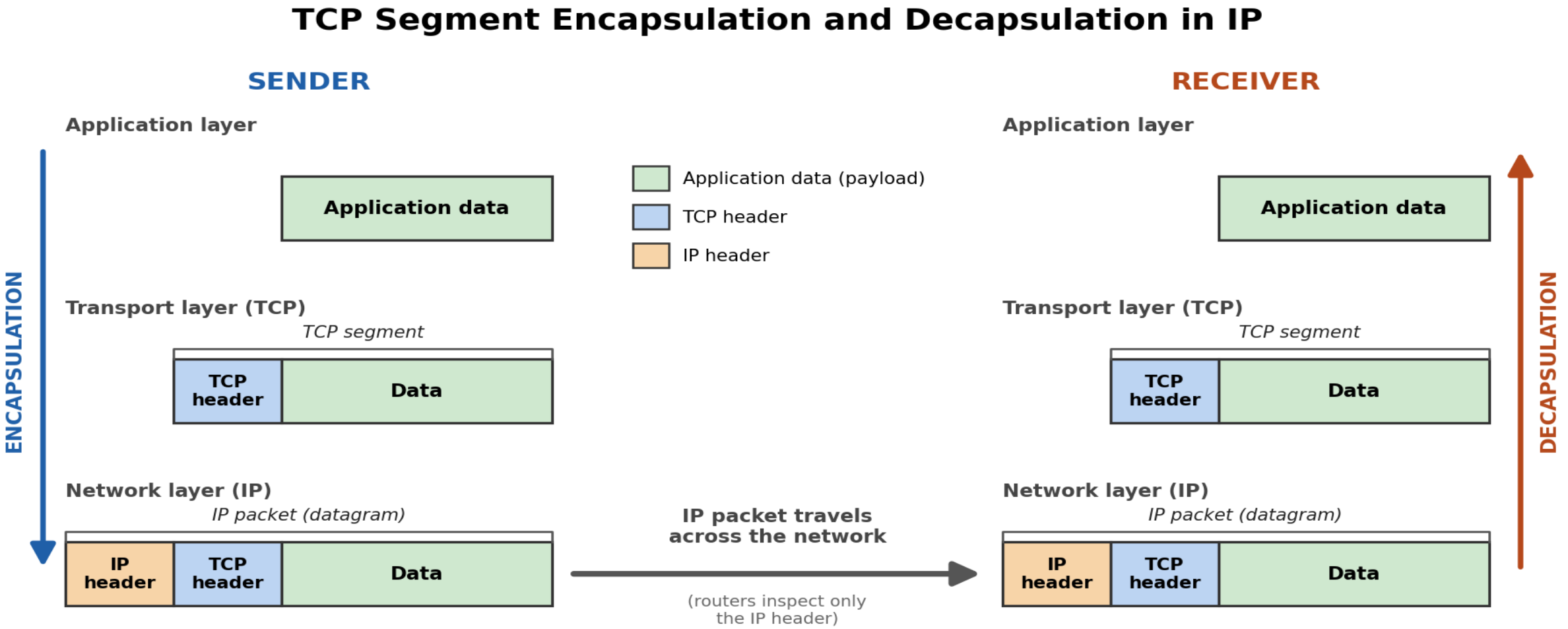


Checksum Calculation

- The inclusion of the checksum in TCP is mandatory.



Encapsulation and decapsulation



① TCP adds its header: source/dest port, seq/ack numbers, flags, window, checksum
② IP adds its header: source/dest IP, TTL, protocol = 6 (TCP), length, header checksum

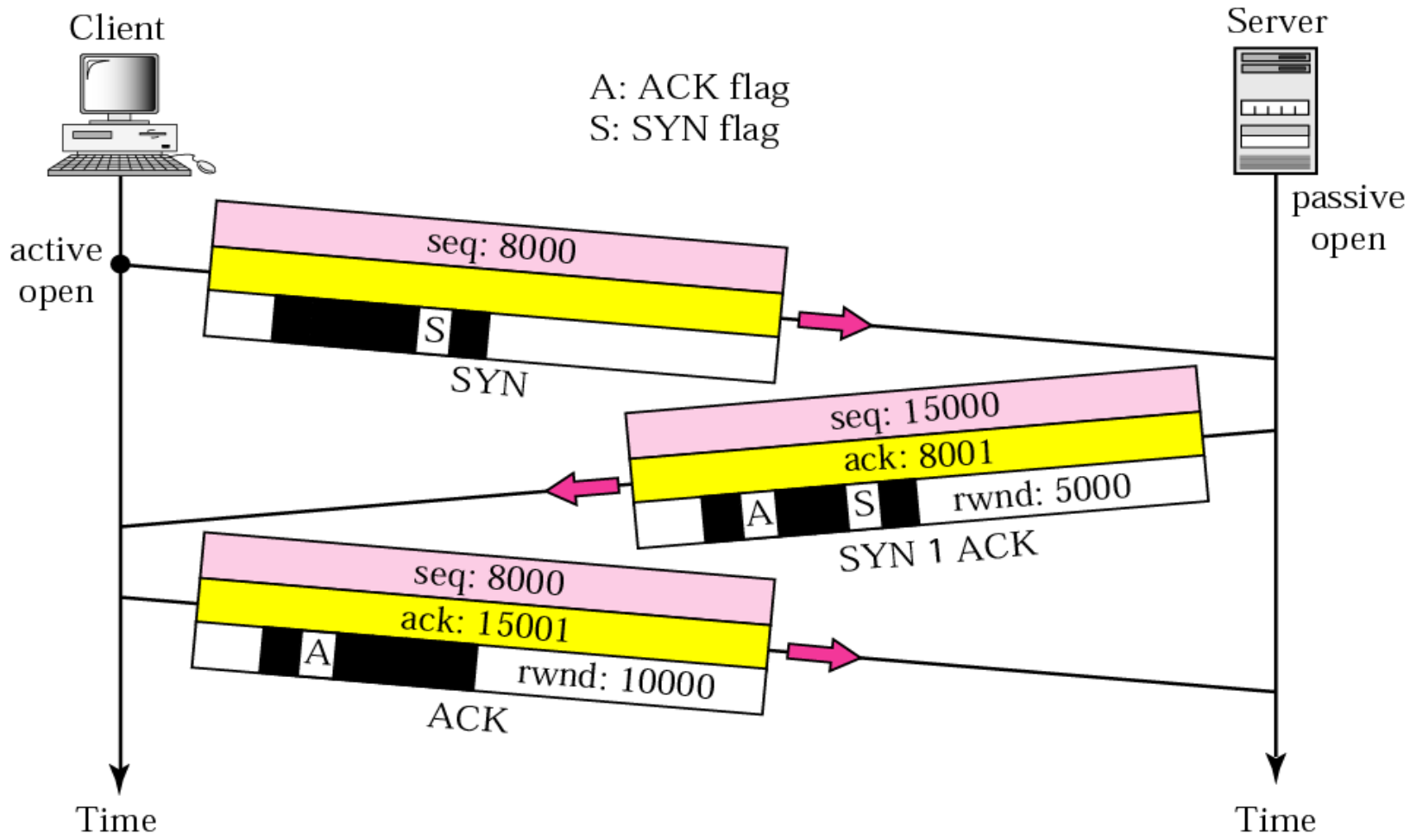
③ IP checks dest address, strips IP header, protocol = 6 → hands segment to TCP
④ TCP verifies checksum, strips header, port number → delivers data to application

V.U.

TCP Connection

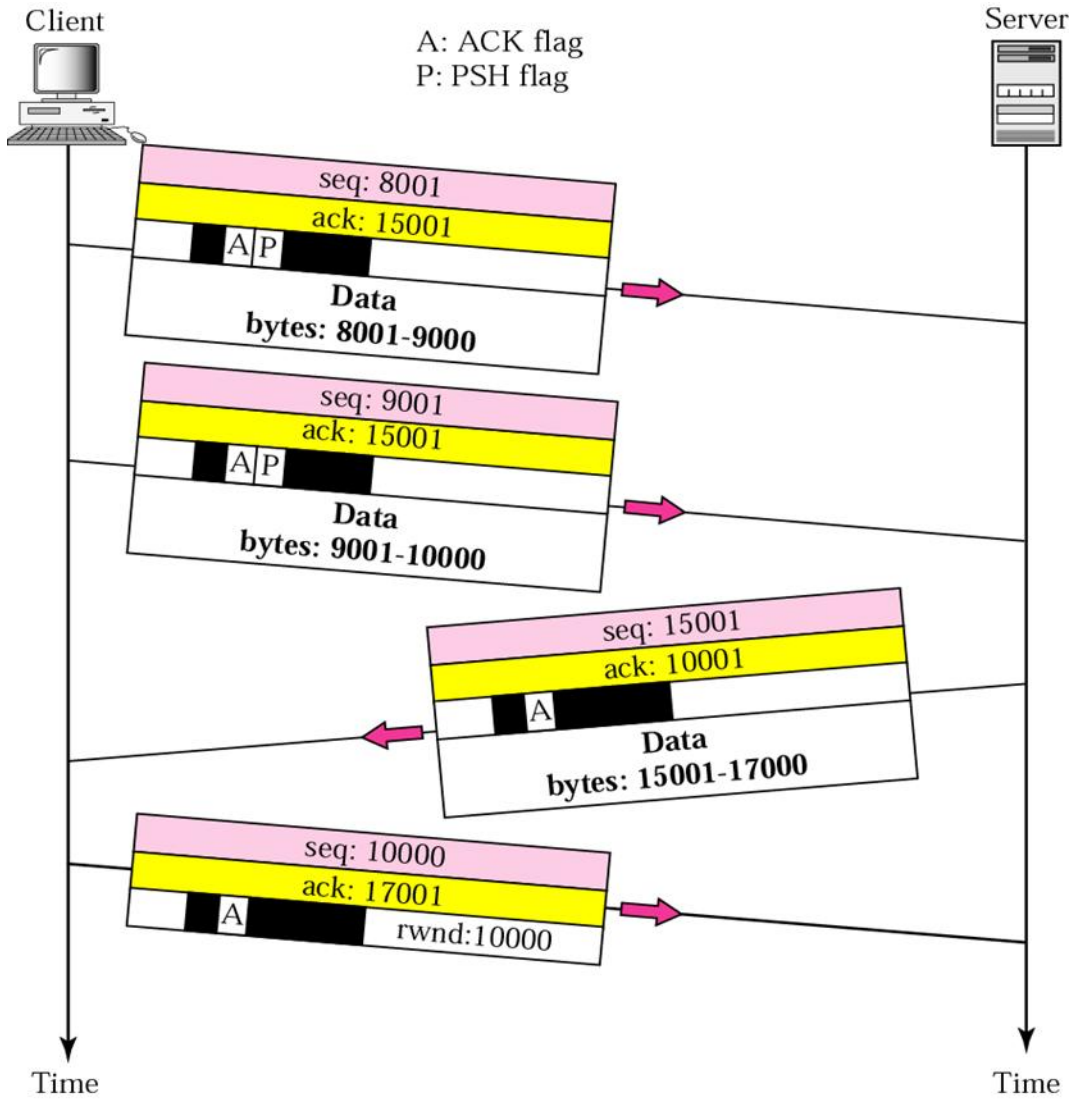
- TCP is a connection-oriented protocol.
- A connection-oriented transport protocol establishes a virtual path between the source and destination.
- All of the segments belonging to a message are then sent over this virtual path.
- A connection-oriented transmission requires three phases: connection establishment, data transfer, and connection termination.

Connection establishment using three-way handshaking

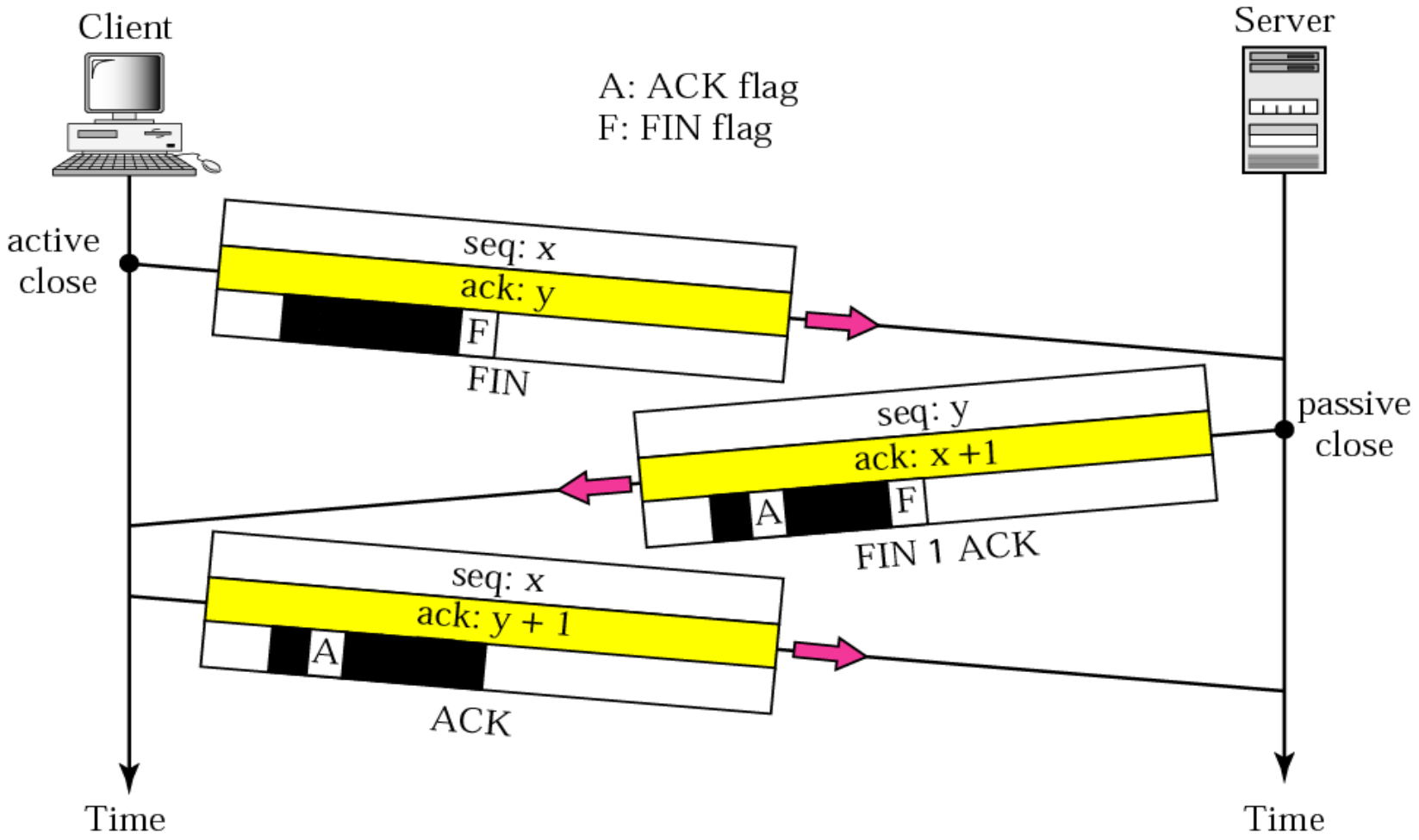


- A SYN segment cannot carry data, but it consumes one sequence number.
- A SYN + ACK segment cannot carry data, but does consume one sequence number
- An ACK segment, if carrying no data, consumes no sequence number

Data transfer

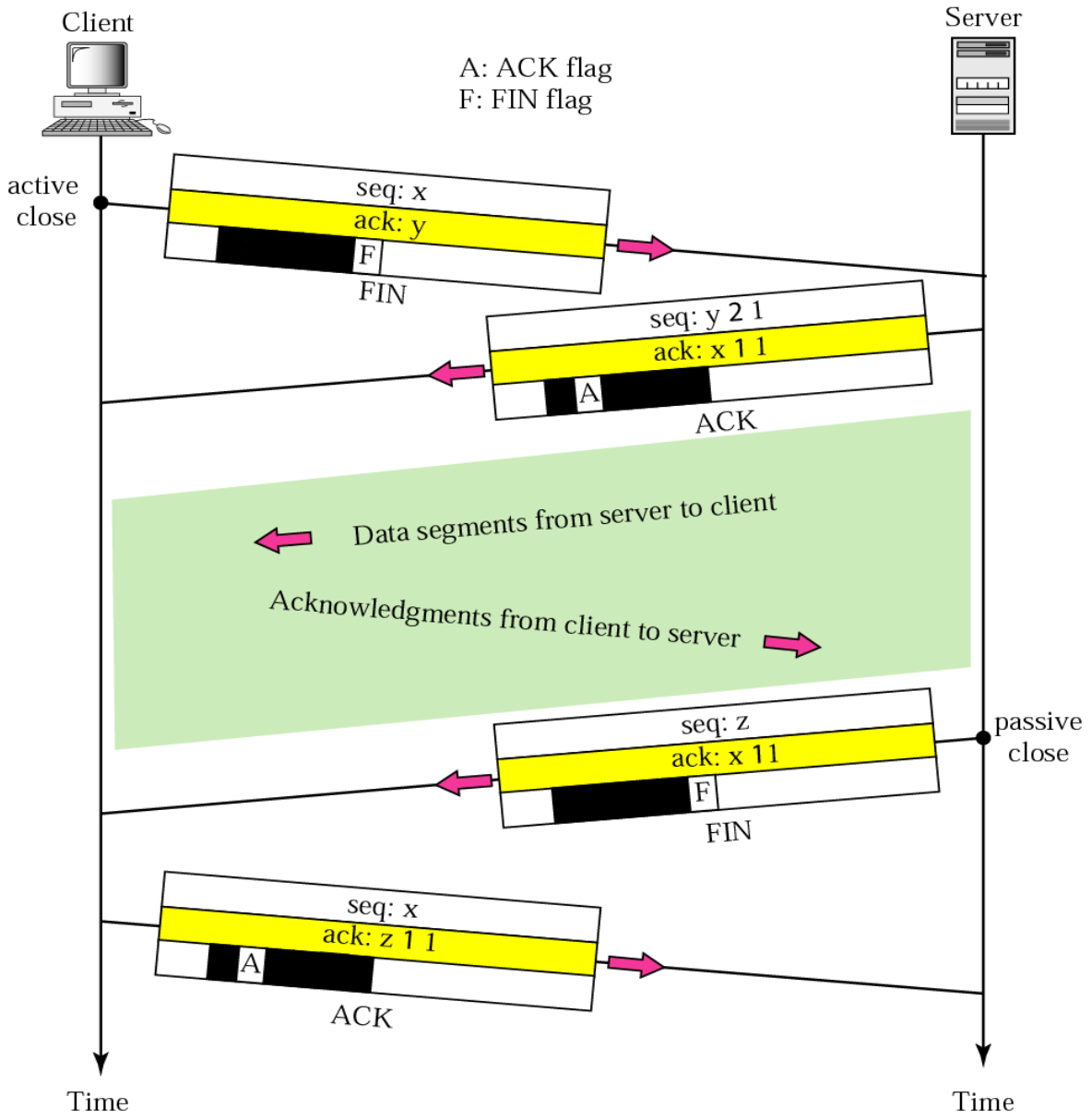


Connection termination using three-way handshaking



- The FIN segment consumes one sequence number if it does not carry data
- The FIN + ACK segment consumes one sequence number if it does not carry data.

Half close connection

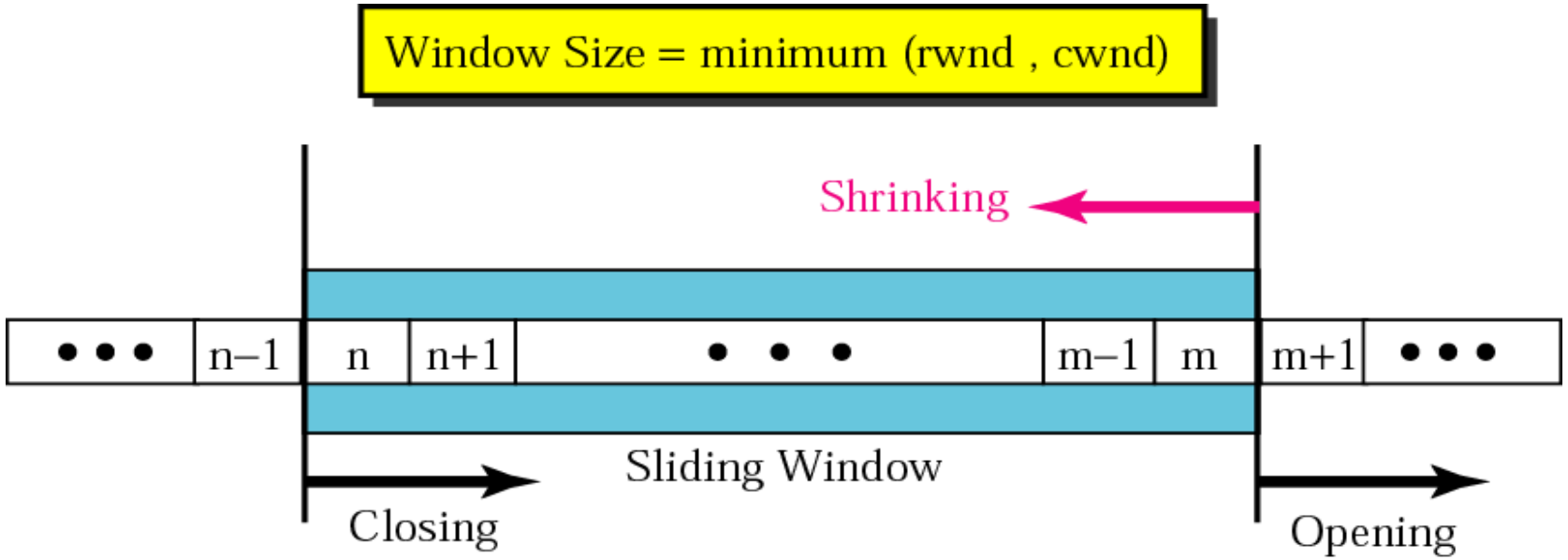


Flow Control

- Flow control regulates the amount of data a source can send before receiving an acknowledgment from the destination.
- TCP defines a window that is imposed on the buffer of data delivered from the application program.

Sliding Window

- A sliding window is used to make transmission more efficient as well as to control the flow of data so that the destination does not become overwhelmed with data.
- TCP's sliding windows are byte-oriented.
- The position of the left and right walls of the window decides the size of the window.
- Anything on the left of left wall indicates byte numbers sent and acknowledged, while everything on the other side of the right wall indicates sequence numbers outside the window
- Sequence numbers within the window are actually stored in the buffer for transmission



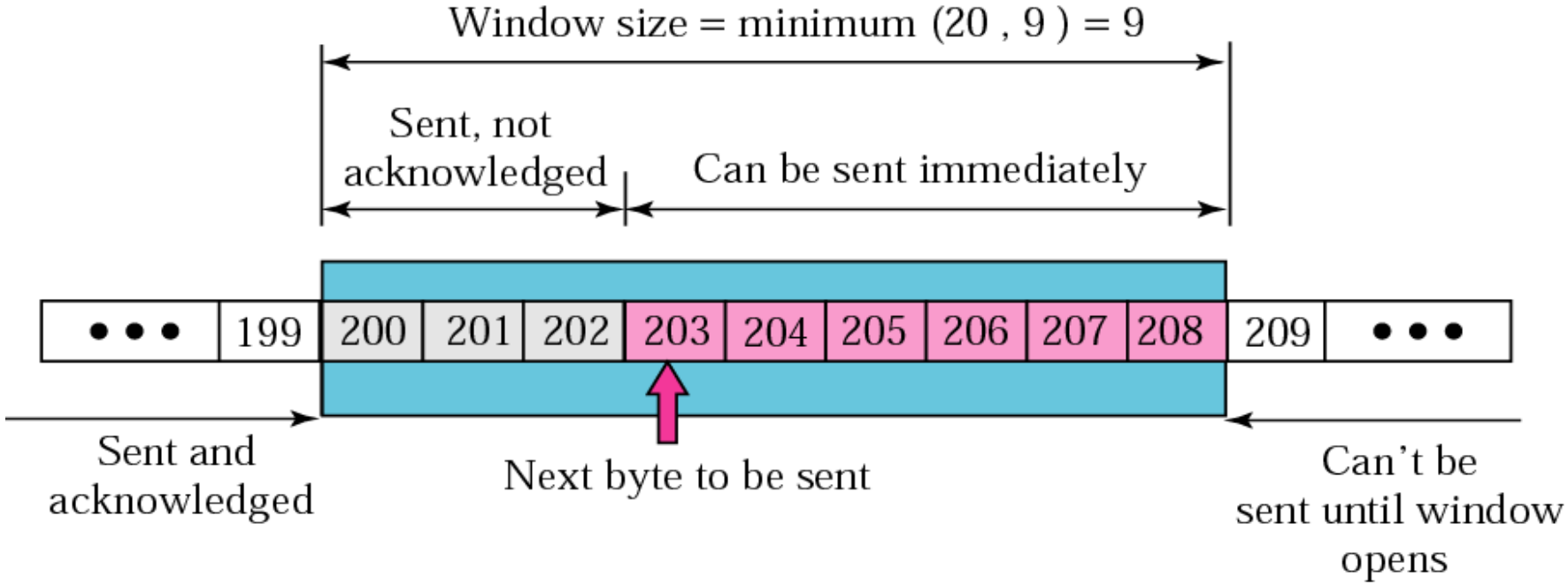
Example

- What is the value of the receiver window (rwnd) for host A if the receiver, host B, has a buffer size of 5,000 bytes and 1,000 bytes of received and unprocessed data?
- The value of rwnd = 5,000 − 1,000 = 4,000. Host B can receive only 4,000 bytes before its buffer overflows. Host B advertises this value in its next segment to A.

- What is the size of the window for host A if the value of rwnd is 3,000 bytes and the value of cwnd is 3,500 bytes?
- The size of the window is the smaller of rwnd and cwnd, which is 3,000 bytes.

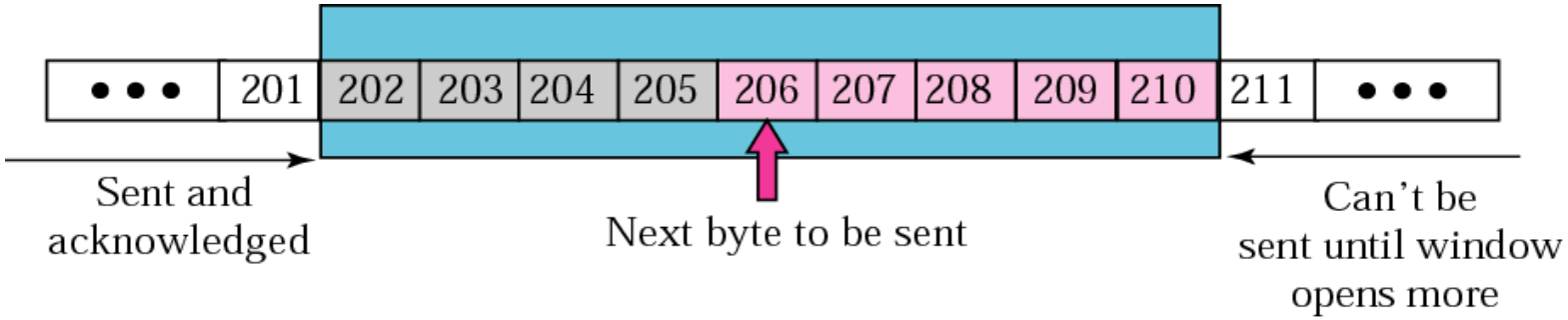
Example

- Figure below shows an unrealistic example of a sliding window. The sender has sent bytes up to 202. We assume that cwnd is 20 (in reality this value is thousands of bytes). The receiver has sent an acknowledgment number of 200 with an rwnd of 9 bytes (in reality this value is thousands of bytes). The size of the sender window is the minimum of rwnd and cwnd or 9 bytes. Bytes 200 to 202 are sent, but not acknowledged. Bytes 203 to 208 can be sent without worrying about acknowledgment. Bytes 209 and above cannot be sent.



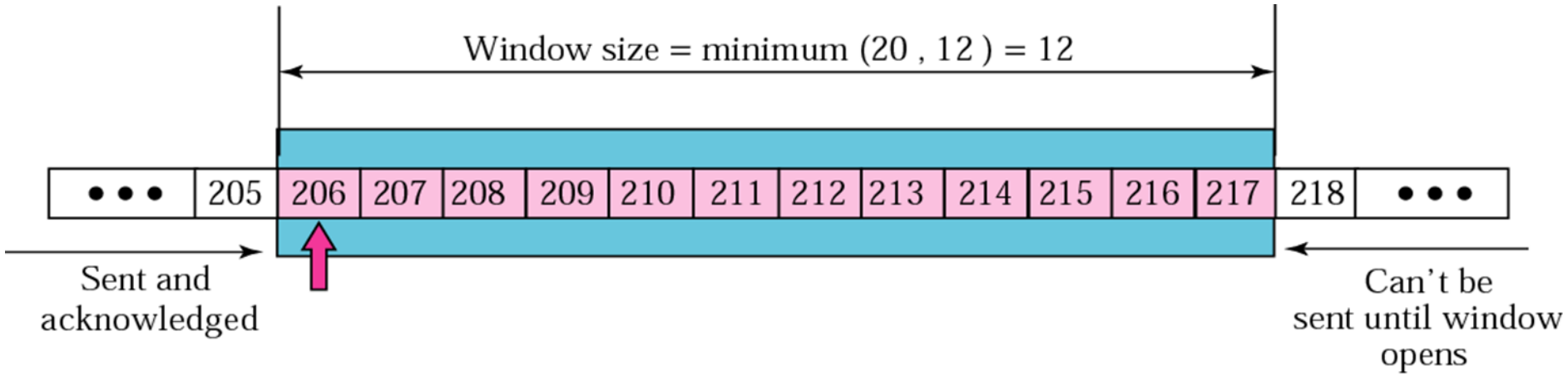
Example

- In Figure on previous slide, the server receives a packet with an acknowledgment value of 202 and an rwnd of 9. The host has already sent bytes 203, 204, and 205. The value of cwnd is still 20. Show the new window.



Example

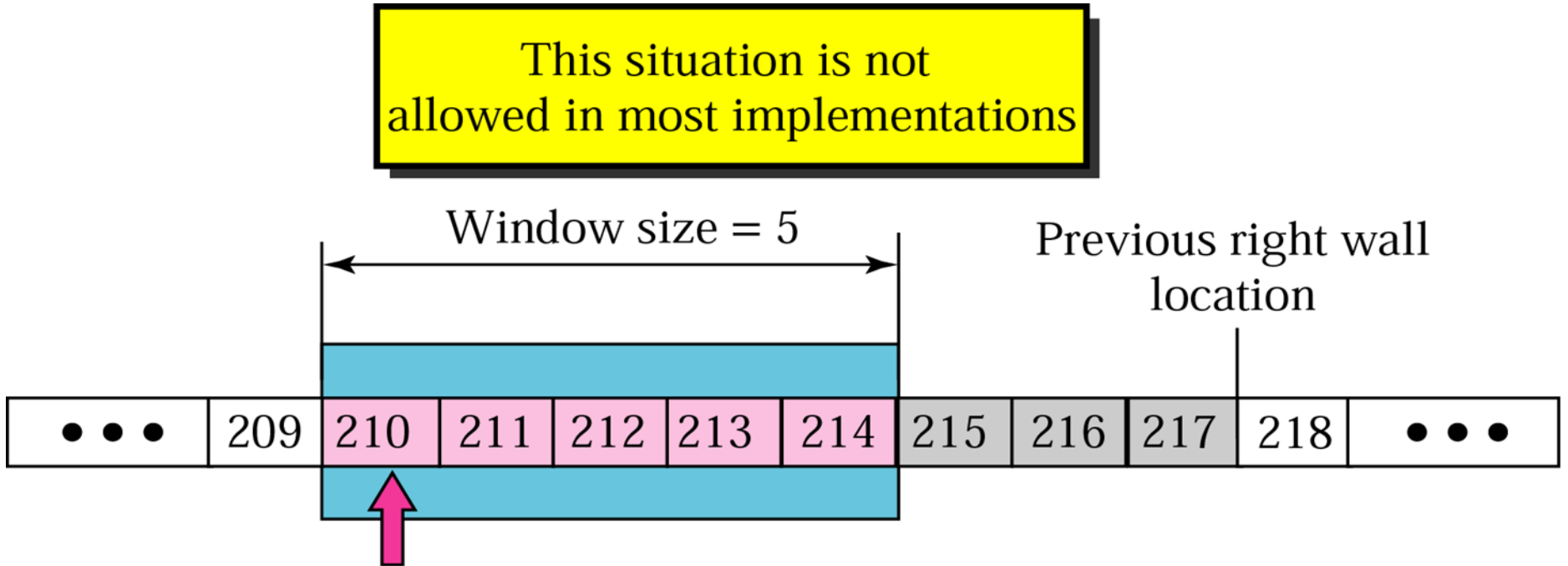
- In previous figure, the sender receives a packet with an acknowledgment value of 206 and an rwnd of 12. The host has not sent any new bytes. The value of cwnd is still 20. Show the new window.
- The value of rwnd is less than cwnd, so the size of the window is 12. Note that the window has been opened from the right by 7 and closed from the left by 4; the size of the window has increased.



Example

- In previous figure, the host receives a packet with an acknowledgment value of 210 and an rwnd of 5. The host has sent bytes 206, 207, 208, and 209. The value of cwnd is still 20. Show the new window.

- The value of rwnd is less than cwnd, so the size of the window is 5. Note that this is a case not allowed by most implementations. Although the sender has not sent bytes 215 to 217, the receiver does not know this



Example

- How can the receiver avoid shrinking the window in the previous example?
- The receiver needs to keep track of the last acknowledgment number and the last rwnd. If we add the acknowledgment number to rwnd we get the byte number following the right wall. If we want to prevent the right wall from moving to the left (shrinking), we must always have the following relationship.

$$new\ ack + new\ rwnd \geq last\ ack + last\ rwnd$$
$$or$$
$$new\ rwnd \geq (last\ ack + last\ rwnd) - new\ ack$$

- **To avoid shrinking the sender window, the receiver must wait until more space is available in its buffer.**

Summary of TCP's sliding window

- The size of the window is the lesser of rwnd and cwnd.
- The source does not have to send a full window's worth of data.
- The window can be opened or closed by the receiver, but should not be shrunk.
- The destination can send an acknowledgment at any time as long as it does not result in a shrinking window.
- The receiver can temporarily shut down the window; the sender, however, can always send a segment of one byte after the window is shut down.

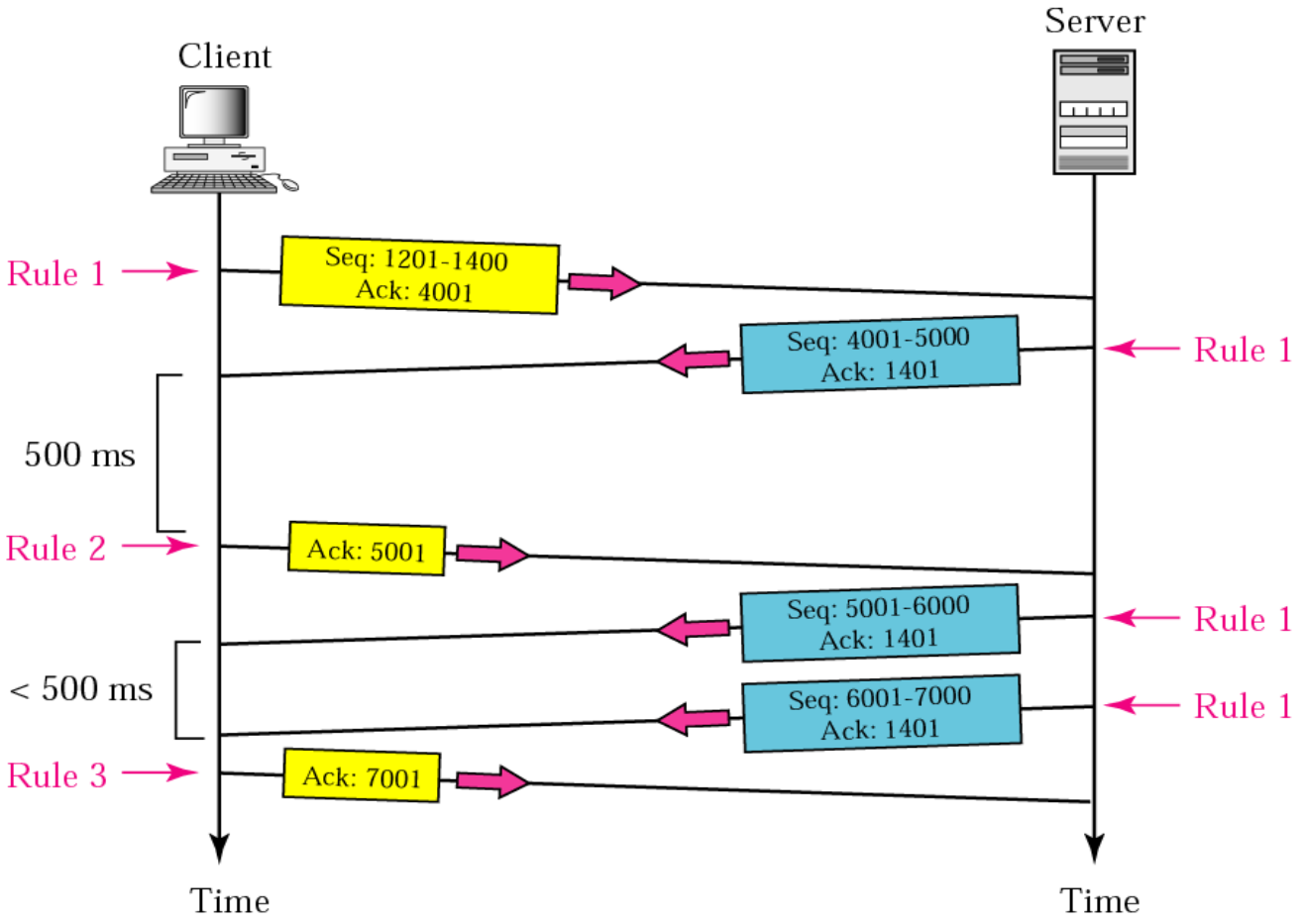
Error Control

- TCP provides reliability using error control, which detects corrupted, lost, out-of-order, and duplicated segments.
- Error control in TCP is achieved through the use of the checksum, acknowledgments, and timeouts.

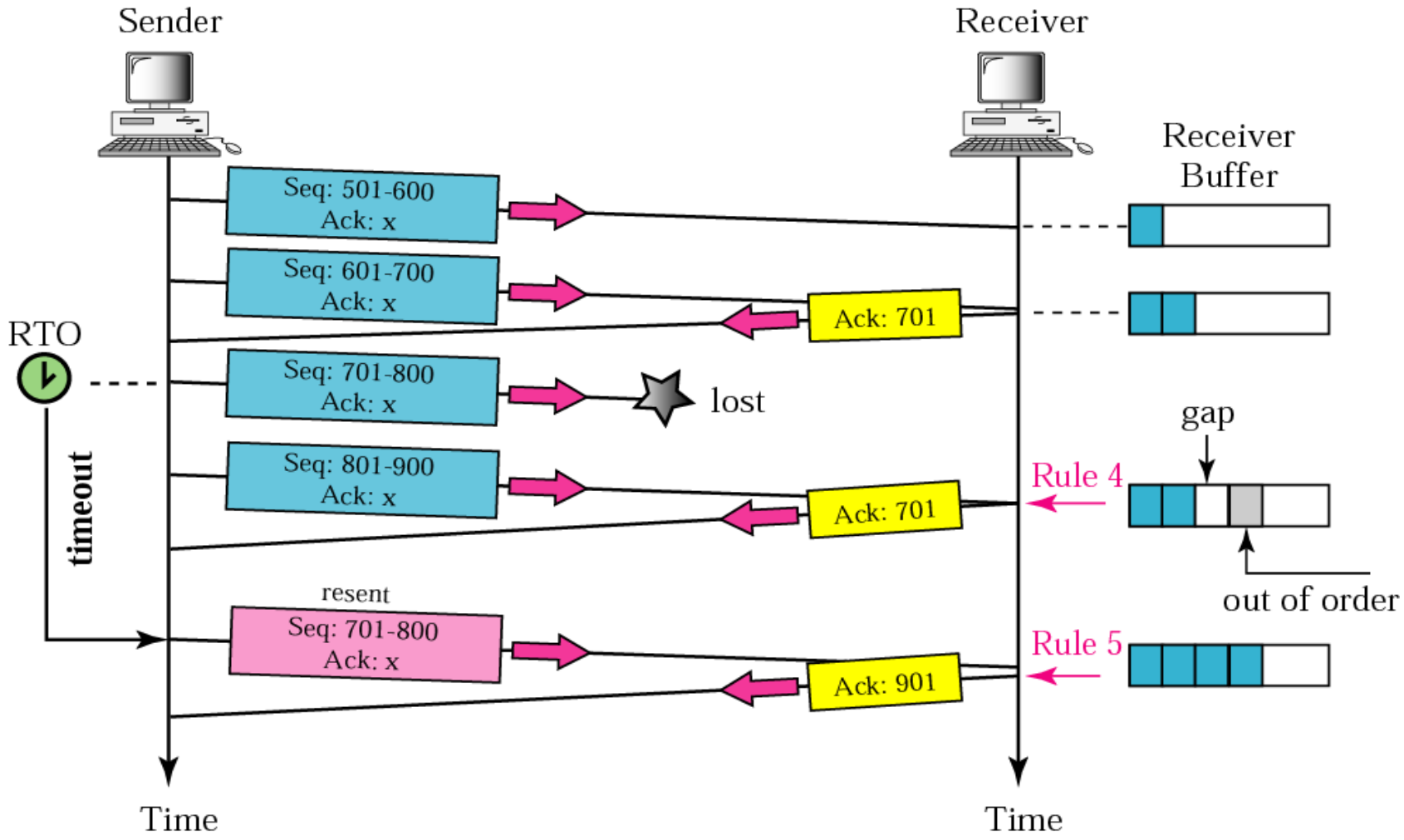
Common guidelines

- ACK segments do not consume sequence numbers and are not acknowledged.
- In modern implementations, a retransmission occurs if the retransmission timer expires or three duplicate ACK segments have arrived.
- No retransmission timer is set for an ACK segment.
- Data may arrive out of order and be temporarily stored by the receiving TCP, but TCP guarantees that no out-of-order segment is delivered to the process.

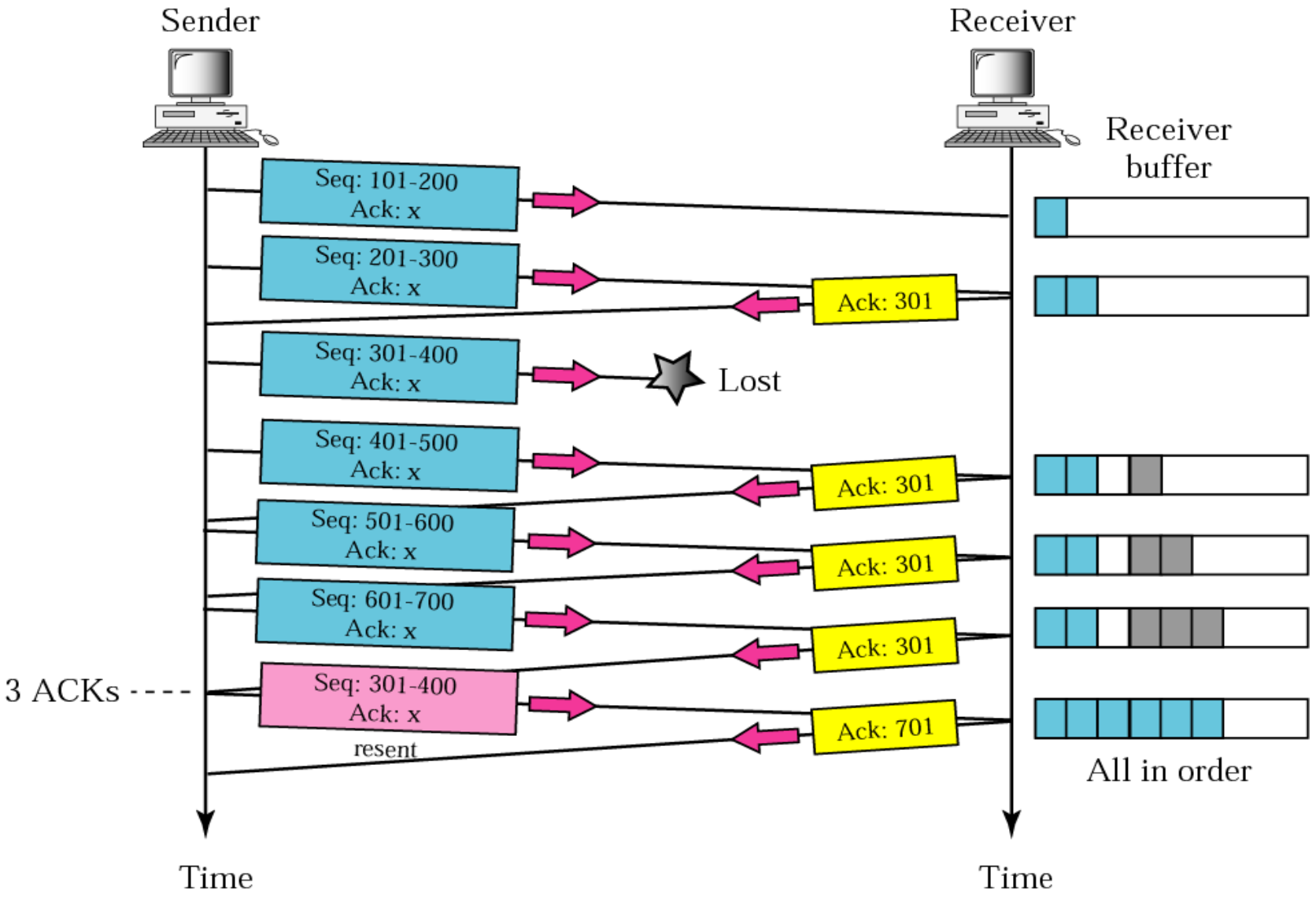
Normal Operation



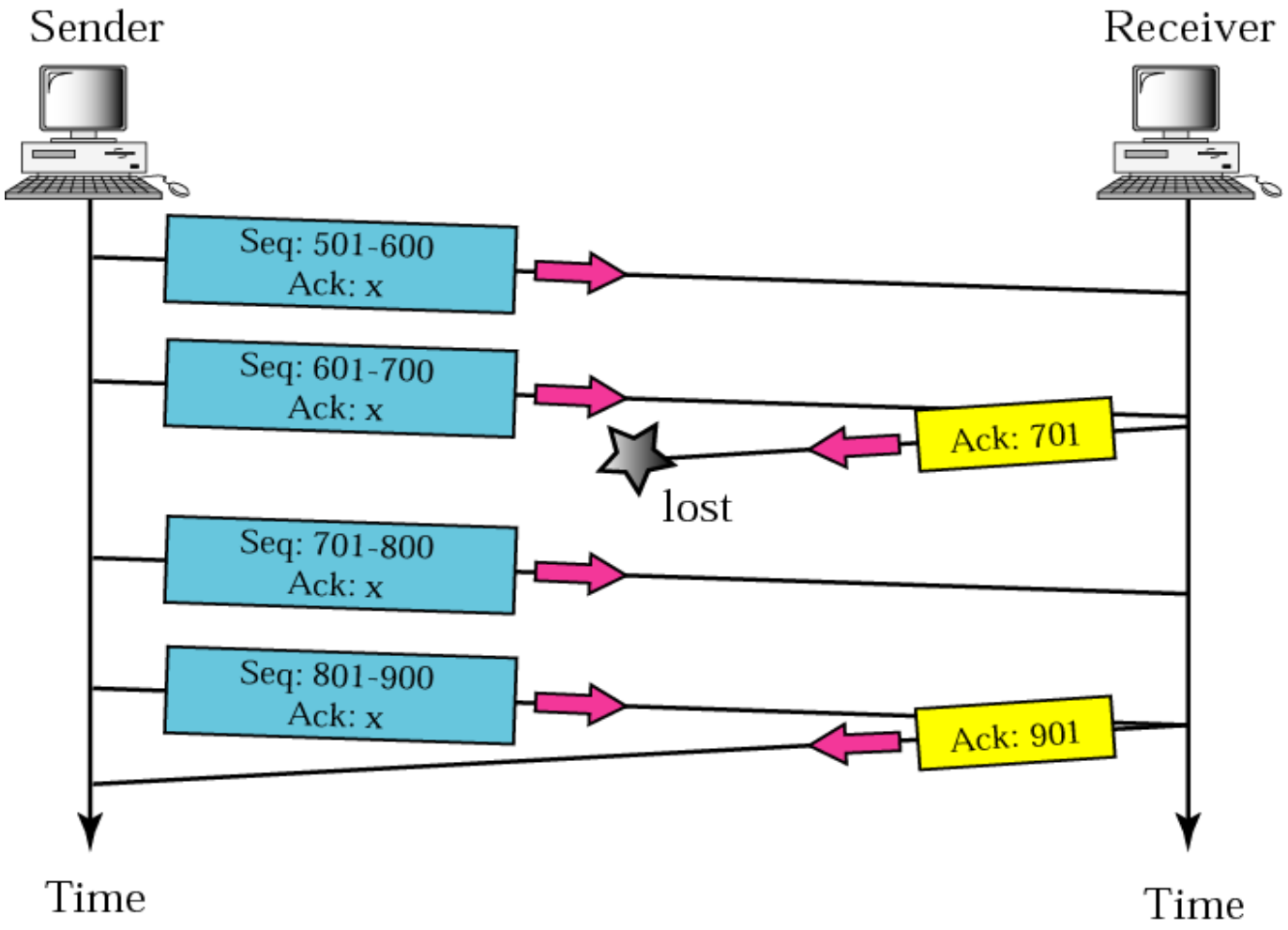
Lost segment



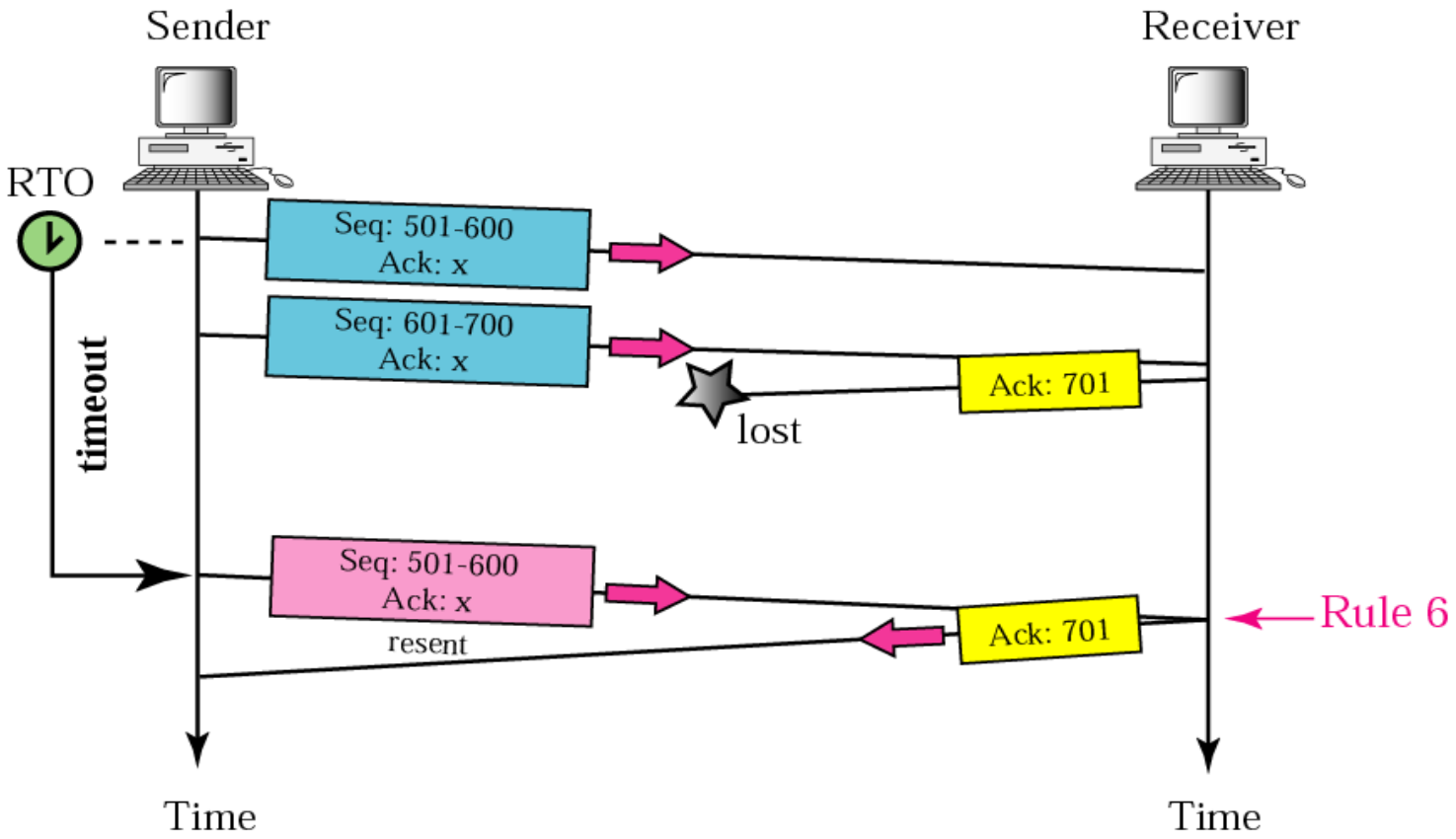
Fast Retransmission



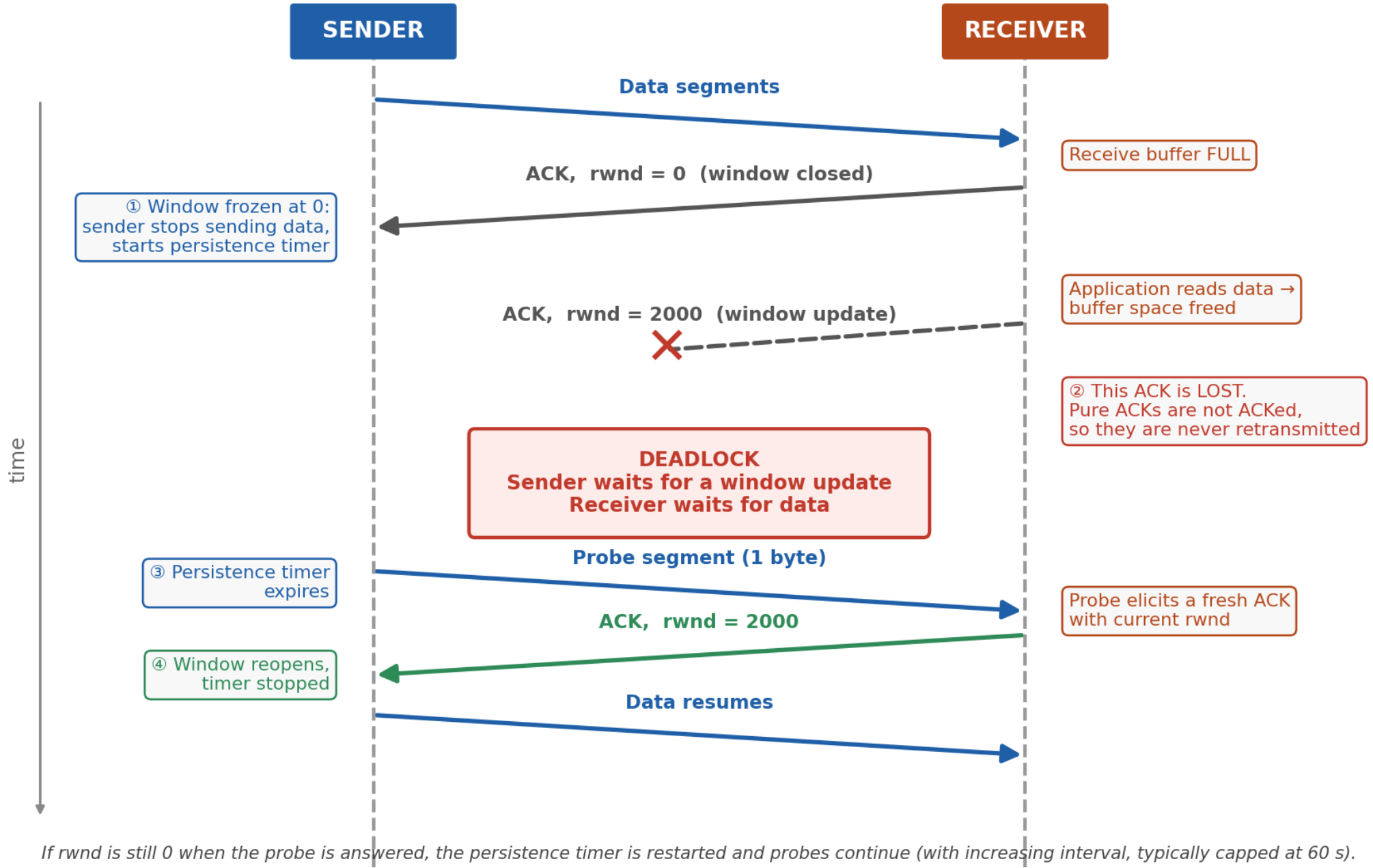
Lost acknowledgment



Lost acknowledgment corrected by resending a segment



Lost acknowledgment may create deadlock

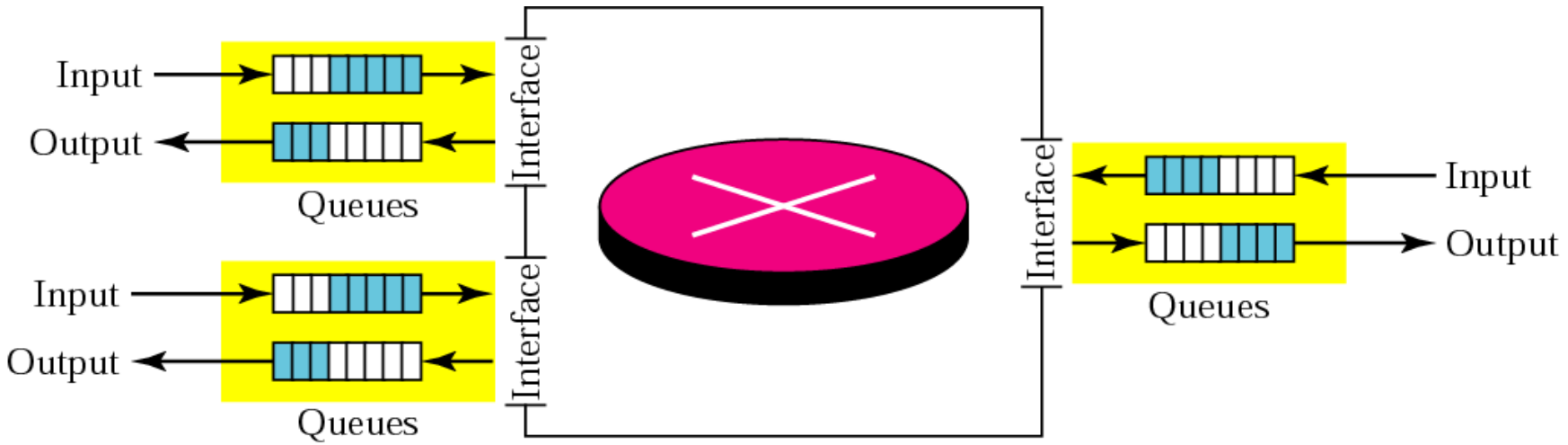


If rwnd is still 0 when the probe is answered, the persistence timer is restarted and probes continue (with increasing interval, typically capped at 60 s).

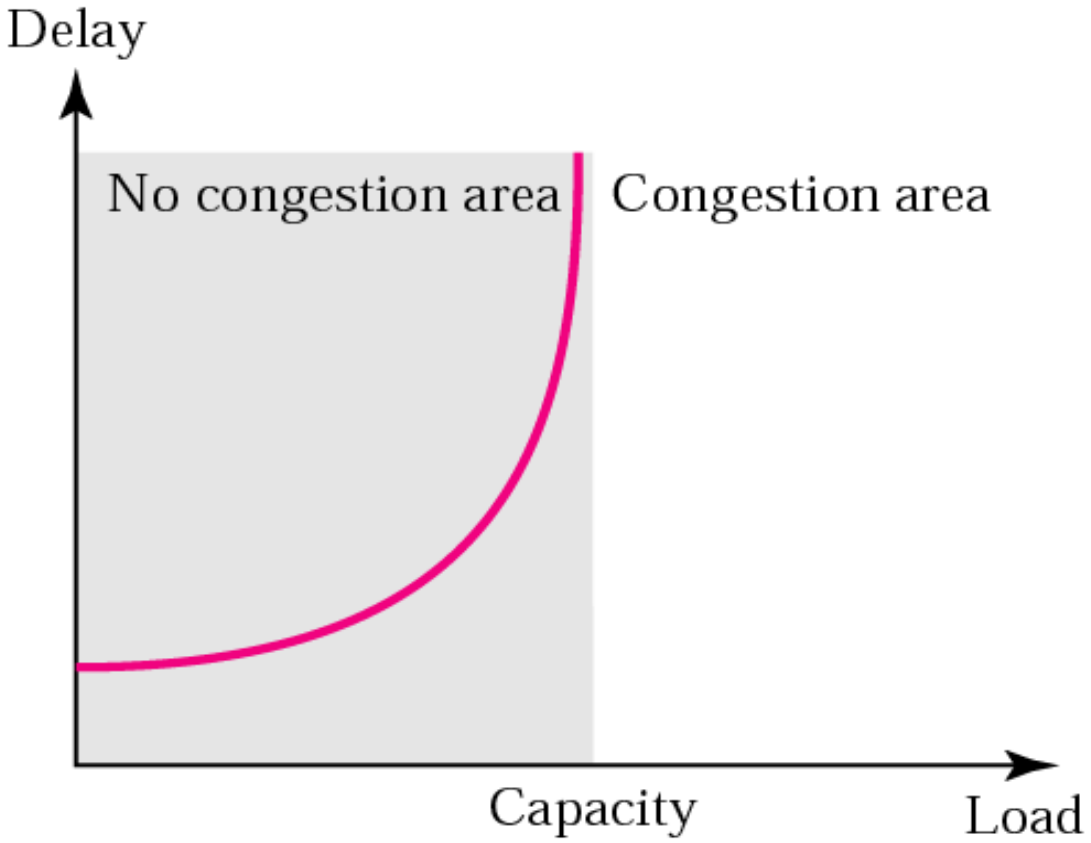
Congestion Control

- Congestion control refers to the mechanisms and techniques to keep the load below the capacity.

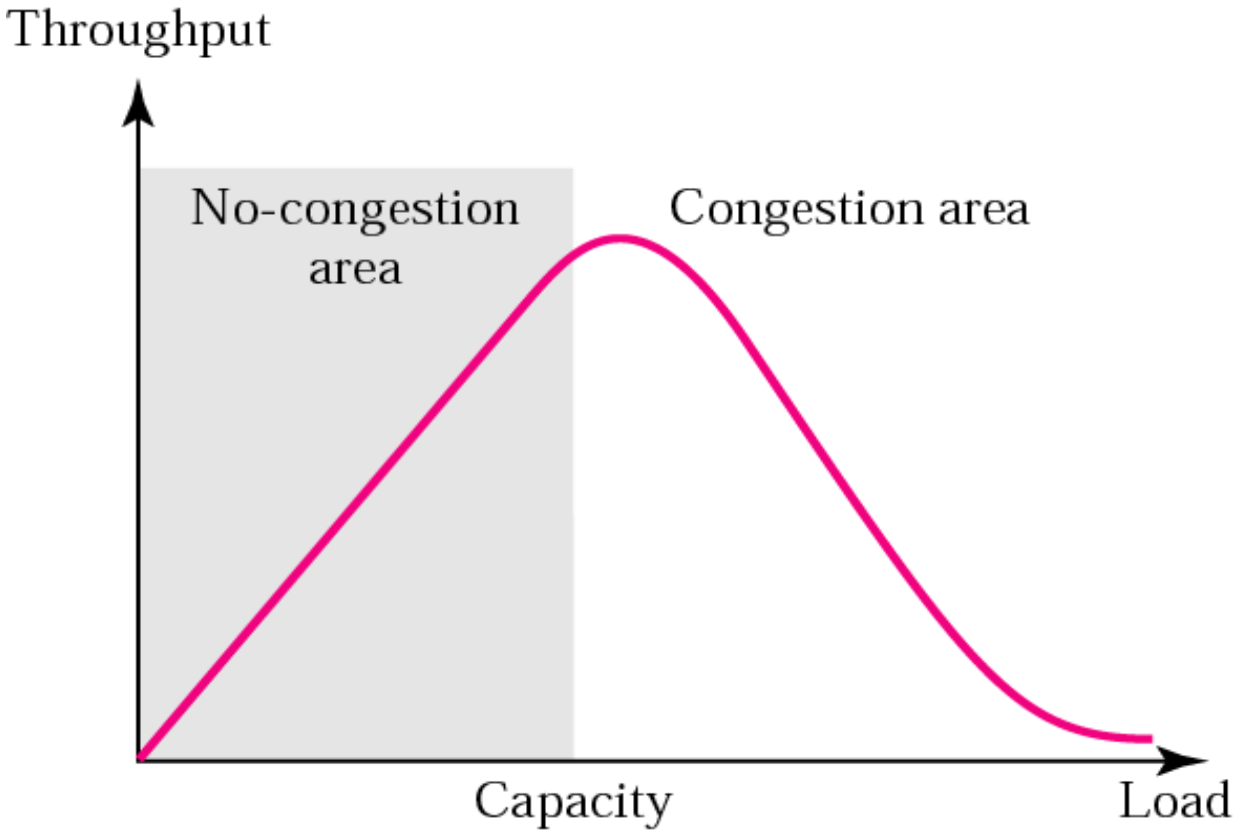
Router Queues



Network performance under congestion

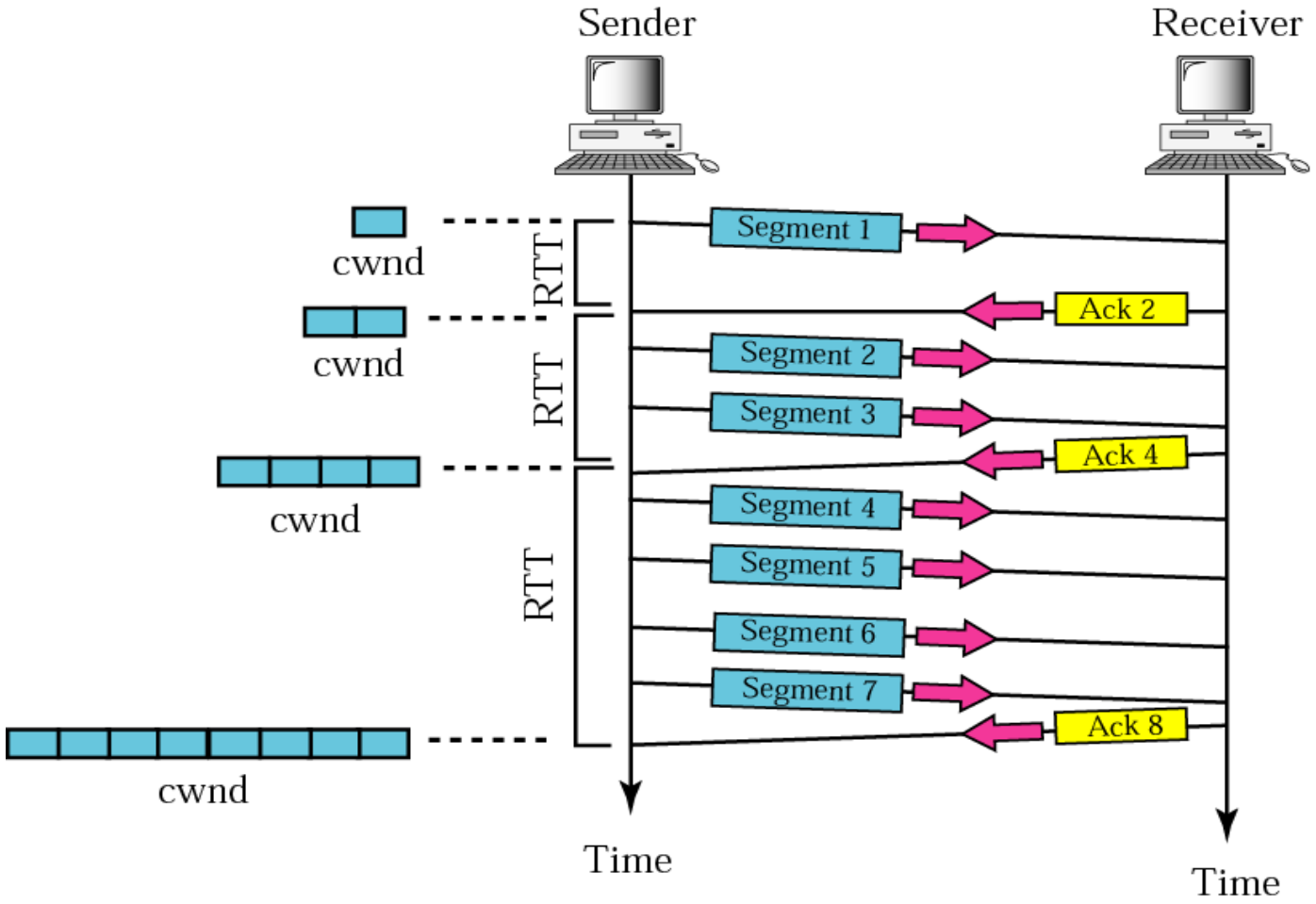


Packet delay and network load



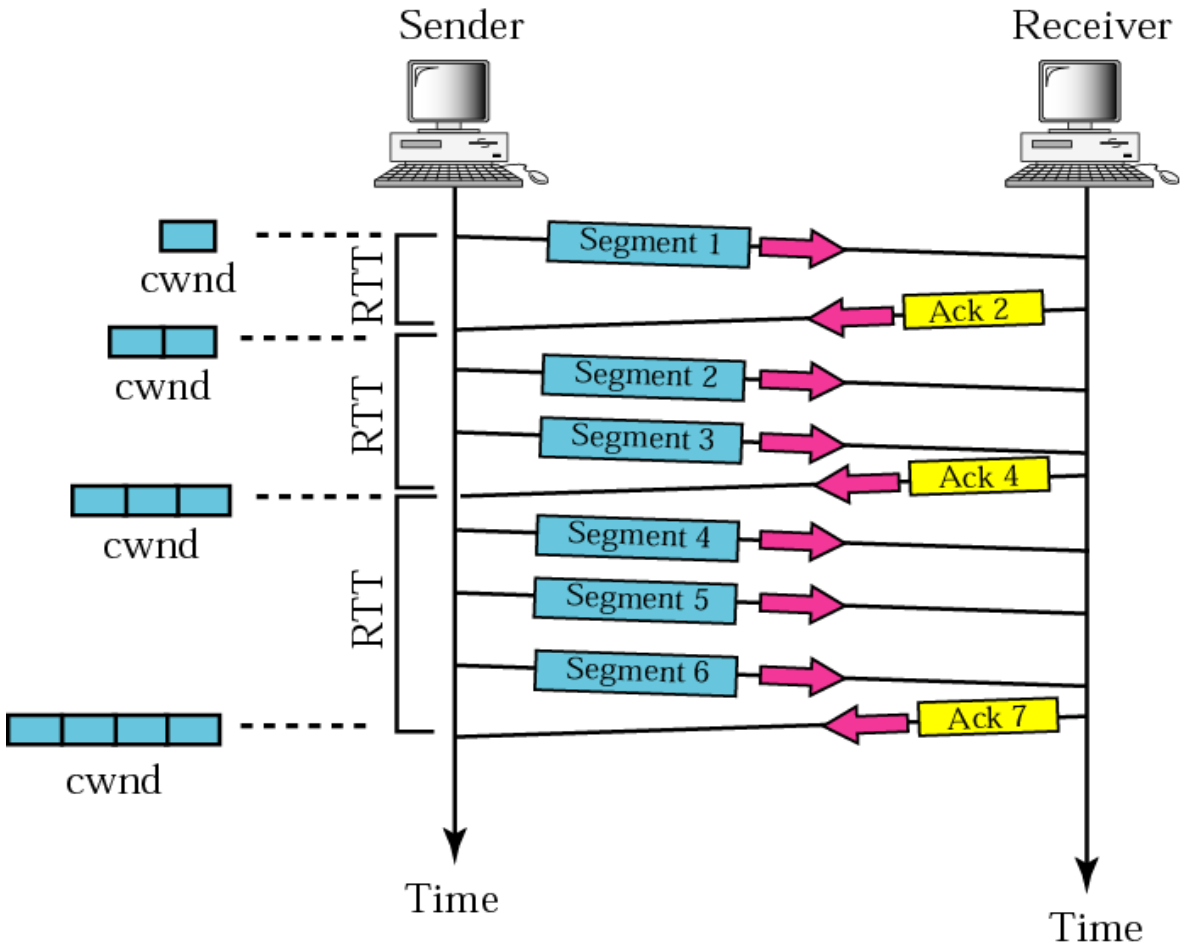
Throughput versus network load

Slow start, exponential increase



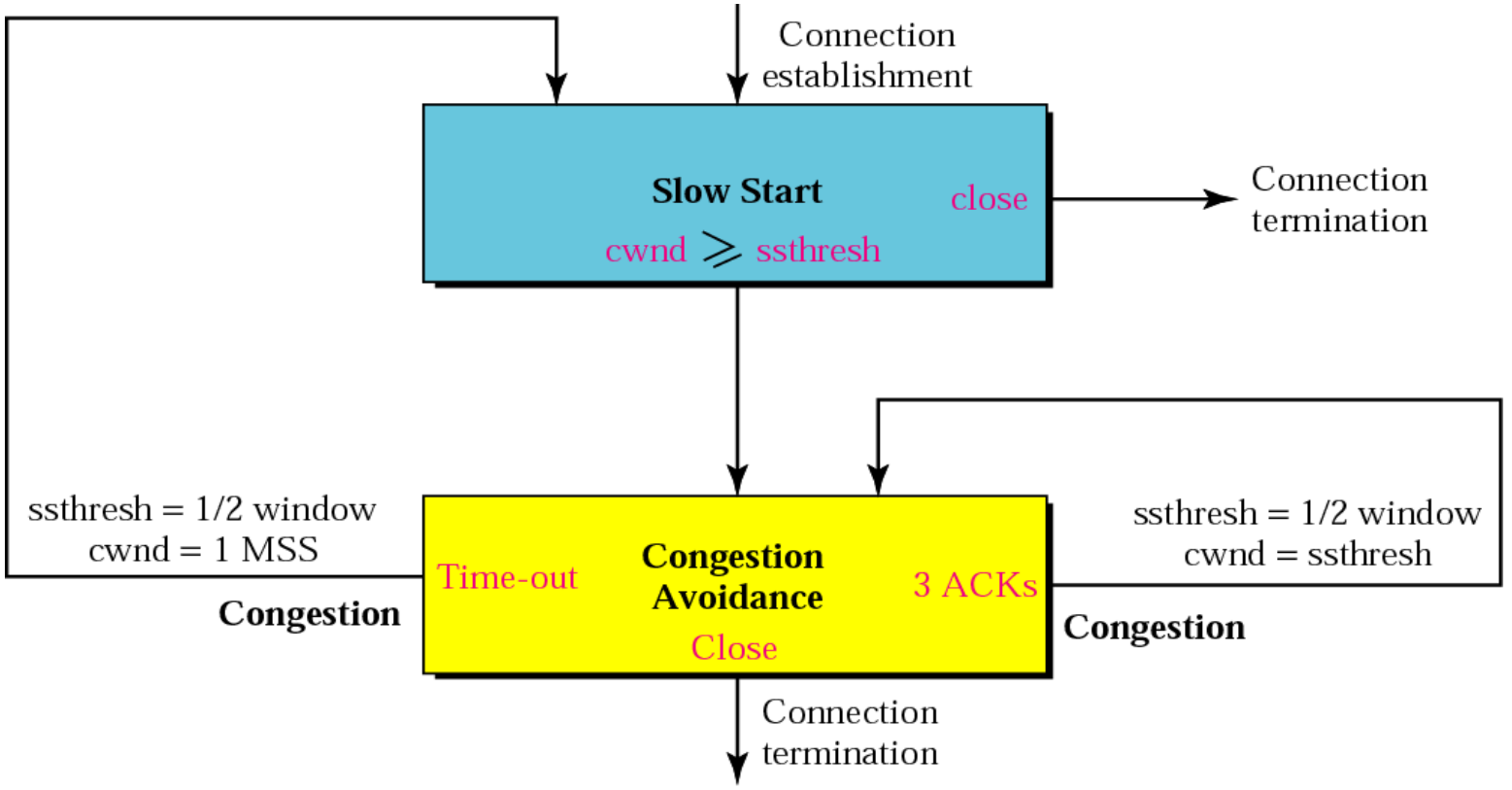
In the slow start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

Congestion avoidance, additive increase



In the congestion avoidance algorithm the size of the congestion window increases additively until congestion is detected.

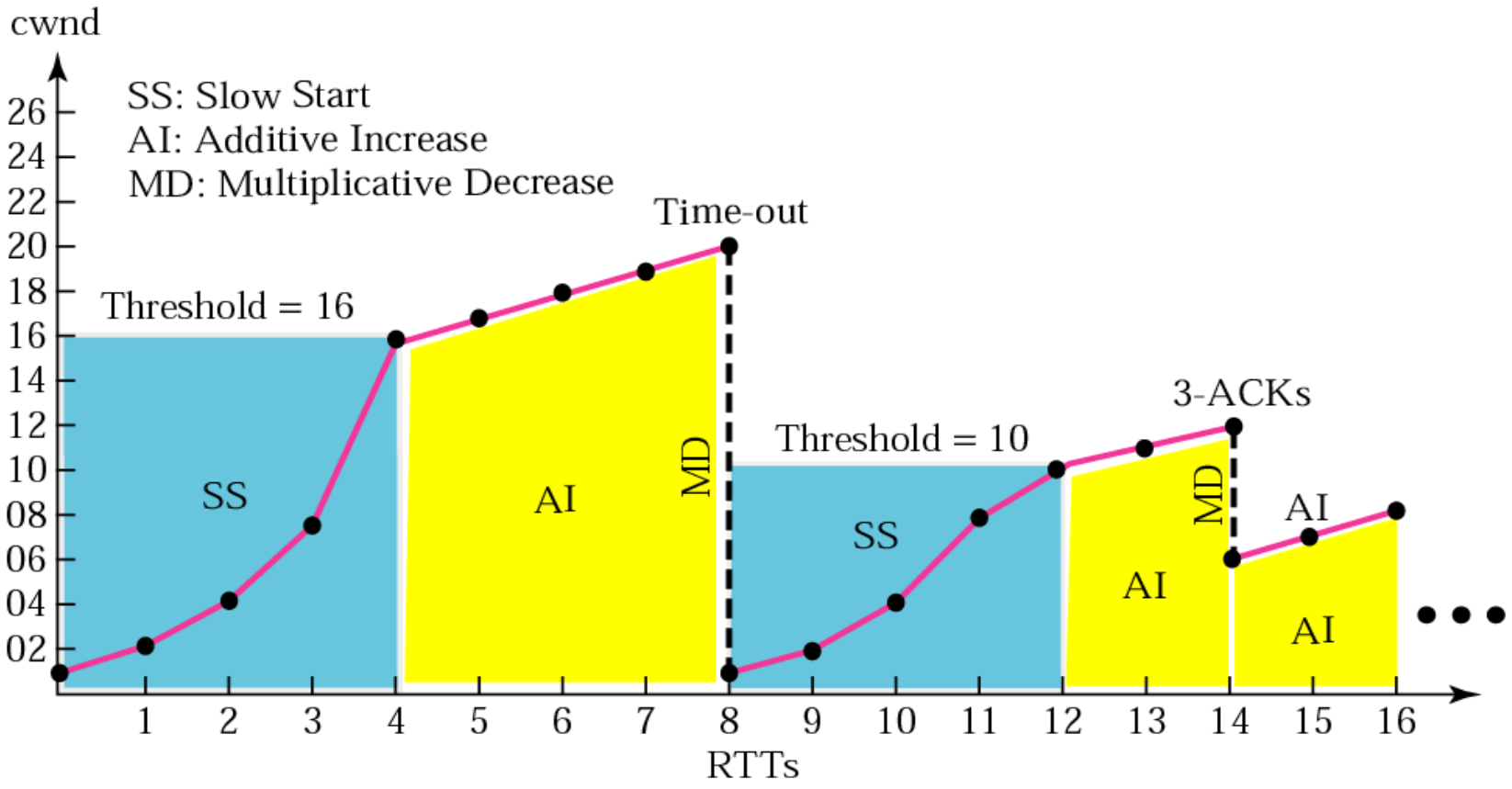
TCP congestion policy summary



Most implementations react differently to congestion detection:

- ❑ **If detection is by time-out, a new slow start phase starts.**
- ❑ **If detection is by three ACKs, a new congestion avoidance phase starts.**

TCP congestion window example



TCP Timers

- To perform its operation smoothly, most TCP implementations use at least four timers.
  - **Retransmission Timer**: Started when a segment is sent. If an acknowledgment is not received before it expires, the segment is assumed lost and is retransmitted. Its value (RTO) is calculated dynamically from measured round-trip times.
  - **Persistence Timer** : Started when the sender receives a zero window advertisement (rwnd = 0). When it expires, the sender transmits a 1-byte probe segment to find out whether the window has reopened, which prevents deadlock if the window-update ACK is lost.
  - **Keepalive Timer**: Used to detect and close connections to a client that has crashed or gone silent. If no segment arrives from the other side for a long idle period (commonly 2 hours), the server sends probe segments, and the connection is terminated if no response comes after a set number of probes.
  - **TIME-WAIT Timer**: Started when a connection is closed by an active close and the final ACK is sent. The connection stays in the TIME-WAIT state for 2 × MSL (Maximum Segment Lifetime), so that the final ACK can be retransmitted if lost and old duplicate segments from the connection disappear from the network.

TCP Retransmission Timeout (RTO) determination

- As TCP hosts are separated by huge network, the RTT measurements across different packets might fluctuate heavily.
- Rather than estimating anything based on instantaneous measurement only, the RTTs across multiple measurements are used.
- **Measured RTT (RTT_M)**
  - Time to send a segment and receive an acknowledgment for it
  - Remember that the segments and their acknowledgments do not have a one-to-one relationship
  - Only one RTT measurement can be in progress at any time.
- **Smoothed RTT (RTT_S)**
  - Since RTTM of various packets might vary considerably, RTT_S is used

Initially	→	**No value**
After first measurement	→	$RTT_S = RTT_M$
After each measurement	→	$RTT_S = (1 - \alpha)\, RTT_S + \alpha \times RTT_M$

  - The value of α is normally set to 1/8

TCP Retransmission Timeout (RTO) determination

- **RTT Deviation (RTT_D)**
  - Since RTTM of various packets might vary considerably, RTTS is used

Initially	→	**No value**
After first measurement	→	$RTT_D = RTT_M / 2$
After each measurement	→	$RTT_D = (1 - \beta)\, RTT_D + \beta \times \lvert RTT_S - RTT_M \rvert$

  - The value of β is normally set to ¼
- **Retransmission Time-out (RTO)**
  - Since RTT_M of various packets might vary considerably, RTTS is used

Original	→	**Initial value**
After any measurement	→	$RTO = RTT_S + 4 \times RTT_D$

TCP Retransmission Timeout (RTO) determination - Example

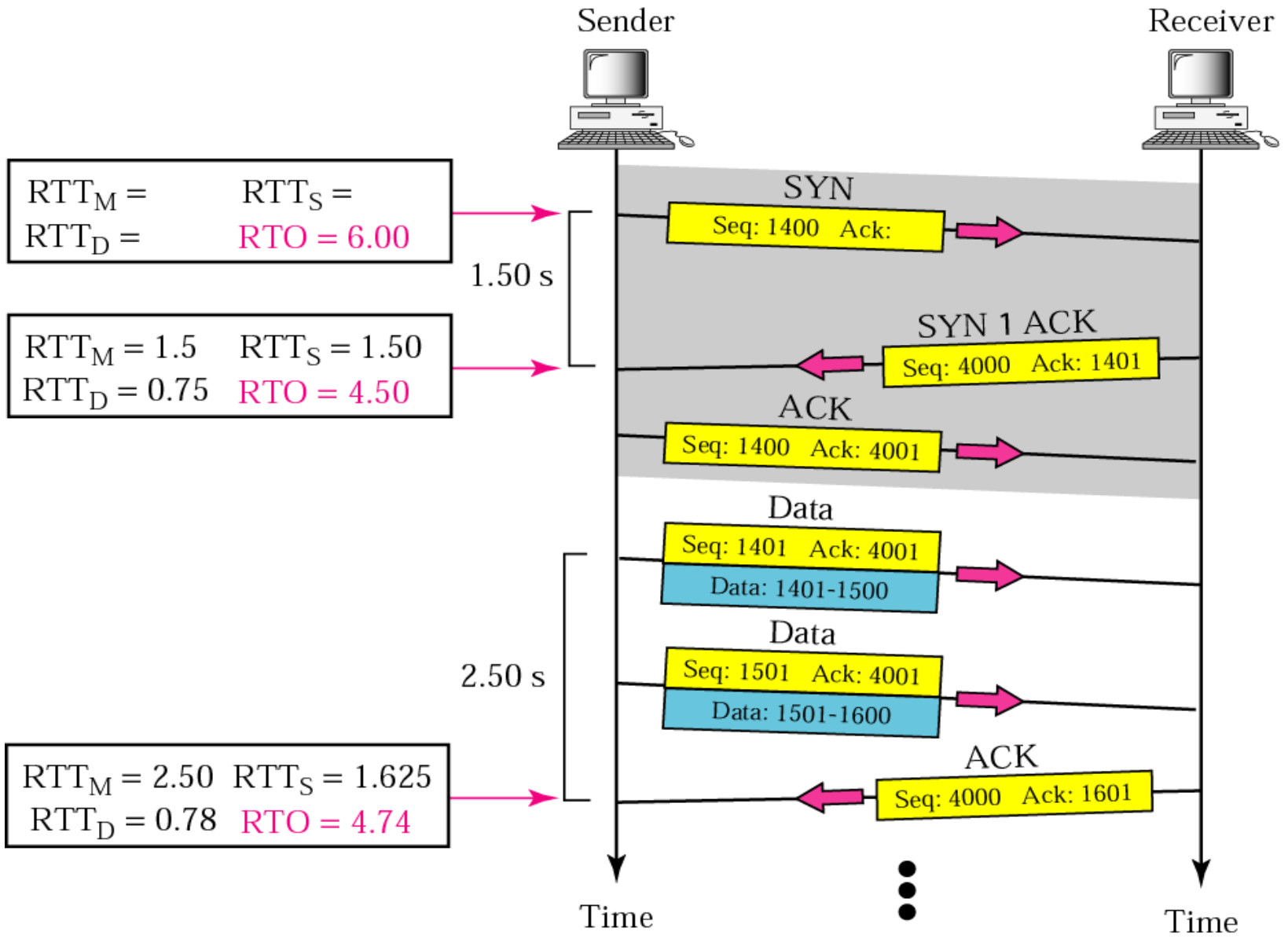
- 1. When the SYN segment is sent, RTTM, RTTS, and RTTD have no value. RTO is set to 6.00 seconds.
- 2. When the SYN+ACK segment arrives, RTTM is measured at 1.5 seconds. The following shows the value of these variables at this moment:

$$RTT_M = 1.5 \qquad RTT_S = 1.5$$
$$RTT_D = 1.5 / 2 = 0.75 \qquad RTO = 1.5 + 4 \,.\, 0.75 = 4.5$$

- 3. When the first data segment is sent, a new RTT measurement starts. Note that the sender does not start an RTT measurement when it sends the ACK segment, because it does not consume a sequence number and there is no timeout. No RTT measurement starts for the second data segment because a measurement is already in progress.

$$RTT_M = 2.5$$
$$RTT_S = 7/8\ (1.5) + 1/8\ (2.5) = 1.625$$
$$RTT_D = 3/4\ (7.5) + 1/4\ \ |1.625 - 2.5| \ = 0.78$$
$$RTO = 1.625 + 4\ (0.78) = 4.74$$

TCP Retransmission Timeout (RTO) determination - Example



Karn's Algorithm

- If a segment is not acknowledged during transmission, and retransmitted, the ack received can not be used for RTT measurement
- The sender will have to measure the RTT from the retransmitted segment in these cases
- Do not consider the round-trip time of a retransmitted segment in the calculation of RTTs
- Do not update the value of RTTs until you send a segment and receive an acknowledgment without the need for retransmission

- **Exponential Backoff**
- In most TCP implementations, the value of RTO is doubled for each retransmission

Karn’s Algorithm - Example

- There is retransmission, and Karn’s algorithm is applied. The first segment in the figure is sent but lost. The RTO timer expires after 4.74 seconds. The segment is retransmitted, and the timer is set to 9.48 seconds, twice the previous RTO. This time, an ACK is received before the timeout. We wait until we send a new segment and receive the ACK for it before recalculating the RTO (Karn’s algorithm).

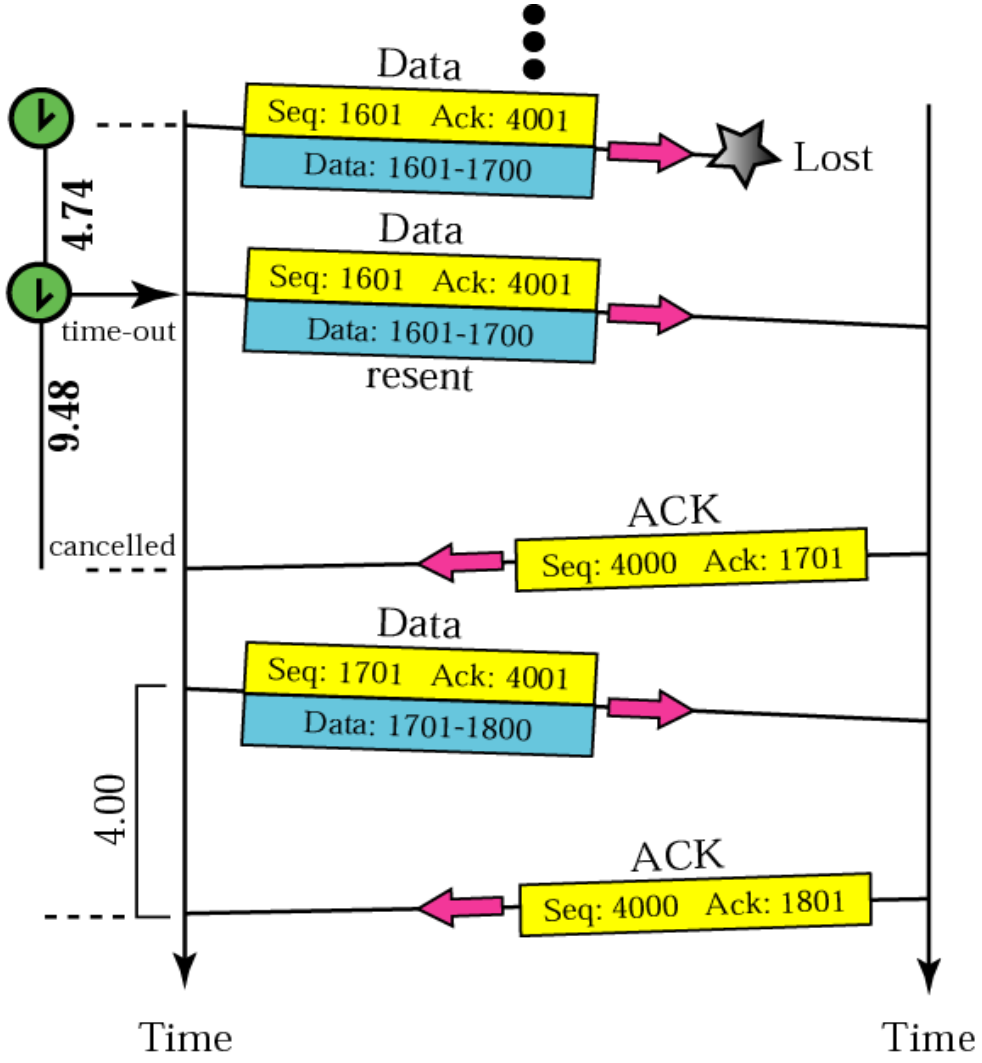
Karn's Algorithm - Example

$RTT_M = 2.50$ $RTT_S = 1.625$
$RTT_D = 0.78$ RTO = 4.74
Values from previous example

RTO = 2 x 4.74 = 9.48
Exponential Backoff of RTO

RTO = 2 x 4.74 = 9.48
No change, Karn's algorithm

$RTT_M = 4.00$ $RTT_S = 1.92$
$RTT_D = 1.105$ RTO = 6.34
New values based on new RTT_M



Persistent Timer

- The receiver TCP can send a zero window advertisement
- The sender will freeze the sender window state and stop transmitting until the receiver sends ack announcing a nonzero window size
- This ack can be lost, and ACKs are not acknowledged
- The sender is waiting for ack and the receiver is waiting for the sender to send data segments
- It leads to deadlock
- To correct the deadlock, a persistent timer is used
- When the sending TCP receives an acknowledgment with a window size of zero, it starts a persistence timer
- On timeout, the sending TCP sends a special segment called a probe
- The probe causes the receiving TCP to resend the acknowledgment

End of Chapter 6

Questions?